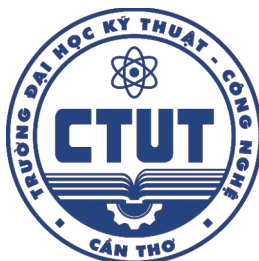


TRƯỜNG ĐẠI HỌC KỸ THUẬT - CÔNG NGHỆ CẦN THƠ
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

Nghành: Kỹ Thuật Phần Mềm

**XÂY DỰNG HỆ THỐNG DẠY VÀ HỌC LẬP TRÌNH
TRỰC TUYẾN**

LÊ THÀNH LỘC

Cần Thơ, năm 2026

TRƯỜNG ĐẠI HỌC KỸ THUẬT - CÔNG NGHỆ CẦN THƠ
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

Nghành: Kỹ Thuật Phần Mềm

**XÂY DỰNG HỆ THỐNG DẠY VÀ HỌC LẬP TRÌNH
TRỰC TUYẾN**

Cán bộ hướng dẫn:

ThS. LÊ HOÀNG MINH

Sinh viên thực hiện:

LÊ THÀNH LỘC

MSSV: KTPM2211011

Cần Thơ, năm 2026

Đồ án tốt nghiệp
Ngành Kỹ thuật phần mềm
XÂY DỰNG HỆ THỐNG DẠY VÀ HỌC LẬP TRÌNH TRỰC TUYẾN

CÁN BỘ HƯỚNG DẪN
(Chữ ký)

SINH VIÊN THỰC HIỆN
(Chữ ký)

Họ và tên

Họ và tên

Ngày báo cáo: ../../....

TRƯỞNG BAN
(Chữ ký)

CÁN BỘ PHẢN BIỆN
(Chữ ký)

ỦY VIÊN, THƯ KÝ
(Chữ ký)

.....
.....

.....

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

Tên đề tài: Xây dựng hệ thống dạy và học lập trình trực tuyến

Tên SVTH: Lê Thành Lộc

MSSV: KTPM2211011

Họ và tên GVHD: ThS. Lê Hoàng Minh

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Cần thơ, ngày ... tháng ... năm 2026

Giảng viên hướng dẫn

ThS. Lê Hoàng Minh

NHẬN XÉT CỦA GIẢNG VIÊN PHẢN BIỆN

Tên đề tài: Xây dựng hệ thống dạy và học lập trình trực tuyến

Tên SVTH: Lê Thành Lộc

MSSV: KTPM2211011

Họ và tên GVPB:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Cần thơ, ngày ... tháng ... năm 2026

Giảng viên phản biện

NHẬN XÉT CỦA HỘI ĐỒNG ĐÁNH GIÁ ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

Tên đề tài: Xây dựng hệ thống dạy và học lập trình trực tuyến

Tên SVTH: Lê Thành Lộc

MSSV: KTPM2211011

Họ và tên Hội đồng đánh giá đồ án tốt nghiệp:

1. ThS.

2. ThS.

3. ThS.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Cần thơ, ngày tháng năm 2026

Hội đồng phản biện

LỜI CAM ĐOAN

Tôi là Lê Thành Lộc

Tôi xin cam đoan rằng: Đồ án tốt nghiệp này là cả quá trình nghiên cứu, tự tìm tòi và học hỏi của chính bản thân tôi. Được thực hiện dưới sự hướng dẫn của thầy Lê Hoàng Minh. Tôi hoàn toàn dựa trên những kiến thức đã được học, những tài liệu mà giảng viên cung cấp và bên cạnh đó là những kiến thức mà tôi đã tích góp được trong suốt những năm đại học, những kinh nghiệm mà bản thân tôi đã tích lũy và xây dựng trong suốt một quá trình dài. Tôi hoàn toàn không sao chép của bất kỳ một cá nhân nào.

Các số liệu, kết quả và nội dung trình bày trong đồ án là trung thực, được thu thập và phân tích từ các nguồn đáng tin cậy, đều được trích dẫn rõ ràng. Những phần có sử dụng tài liệu tham khảo trong đồ án đều được liệt kê rõ ở phần tài liệu tham khảo.

Tôi xin chịu hoàn toàn trách nhiệm trước pháp luật và quy định của trường về tính trung thực của đồ án này.

Cần Thơ, ngày tháng năm 2026

Sinh viên thực hiện

Lê Thành Lộc

LỜI CẢM ƠN

Trong suốt khoảng thời gian làm đồ án, Em đã nhận được nhiều sự giúp đỡ, đóng góp ý kiến của quý thầy cô và bạn bè. Em xin chân thành cảm ơn sâu sắc đến các thầy cô Khoa Công Nghệ Thông Tin của Trường Đại Học Kỹ Thuật – Công Nghệ Cần Thơ nói chung và thầy ThS. Lê Hoàng Minh nói riêng. Trong quá trình học tập thì em được học những môn cơ sở ngành đến những môn chuyên ngành đó cũng là một nền tảng để hoàn thành tốt đồ án này.

Để hoàn thành được đề tài “Xây dựng hệ thống dạy và học lập trình trực tuyến ” thì em đã nhận được sự giúp đỡ tận tình của thầy ThS. Lê Hoàng Minh, sự tận tụy, kiên nhẫn và những kiến thức quý báu của thầy đã truyền đạt giúp Em không chỉ hoàn thành đồ án này mà còn mở rộng tầm hiểu biết về ngành công nghệ thông tin. Nhóm em đã học được rất nhiều điều bổ ích từ những góp ý của thầy, từ kỹ năng lập trình đến kỹ năng tự học.

Trong quá trình thực hiện còn nhiều hạn chế trong kỹ năng lẫn kiến thức chuyên sâu, có thể chưa phát huy hết những ý tưởng của thầy. Nhưng những kinh nghiệm và kỹ năng mà em đã tích lũy từ đồ án này sẽ là nền tảng vững cho những dự án tiếp theo.

Em xin chân thành cảm ơn!

Cần Thơ, ngày...tháng... năm 2026

TÓM TẮT

Trong kỷ nguyên chuyển đổi số, việc đào tạo và phát triển nguồn nhân lực công nghệ thông tin ngày càng trở nên cấp thiết. Sự bùng nổ của các nền tảng học trực tuyến đã tạo điều kiện thuận lợi cho việc tiếp cận tri thức, đặc biệt là trong lĩnh vực lập trình. Tuy nhiên, với đặc thù của bộ môn lập trình, một hệ thống học tập không chỉ cần cung cấp tài liệu lý thuyết mà còn phải trang bị môi trường thực hành trực tiếp và khả năng đánh giá mã nguồn một cách tự động, chính xác. Chính vì vậy, việc tự động hóa quá trình chấm bài và kiểm soát tính trung thực trong học tập đóng vai trò vô cùng quan trọng, giúp tối ưu hóa thời gian cho giảng viên và nâng cao chất lượng học hành của sinh viên. Việc đánh giá mã nguồn tự động không chỉ đơn thuần là thao tác kiểm tra kết quả đầu ra, mà còn là một quy trình phức tạp đòi hỏi tính bảo mật, sự ổn định và tốc độ xử lý cao, qua đó giúp bảo vệ hệ thống khỏi các đoạn mã độc hại và cung cấp phản hồi tức thì cho người học, góp phần xây dựng một môi trường giáo dục công nghệ hiện đại và minh bạch.

Đề án “Xây dựng hệ thống dạy và học lập trình trực tuyến” đã kết hợp quá trình phát triển nền tảng quản lý học tập với việc triển khai các cơ sở hạ tầng thực thi mã nguồn độc lập. Mục tiêu chính là tạo ra một hệ sinh thái giáo dục khép kín, hỗ trợ quản lý lớp học, tổ chức bài tập và kỳ thi, đồng thời tích hợp tính năng chấm điểm tự động và theo dõi chống gian lận hiệu quả. Trong quá trình thực hiện đề tài, hệ thống đã được xây dựng hoàn chỉnh với nhiều luồng chức năng chuyên sâu. Đối với sinh viên, hệ thống cung cấp môi trường lập trình tích hợp ngay trên trình duyệt, hỗ trợ đa ngôn ngữ và các cảnh báo thời gian thực. Đối với giảng viên, hệ thống hỗ trợ quản lý ngân hàng bài tập thông qua các bộ dữ liệu kiểm thử, theo dõi trạng thái phòng thi trực tiếp, tính toán điểm số tự động và đánh giá linh hoạt thông qua bộ tiêu chí chấm điểm thủ công. Quá trình xây dựng hệ thống luôn đặt trọng tâm vào trải nghiệm người dùng, tính mở rộng và an toàn dữ liệu.

Để hoàn thiện hệ thống, đề án đã nghiên cứu và ứng dụng nhiều kỹ thuật, thuật toán tiên tiến. Về mặt hệ thống cốt lõi, kiến trúc Web và cơ sở dữ liệu được xây dựng bằng các nền tảng Framework lập trình Web mạnh mẽ nhằm giải quyết các luồng nghiệp vụ phức tạp từ phân quyền, giao bài tập đến lưu trữ lịch sử nộp bài. Cùng với đó, môi trường thực thi mã nguồn an toàn được triển khai thông qua công nghệ ảo hóa bằng Sandbox để tạo ra các không gian thực thi hoàn toàn cô lập, giúp biên dịch mã nguồn đa ngôn ngữ và giới hạn khắt khe các tài nguyên máy chủ để ngăn chặn hành vi phá hoại. Đặc biệt, chức năng phát hiện đạo văn cũng được tích hợp thông qua các thuật toán phân tích độ tương đồng cấu trúc bằng công nghệ nhận diện vân tay mã nguồn để tự động đối chiếu chéo các bài nộp, giúp phát hiện chính xác các hành vi sao chép tinh vi giữa các sinh viên. Thông qua việc tích hợp các công nghệ xử lý mã nguồn hiện đại này, đề tài hướng tới việc chuẩn hóa quy trình đào tạo thực hành và nâng cao hiệu suất của giảng viên. Kết quả của đề án là một nền tảng giáo dục công nghệ

hoàn chỉnh, hoạt động ổn định và hoàn toàn có khả năng triển khai ứng dụng vào các trường học hoặc trung tâm đào tạo thực tế.

MỤC LỤC

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN.....	ii
NHẬN XÉT CỦA GIẢNG VIÊN PHẢN BIỆN.....	iii
NHẬN XÉT CỦA HỘI ĐỒNG ĐÁNH GIÁ ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC.....	iv
LỜI CAM ĐOAN.....	v
LỜI CẢM ƠN.....	vi
TÓM TẮT.....	vii
MỤC LỤC.....	ix
DANH MỤC HÌNH ẢNH.....	xii
DANH MỤC BẢNG.....	xiv
LỜI MỞ ĐẦU.....	1
5.1 Ý nghĩa lý luận.....	4
5.2 Ý nghĩa thực tiễn.....	4
CHƯƠNG 1: TỔNG QUAN LÝ THUYẾT CƠ SỞ NGHIÊN CỨU.....	5
1.7.1 Framework Django.....	9
1.7.2 Nền tảng ảo hóa Docker.....	9
1.7.3 Thư viện trình soạn thảo Monaco Editor.....	10
1.7.4 Thuật toán nhận diện vân tay mã nguồn Winnowing.....	11
1.7.5 Ngôn ngữ lập trình Python.....	11
1.7.6 Các công nghệ Front-end (HTML, CSS, JavaScript, Bootstrap).....	12
1.7.7 Nền tảng Cơ sở dữ liệu đám mây Supabase.....	12
1.7.8 Dịch vụ lưu trữ và quản lý tài nguyên Cloudinary.....	13
1.7.9 Kỹ thuật xử lý bất đồng bộ AJAX (Asynchronous JavaScript and XML).....	13
CHƯƠNG 2: PHƯƠNG PHÁP NGHIÊN CỨU.....	15
2.1. Phương Pháp Nghiên Cứu.....	15
2.2. Phân Tích Thiết Kế.....	17
2.2.1. Mô tả hệ thống.....	17
2.2.2. Thiết kế hệ thống.....	18
2.2.2.1 Luồng nộp bài và chấm điểm tự động (Sandbox).....	18

2.2.2.2 Luồng quét đạo văn hàng loạt (Winnowing Algorithm).....	19
2.2.3. Mô tả chức năng.....	21
2.2.4. Sơ đồ Usecase.....	22
2.2.4.1. Biểu đồ Use Case Module Xác thực và Thông tin cá nhân.....	23
2.2.4.2. Biểu đồ Use Case Module Quản trị Hệ thống (Chỉ dành cho Admin)...	28
2.2.4.3. Biểu đồ Use Case Module Quản lý Lớp học và Môn học.....	32
2.2.4.4. Biểu đồ Use Case Module Bài tập và Chống đạo văn.....	38
2.2.4.5. Biểu đồ Use Case Module Làm bài và Chấm điểm (Submissions & Grading).....	43
2.2.4.6. Biểu đồ Use Case Module Thảo luận & Q&A.....	48
2.2.4.7. Biểu đồ Use Case Module Thông báo.....	50
2.2.5. Xây dựng sơ đồ Class (Class Diagram).....	52
2.2.5.1. Sơ đồ lớp Module Lớp học & Môn học.....	53
2.2.5.2 Sơ đồ lớp Module Bài tập & Chống đạo văn.....	54
2.2.5.3. Sơ đồ lớp Module Thi Trắc nghiệm (Quiz).....	55
2.2.5.4. Sơ đồ lớp Module Làm bài & Chấm điểm (Submissions).....	56
2.2.5.5. Sơ đồ lớp Module Tương tác (Discussions).....	57
CHƯƠNG 3: KẾT QUẢ VÀ THẢO LUẬN.....	58
3.1. kết quả triển khai giao diện và chức năng.....	58
3.1.1 Chức năng đăng nhập và xác thực.....	58
3.1.2 Tham gia lớp học đối với học sinh.....	59
3.1.3 Làm bài tập.....	61
3.1.4 Làm bài thi.....	68
3.1.5 Điểm số Học sinh.....	69
3.1.6 Lịch đóng bài tập, bài thi và thông báo.....	70
3.1.7 Lớp học của giảng viên.....	72
3.1.8 Tạo bài tập và bài thi.....	74
3.1.9 Chức năng sổ điểm.....	77
3.1.10 Kiểm tra đạo văn.....	79
3.1.11 Chấm điểm thủ công.....	81
3.1.12 Thảo luận trong bài tập, bài thi.....	84

3.2. kiểm thử hệ thống và đánh giá hiệu năng.....	87
3.2.1. Môi trường triển khai và kiểm thử (Test Environment).....	87
3.2.2. Đánh giá tốc độ xử lý của Sandbox.....	87
3.2.3. Đánh giá độ chính xác của Thuật toán quét đạo văn Winnowing.....	90
3.2.4. Kiểm thử tính năng xử lý ngoại lệ Sandbox.....	98
3.3 Thảo Luận.....	104
CHƯƠNG 4: KẾT LUẬN VÀ ĐỀ XUẤT.....	107
4.1 kết quả đạt được.....	107
4.2. Hạn chế của đề tài.....	107
4.3. Hướng phát triển.....	108
TÀI LIỆU THAM KHẢO.....	109

DANH MỤC HÌNH ẢNH

<i>Hình 1.1 Sơ đồ biểu diễn mối quan hệ giữa các thành phần của hệ thống</i>	5
<i>Hình 1.2 Quy trình chấm điểm mã nguồn tự động</i>	7
<i>Hình 2.1 Sơ đồ tuần tự luồng nộp bài và chấm điểm tự động</i>	18
<i>Hình 2.2 \ Sơ đồ tuần tự quy trình phát hiện đạo văn</i>	20
Hình 2.3. Biểu đồ Use Case Module Xác thực và Thông tin cá nhân.....	23
Hình 2.4. Biểu đồ Use Case Module Quản trị Hệ thống.....	28
Hình 2.5. Biểu đồ Use Case Module Quản lý Lớp học và Môn học.....	33
Hình 2.6. Biểu đồ Use Case Module Bài tập và Chống đạo văn.....	38
Hình 2.7. Biểu đồ Use Case Module Làm bài và Chấm điểm.....	43
Hình 2.8. Biểu đồ Use Case Module Thảo luận & Q&A.....	48
Hình 2.9. Biểu đồ Use Case Module Thông báo.....	50
Hình 2.10. Sơ đồ Class Module Lớp học & Môn học.....	53
Hình 2.11. Sơ đồ Class Module Bài tập & Chống đạo văn.....	54
Hình 2.12. Sơ đồ Class Module Thi Trắc nghiệm.....	55
Hình 2.13. Sơ đồ Class Module Làm bài & Chấm điểm.....	56
Hình 2.14. Sơ đồ Class Module Tương tác & Thông báo.....	57
Hình 3.1. Giao diện form đăng nhập.....	59
Hình 3.2. Tham gia lớp học với mật khẩu.....	60
Hình 3.3. Giao diện lớp học.....	60
Hình 3.4. Giao diện môn học.....	61
Hình 3.5. Giao diện bài tập.....	62
Hình 3.6. Giao diện bài tập lập trình.....	63
Hình 3.7. Giao diện kết quả.....	63
Hình 3.8. Giao diện làm bài trắc nghiệm.....	65
Hình 3.9. Giao diện kết quả trắc nghiệm.....	66
Hình 3.10. Giao diện nộp bài tập dạng tệp tin.....	67
Hình 3.11. Giao diện xác nhận trước khi thi.....	68
Hình 3.12. Giao diện xem thống kê điểm số của học sinh.....	70

Hình 3.13. Giao diện xem thời gian hạn làm bài tập và thi.....	70
Hình 3.14. Giao diện thông báo của website.....	71
Hình 3.15. Email thông báo tự động khi có bài tập gần hết hạn.....	72
Hình 3.16. Danh sách lớp học của giảng viên.....	73
Hình 3.17. Giao diện quản lý lớp học.....	74
Hình 3.18. Form tạo bài tập bài thi dạng code.....	75
Hình 3.19. Chỉnh sửa bài tập qua chế độ thi.....	76
Hình 3.20. Giao diện sổ điểm thống kê.....	77
Hình 3.21. Giao diện xem điểm của sinh viên.....	78
Hình 3.22. Thống kê điểm theo bài tập bài thi.....	79
Hình 3.23. Giao diện kiểm tra đạo văn.....	80
Hình 3.24. Giao diện so sánh bài làm.....	81
Hình 3.25. Tạo rubric chấm điểm thủ công.....	82
Hình 3.26. Chấm điểm cho bài tập cụ thể theo rubric dạng code.....	83
Hình 3.27. Chấm điểm cho bài tập cụ thể theo rubric dạng nộp file.....	84
Hình 3.28. Danh sách các thảo luận trong bài tập, bài thi.....	85
Hình 3.29. Chi tiết thảo luận.....	86
Hình 3.30. Mã nguồn của Sinh viên 1.....	92
Hình 3.31. Mã nguồn của Sinh viên 2.....	92
Hình 3.32. Mã nguồn của Sinh viên 3.....	93
Hình 3.33. Mã nguồn của Sinh viên 4.....	94
Hình 3.34. Mã nguồn của Sinh viên 5.....	95
Hình 3.35. Test Time Limit Exceeded (TLE) khi chạy testcase mẫu.....	98
Hình 3.36. Test nộp bài (Submit) với mã nguồn gây lỗi Time Limit Exceeded.....	99
Hình 3.37. Test Memory Limit Exceeded (MLE) với testcase mẫu trên IDE.....	100
Hình 3.38. Test tràn RAM (MLE) khi nộp bài chính thức.....	101
Hình 3.39. Test Runtime Error (RE) thông qua tính năng chạy thử testcase mẫu.....	102
Hình 3.40. Test Runtime Error (RE) tại màn hình Nộp bài.....	102
Hình 3.41. Test Compilation Error (CE) với testcase mẫu.....	103
Hình 3.42. Test Compilation Error (CE) khi nộp bài chính thức.....	104

DANH MỤC BẢNG

Bảng 2.1. Đặc tả Use Case "Đăng nhập"	24
Bảng 2.2. Đặc tả Use Case "Đăng ký tài khoản"	25
Bảng 2.3. Đặc tả Use Case "Khôi phục mật khẩu"	25
Bảng 2.4. Đặc tả Use Case "Đổi mật khẩu"	26
Bảng 2.5. Đặc tả Use Case "Quản lý hồ sơ"	26
Bảng 2.6. Đặc tả Use Case "Đăng ký Giảng viên"	27
Bảng 2.7. Đặc tả Use Case "Xem Dashboard"	28
Bảng 2.8. Đặc tả Use Case "Xem Dashboard Quản trị"	29
Bảng 2.9. Đặc tả Use Case "Quản lý Phê duyệt"	29
Bảng 2.10. Đặc tả Use Case "Quản lý Người dùng toàn cục"	30
Bảng 2.11. Đặc tả Use Case "Quản lý Lớp học/Môn học toàn cục"	31
Bảng 2.12. Đặc tả Use Case "Quản lý Ngôn ngữ lập trình"	31
Bảng 2.13. Đặc tả Use Case "Quản lý Sandbox"	32
Bảng 2.14. Đặc tả Use Case "Quản lý Cấu hình & Logs"	32
Bảng 2.15. Đặc tả Use Case "Tìm kiếm Lớp học"	34
Bảng 2.16. Đặc tả Use Case "Tham gia Lớp học"	34
Bảng 2.17. Đặc tả Use Case "Quản lý Lớp học"	35
Bảng 2.18. Đặc tả Use Case "Quản lý Thành viên"	35
Bảng 2.19. Đặc tả Use Case "Import danh sách học viên"	36
Bảng 2.20. Đặc tả Use Case "Quản lý Môn học & Học kỳ"	36
Bảng 2.21. Đặc tả Use Case "Bảng tin & Thông báo"	37
Bảng 2.22. Đặc tả Use Case "Sổ điểm & Bảng xếp hạng"	37
Bảng 2.23. Đặc tả Use Case "Tạo bài tập/bài thi"	39
Bảng 2.24. Đặc tả Use Case "Quản lý Bài tập/Bài thi"	40
Bảng 2.25. Đặc tả Use Case "Quản lý File đính kèm"	40
Bảng 2.26. Đặc tả Use Case "Quản lý Testcase"	41
Bảng 2.27. Đặc tả Use Case "Quản lý Tiêu chí chấm (Rubric)"	41

Bảng 2.28. Đặc tả Use Case "Quản lý Quiz".....	42
Bảng 2.29. Đặc tả Use Case "Kiểm tra Đạo văn".....	43
Bảng 2.30. Đặc tả Use Case "Làm bài Lập trình (IDE)".....	44
Bảng 2.31. Đặc tả Use Case "Làm bài Trắc nghiệm".....	45
Bảng 2.32. Đặc tả Use Case "Làm bài Nộp File".....	45
Bảng 2.33. Đặc tả Use Case "Phòng chờ thi".....	46
Bảng 2.34. Đặc tả Use Case "Giám sát phòng thi".....	46
Bảng 2.35. Đặc tả Use Case "Chấm điểm tự động".....	47
Bảng 2.36. Đặc tả Use Case "Chấm điểm thủ công".....	47
Bảng 2.37. Đặc tả Use Case "Xem Lịch sử bài nộp".....	48
Bảng 2.38. Đặc tả Use Case "Quản lý Chủ đề".....	49
Bảng 2.39. Đặc tả Use Case "Tương tác bình chọn".....	49
Bảng 2.40. Đặc tả Use Case "Quản lý nâng cao".....	50
Bảng 2.41. Đặc tả Use Case "Nhận thông báo".....	51
Bảng 2.42. Đặc tả Use Case "Lọc thông báo".....	52
Bảng 2.43. Đặc tả Use Case "Quản lý trạng thái".....	52
Bảng 3.1. Thông số đầu vào và kết quả tổng quan.....	89
Bảng 3.2. Thống kê chi tiết tài nguyên và thời gian phản hồi.....	90
Bảng 3.3. Ma trận nhầm lẫn và các chỉ số đo lường độ chính xác của thuật toán.....	96
Bảng 3.4. Điểm tương đồng mã nguồn (Similarity Score) chi tiết từng cặp sinh viên.	97

LỜI MỞ ĐẦU

1. Lý Do Chọn Đề Tài

Trong những năm gần đây, cùng với sự bùng nổ của mạng toàn cầu và quá trình chuyển đổi số trong giáo dục, việc dạy và học trực tuyến đã trở thành xu hướng tất yếu. Đặc biệt, đối với các ngành thuộc lĩnh vực Công nghệ Thông tin, việc rèn luyện kỹ năng thực hành lập trình là vô cùng quan trọng. Tuy nhiên, các giải pháp công nghệ phục vụ giảng dạy hiện nay vẫn còn tồn đọng nhiều bất cập và thường bị chia thành hai nhóm riêng biệt.

Nhóm thứ nhất là các hệ thống thi đấu thuật toán như HackerRank, LeetCode hay DOMjudge. Mặc dù sở hữu hệ thống chấm điểm tự động (Online Judge) vô cùng mạnh mẽ, các nền tảng này lại được thiết kế chuyên biệt cho việc luyện tập cá nhân hoặc thi đấu. Do đó, chúng thiếu vắng hoàn toàn các tính năng quản lý lớp học cốt lõi (LMS) như: giao bài tập tự luận, làm bài trắc nghiệm lý thuyết, hệ thống chấm điểm theo tiêu chí (Rubric) hay không gian thảo luận trực tiếp giữa giảng viên và sinh viên.

Nhóm thứ hai là các hệ thống quản lý học tập đa năng như Moodle, Canvas hay Microsoft Teams. Các hệ thống này đáp ứng rất tốt nhu cầu quản lý lớp học, nhưng lại mang tính đại trà cho mọi môn học. Chúng thường chỉ hỗ trợ chia sẻ tài liệu, video bài giảng hoặc thi trắc nghiệm lý thuyết mà không có sẵn môi trường thực hành lập trình trực tiếp (IDE) và tính năng tự động chạy testcase (Auto-grader). Để chấm bài tập mã nguồn, giảng viên thường phải tải từng file về máy cá nhân hoặc cấu hình thêm các plugin bên thứ ba phức tạp, gây tiêu tốn rất nhiều thời gian và dễ xảy ra sai sót. Hơn thế nữa, cả hai nhóm hệ thống trên đều thiếu vắng các công cụ phát hiện đạo văn chuyên sâu dành riêng cho mã nguồn, khiến việc ngăn chặn các hành vi gian lận và sao chép tinh vi giữa các sinh viên trở nên cực kỳ khó khăn.

Từ những nhu cầu và tính cấp thiết của thực tiễn đó, em quyết định chọn đề tài: “Xây dựng hệ thống dạy và học lập trình trực tuyến”. Khác biệt hoàn toàn so với các nền tảng đơn lẻ nêu trên, đề án hướng tới việc xây dựng một nền tảng lai (Hybrid Platform), kết hợp hoàn hảo giữa Hệ thống quản lý học tập (LMS) và Hệ thống chấm điểm tự động (Online Judge). Hệ thống không chỉ cung cấp một môi trường lớp học ảo toàn diện với các công cụ điểm danh, thảo luận, giao bài tập, thi trắc nghiệm, mà còn tích hợp sẵn trình biên dịch mã nguồn trực tiếp trên trình duyệt (Web IDE), Sandbox chấm điểm tự động (bắt lỗi TLE/MLE) và lỗi phát hiện đạo văn bằng thuật toán

Winnowing. Sự kết hợp đồng bộ này tạo ra một vòng lặp khép kín "Học - Hành - Đánh giá", góp phần giải quyết triệt để những bài toán đặc thù và nâng cao chất lượng giảng dạy các bộ môn lập trình tại trường đại học hiện nay.

2. Mục Tiêu Nghiên Cứu

Mục tiêu cốt lõi của đề án này là nghiên cứu, thiết kế và phát triển một hệ thống giáo dục trực tuyến chuyên biệt dành cho bộ môn lập trình, tích hợp hạ tầng chấm điểm mã nguồn tự động đa ngôn ngữ. Để giải quyết những hạn chế của phương pháp giảng dạy truyền thống, hệ thống hướng tới các mục tiêu cụ thể sau:

Tự động hóa quy trình đánh giá (Giải quyết bài toán thời gian): Theo nghiên cứu của Ala-Mutka (2005),[1] giảng viên trung bình phải mất từ 10 đến 15 phút để đọc hiểu, biên dịch và chấm điểm thủ công một bài tập lập trình, dẫn đến tình trạng quá tải nghiêm trọng ở các lớp học đông sinh viên và làm chậm trễ quá trình phản hồi (feedback) cho người học. Hệ thống giải quyết triệt để vấn đề này bằng cách tự động thực thi mã nguồn qua môi trường ảo hóa cách ly, đối chiếu với bộ dữ liệu kiểm thử (Testcase) và trả về điểm số tức thì.

Loại bỏ rào cản kỹ thuật cho sinh viên (Giải quyết rào cản môi trường): Theo khảo sát của Pears và cộng sự (2007) về giáo dục khoa học máy tính [2], khoảng 25% - 30% sinh viên mới bắt đầu gặp khó khăn và nản chí ngay trong giai đoạn cài đặt môi trường, cấu hình trình biên dịch và IDE trên máy cá nhân. Đề án xây dựng một không gian lập trình trực tiếp trên trình duyệt, giúp sinh viên có thể viết, chạy thử và nộp bài ngay lập tức mà không cần cài đặt bất kỳ phần mềm thứ ba nào.

Kiểm soát gian lận và tính nguyên bản (Giải quyết vấn nạn đạo văn): Nghiên cứu của Cosma & Joy (2008) chỉ ra rằng nếu thiếu các công cụ quét tự động, tỷ lệ sao chép mã nguồn trong các bài tập lớn tại đại học có thể lên tới 20% [3]. Mục tiêu của đề án là tích hợp thuật toán nhận diện vân tay mã nguồn (Winnowing) để quét chéo tự động toàn bộ bài nộp, hỗ trợ giảng viên phát hiện chính xác các hành vi sao chép tinh vi.

Số hóa toàn diện quản lý học tập: Thiết lập nền tảng quản trị khép kín từ khâu tạo lớp học, giao bài tập, tổ chức kỳ thi, giám sát phòng thi theo thời gian thực đến thống kê, xuất báo cáo điểm số.

3. Đối Tượng Nghiên Cứu

Để hoàn thành các mục tiêu trên, đối tượng nghiên cứu của đề tài tập trung vào các phạm trù sau:

Hệ thống Đánh giá Lập trình Tự động (APAS): Kiến trúc tổng thể, quy trình vận hành và các luồng tương tác giữa máy khách (Client) và máy chủ (Server) trong việc truyền nhận, đánh giá mã nguồn.

Công nghệ ảo hóa và Sandbox: Cơ chế cô lập tài nguyên hệ thống (CPU, RAM, Thời gian thực thi) thông qua container để chạy các đoạn mã đa ngôn ngữ (Python, C++, Java...) một cách an toàn, ngăn chặn mã độc tấn công máy chủ.

Thuật toán so khớp mã nguồn: Các kỹ thuật trích xuất đặc trưng văn bản, loại bỏ nhiễu (khoảng trắng, chú thích, tên biến) và thuật toán Winnowing để đo lường tỷ lệ trùng lặp.

Trải nghiệm người dùng (UX/UI) trong giáo dục trực tuyến: Giao diện trình soạn thảo mã nguồn trên nền tảng Web và cơ chế xử lý tương tác bất đồng bộ (AJAX/WebSockets) để phản hồi trạng thái thi theo thời gian thực.

4. Phạm Vi Nghiên Cứu

Đề tài giới hạn phạm vi nghiên cứu và triển khai trong các giới hạn cụ thể nhằm đảm bảo tính khả thi và hiệu năng của hệ thống:

Về mặt chức năng: Phát triển hoàn chỉnh Website với các luồng nghiệp vụ cốt lõi: Đăng ký/Đăng nhập, Quản lý danh sách lớp/sinh viên, Ngân hàng bài tập, Tổ chức thi trực tuyến có tính giờ, Chấm điểm tự động và Quét đạo văn.

Về môi trường thực thi (Sandbox): Hệ thống ban đầu sẽ hỗ trợ chấm điểm tự động cho các ngôn ngữ lập trình phổ biến nhất trong môi trường học thuật bao gồm: C/C++, Python và Java. Việc kiểm thử được thực hiện qua luồng Standard Input/Output (stdin/stdout).

Về cơ chế giám sát: Áp dụng các biện pháp chống gian lận trên trình duyệt trong thời gian thi như: vô hiệu hóa Copy/Paste, cảnh báo chuyển tab, và ghi nhận sự kiện gõ phím. Cung cấp thêm bộ tiêu chí đánh giá thủ công để giảng viên chấm điểm linh hoạt cho các bài tập đặc thù.

5. Ý Nghĩa Lý Luận Và Thực Tiễn Của Đề Tài

5.1 Ý Nghĩa Lý Luận

Đề tài góp phần hệ thống hóa các tài liệu và cơ sở lý luận về việc xây dựng Hệ thống Đánh giá Lập trình Tự động (APAS).

Làm rõ phương pháp tích hợp các công nghệ phân tán hiện đại (khung làm việc Web, cơ sở dữ liệu đám mây, nền tảng ảo hóa Container) thành một kiến trúc phần mềm thống nhất, đảm bảo tính an toàn và khả năng mở rộng.

Khẳng định tính đúng đắn của việc đưa các thuật toán phân tích dấu vân tay cục bộ (như Winnowing) vào việc giải quyết bài toán đạo văn mã nguồn - vốn có cấu trúc logic phức tạp hơn văn bản thông thường.

5.2 Ý Nghĩa Thực Tiễn

Đối với Giảng viên: Cung cấp một bộ công cụ đặc lực giúp cắt giảm tới 90% thời lượng chấm bài thủ công. Giảng viên được giải phóng khỏi các thao tác lặp đi lặp lại để tập trung vào việc phân tích chất lượng mã nguồn, cải tiến đề thi và tư vấn học thuật cho sinh viên.

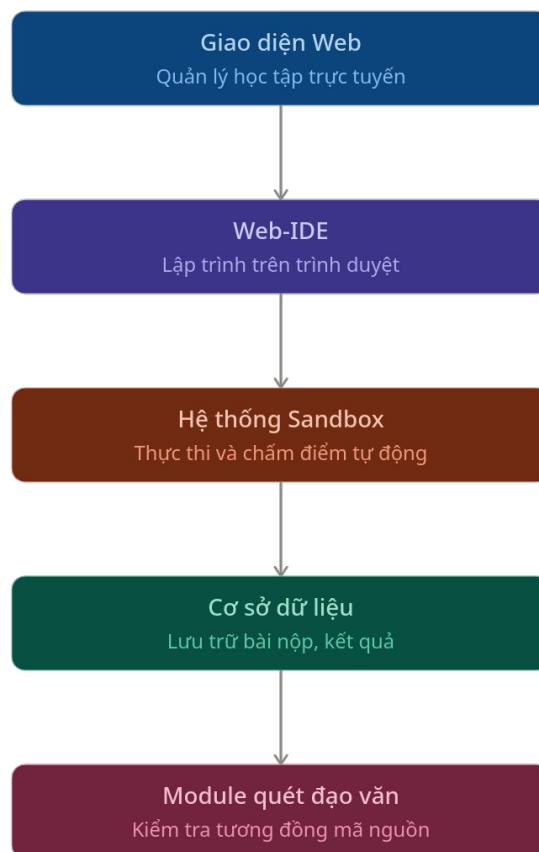
Đối với Sinh viên: Tạo ra một môi trường thực hành trực quan, công bằng và "không rào cản". Việc nhận được phản hồi (Feedback) và điểm số ngay lập tức sau khi nộp bài giúp sinh viên nhanh chóng nhận ra lỗi sai, từ đó rút ngắn chu kỳ học tập và tạo động lực rèn luyện kỹ năng lập trình.

Đối với Cơ sở đào tạo: Sản phẩm của đề án có độ hoàn thiện cao, hoàn toàn đủ điều kiện để triển khai thực tế tại các trường Đại học, Cao đẳng, góp phần thúc đẩy công cuộc chuyển đổi số toàn diện trong công tác kiểm tra, đánh giá chất lượng nguồn nhân lực Công nghệ thông tin.

CHƯƠNG 1: TỔNG QUAN LÝ THUYẾT CƠ SỞ NGHIÊN CỨU

1.1 Sự Liên Kết Giữa Các Thành Phần Trong Đồ Án

Sơ đồ mô tả mối quan hệ giữa các thành phần cốt lõi của hệ thống bao gồm: Nền tảng Website quản lý học tập trực tuyến, Môi trường lập trình trên trình duyệt (Web-IDE), Hệ thống thực thi và chấm điểm mã nguồn tự động, và Hệ thống quét đạo văn. Mối quan hệ chặt chẽ giữa các thành phần này được thể hiện rõ trong Hình 1.1.



Hình 1.1 Sơ đồ biểu diễn mối quan hệ giữa các thành phần của hệ thống

Qua sơ đồ cho thấy rõ nền tảng Website là nơi tương tác trực tiếp với người dùng, làm trung tâm liên kết các hệ thống ngầm bên dưới. Các thành phần kết hợp tạo nên một môi trường dạy và học lập trình khép kín, minh bạch và an toàn:

- Bước 1: Xây dựng thành công hệ thống quản lý học tập trực tuyến. Về cơ bản, nền tảng đáp ứng đủ các chức năng của một hệ thống giáo dục như: quản lý tài khoản, quản lý lớp học, quản lý danh sách bài tập và theo dõi điểm số.

- Bước 2: Tích hợp môi trường lập trình trực tiếp (IDE) vào giao diện Web. Sinh viên sẽ tiến hành gõ mã nguồn, biên dịch và chạy thử ngay trên trình duyệt mà không cần cài đặt môi trường lập trình phức tạp ở máy cá nhân.

- Bước 3: Tiếp theo là tiến trình chấm điểm tự động. Mã nguồn của sinh viên sau khi nộp sẽ được gửi vào hệ thống không gian ảo hóa cách ly (Sandbox). Tại đây, mã nguồn được biên dịch và đối chiếu kết quả đầu ra với các bộ dữ liệu kiểm thử (Testcase) do giảng viên thiết lập sẵn nhằm đánh giá tính đúng đắn và tính toán điểm số.

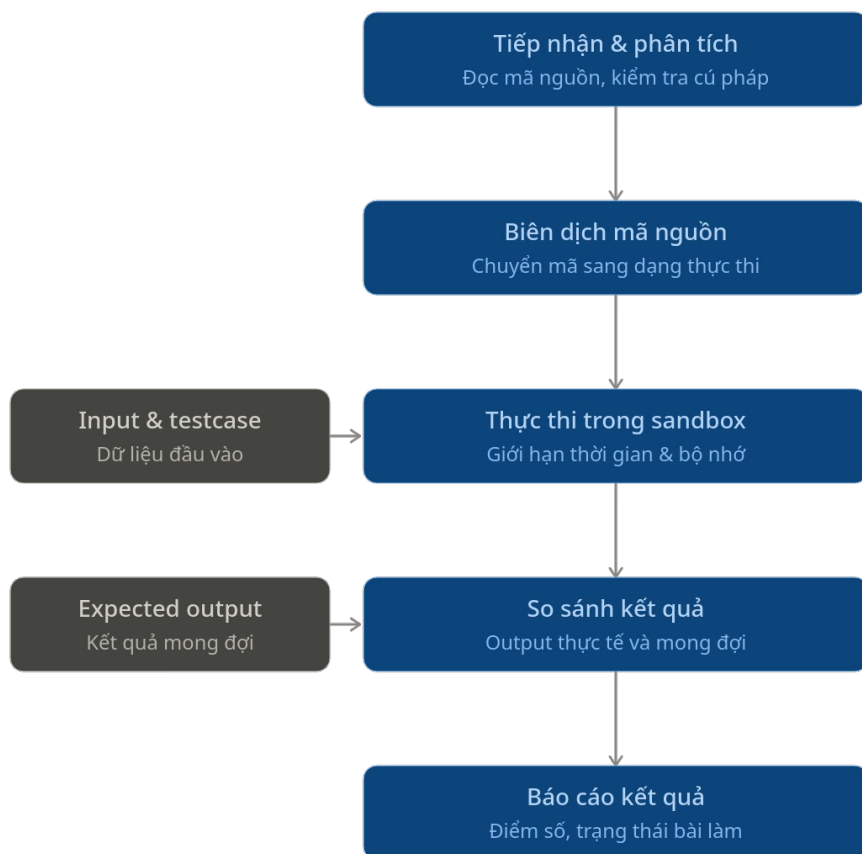
- Bước 4: Cuối cùng là quản lý và phát hiện gian lận. Toàn bộ bài nộp của sinh viên sẽ được đưa qua module phân tích mã nguồn. Ứng dụng các thuật toán so sánh độ tương đồng để dò tìm các hành vi sao chép, giúp giảng viên phát hiện đạo văn và đánh giá lại chất lượng học tập.

1.2 Khái Niệm Về Hệ Thống Quản Lý Học Tập

Hệ thống quản lý học tập (Learning Management System - LMS) là một phần mềm hoặc ứng dụng nền tảng Web được thiết kế để quản lý, theo dõi, báo cáo và phân phối các chương trình giáo dục, khóa học trực tuyến. Mục đích cuối cùng của công việc này là cung cấp một không gian số hóa giúp giảng viên truyền đạt kiến thức và sinh viên có thể tiếp thu, nộp bài kiểm tra mọi lúc mọi nơi. LMS giúp tự động hóa việc lưu trữ dữ liệu giáo dục, từ đó nâng cao chất lượng đào tạo và mang lại nhiều lợi ích cho công tác quản lý nhà trường.

1.3 Khái Niệm Về Chấm Điểm Mã Nguồn Tự Động

Chấm điểm mã nguồn tự động là quá trình sử dụng các hệ thống máy tính để thực thi các đoạn mã do người học viết ra, sau đó kiểm chứng độ chính xác bằng cách so sánh kết quả đầu ra của chương trình với kết quả mong đợi dựa trên các đầu vào cho trước. Quá trình này bao gồm nhiều giai đoạn khác nhau như: Tiếp nhận mã nguồn, phân tích cú pháp, biên dịch mã, thực thi trong môi trường cô lập, và báo cáo kết quả.



Hình 1.2 Quy trình chấm điểm mã nguồn tự động

Công việc chấm điểm mã nguồn tự động đóng vai trò thiết yếu trong các hệ thống học lập trình, giúp thẩm định liệu giải thuật của người học có đáp ứng được các yêu cầu kỹ thuật, giới hạn thời gian và giới hạn bộ nhớ hay không.

1.4 Vai Trò Của Việc Tự Động Hóa Trong Dạy Học Lập Trình

Trong thời đại phát triển vượt bậc hiện nay, ngành công nghệ thông tin đang chiếm ưu thế lớn. Nhu cầu đào tạo lập trình viên ngày càng gia tăng đòi hỏi các cơ sở giáo dục phải tối ưu hóa phương pháp giảng dạy. Chấm điểm thủ công truyền thống tốn rất nhiều công sức và thời gian của giảng viên, đặc biệt khi quy mô lớp học lớn.

Việc áp dụng hệ thống tự động hóa vào dạy học lập trình mang lại các vai trò cốt lõi bao gồm:

- Tiết kiệm thời gian và công sức: Việc phát hiện lỗi cú pháp và chạy kiểm thử tự động giúp tiết kiệm đáng kể thời gian so với việc giảng viên phải tải từng bài nộp về máy cá nhân, cấu hình môi trường và chấm thủ công.

- Phản hồi tức thì cho người học: Sinh viên nhận được kết quả (đúng/sai) và lỗi phát sinh ngay lập tức sau khi nộp bài. Điều này giúp nâng cao trải nghiệm học tập, cho phép người học tự điều chỉnh và khắc phục lỗi ngay tại thời điểm thực hành.

- Đảm bảo tính khách quan và minh bạch: Điểm số được hệ thống máy tính đo lường dựa trên các bài kiểm tra logic định sẵn, loại bỏ các sai sót hoặc thiên kiến từ con người.

- Tăng cường khả năng tự học: Phần mềm được tích hợp sẵn cảnh báo thời gian thực và các lời khuyên khi mã nguồn gặp lỗi, giúp người học chủ động tìm tòi giải pháp thay vì phụ thuộc hoàn toàn vào giảng viên.

1.5 Khái Niệm Và Đặc Điểm Của Môi Trường Cô Lập

Trong lĩnh vực an toàn thông tin và thực thi mã nguồn, Sandbox là một môi trường thử nghiệm cô lập và được quản lý chặt chẽ. Cơ chế này cho phép chạy các chương trình hoặc mã nguồn không xác định mà không làm ảnh hưởng đến hệ điều hành chủ hoặc dữ liệu của máy chủ.

Đặc điểm của công nghệ Sandbox trong giáo dục lập trình:

- An toàn tuyệt đối: Mã nguồn do sinh viên nộp lên có thể chứa các đoạn mã độc (như xóa dữ liệu hệ thống, tấn công mạng, chiếm quyền điều khiển). Sandbox khóa chặt quyền truy cập của chương

trình, không cho phép tương tác ra bên ngoài.

- Quản lý tài nguyên: Cho phép hệ thống giới hạn khắt khe số lượng RAM (dung lượng bộ nhớ) và CPU (thời gian chạy) đối với mỗi bài nộp, tránh tình trạng mã nguồn sinh viên rơi vào vòng lặp vô tận gây sập hệ thống máy chủ.

1.6 Phát Hiện Đạo Văn Trong Lập Trình

Khác với văn bản thông thường, đạo văn trong mã nguồn phức tạp hơn rất nhiều. Người học có thể lách luật bằng cách đổi tên biến, thêm dòng trống, hoặc hoán đổi vị trí các hàm mà vẫn giữ nguyên logic cốt lõi. Do đó, các công cụ so sánh văn bản đơn thuần không thể phát hiện được sự sao chép này.

Phát hiện đạo văn mã nguồn là kỹ thuật phân tích các tập lệnh dựa trên cấu trúc trừu tượng và sự tương đồng về chuỗi ký hiệu. Quá trình này giúp hệ thống xác định

chính xác các bài làm có mức độ giống nhau bất thường, qua đó duy trì tính trung thực và kỷ luật trong các kỳ thi lập trình.

1.7 Giới Thiệu Về Các Công Nghệ Áp Dụng Trong Đồ Án

Để hiện thực hóa hệ thống dạy và học lập trình, đồ án đã nghiên cứu và tích hợp các nền tảng công nghệ mạnh mẽ và phổ biến nhất hiện nay:

1.7.1 Framework Django

Django là một khung phát triển ứng dụng web mã nguồn mở dựa trên ngôn ngữ Python, tuân thủ kiến trúc Model-Template-View (MTV). Theo định hướng phát triển nhanh và tinh gọn, Django cung cấp một hệ sinh thái toàn diện với các mô-đun tích hợp sẵn phục vụ xác thực người dùng, quản trị cơ sở dữ liệu và xử lý luồng dữ liệu một cách an toàn[4]

Lý do lựa chọn: Khác với các micro-framework như Flask, FastAPI hay Node.js (Express) đòi hỏi lập trình viên phải tự tích hợp thủ công các thư viện bên ngoài (ORM, Authentication, Admin Panel), Django áp dụng triết lý "Batteries-Included" (tích hợp sẵn mọi thứ). Điều này giúp rút ngắn tối đa thời gian xây dựng các tính năng cốt lõi của một hệ thống giáo dục (quản lý sinh viên, bài tập, phân quyền). So với Spring Boot (Java), Django kết hợp với Python mang lại tốc độ phát triển nhanh hơn với cú pháp bất rườm rà [5]

Ứng dụng: Trong đồ án này, Django đóng vai trò là "bộ não" của hệ thống, xử lý toàn bộ các luồng nghiệp vụ từ cơ sở dữ liệu, quản lý phiên đăng nhập, cấp quyền giáo viên/sinh viên, cho đến việc phân phối các giao diện Web tương tác.

1.7.2 Nền Tảng Ảo Hóa Docker

Docker là một nền tảng ảo hóa cấp hệ điều hành (OS-level virtualization) cho phép triển khai phần mềm bên trong các gói chuẩn hóa gọi là container. Các container này cô lập môi trường thực thi, bao gồm mã nguồn, thư viện và cấu hình hệ thống, đảm bảo tính nhất quán trên các hạ tầng khác nhau [6].

Lý do lựa chọn: Nếu sử dụng máy ảo truyền thống (Virtual Machines - VMs) như VMware hay VirtualBox, mỗi môi trường Sandbox sẽ phải gánh thêm một hệ điều hành khách (Guest OS) hoàn chỉnh, gây lãng phí tài nguyên và tốn vài phút để khởi động. Docker chia sẻ nhân hệ điều hành máy chủ (Host Kernel), giúp container khởi động chỉ trong vài phần nghìn giây và tiêu tốn rất ít RAM. Điều này là yếu tố quyết định để hệ thống có thể chấp hàng chục bài nộp cùng lúc mà không bị sập [7].

Trong quá trình lựa chọn nền tảng chấm điểm tự động (Online Judge), em đã cân nhắc đến việc sử dụng các dịch vụ có sẵn như Judge0. Mặc dù Judge0 cung cấp một

API mạnh mẽ, nhưng em quyết định tự xây dựng một hệ thống Sandbox tùy chỉnh dựa trên lõi công nghệ Docker. Nguyên nhân chính là việc tự xây dựng Sandbox giúp kiểm soát hoàn toàn tài nguyên và tăng cường tối đa tính bảo mật. Thay vì phụ thuộc vào cấu hình của bên thứ ba, hệ thống Sandbox tự thiết kế cho phép ngắt toàn bộ kết nối mạng ngoài (`network=none`), giới hạn quyền ghi ở mức độ tệp tin (`read-only`), cũng như kiểm soát khắt khe lượng RAM, CPU và số lượng tiến trình (`pids-limit`) được cấp phát cho mỗi bài nộp. Hơn thế nữa, việc tự phát triển Sandbox cho phép tích hợp trực tiếp tiến trình chấm bài vào hệ thống xử lý bất đồng bộ Celery của Django. Kiến trúc này giúp loại bỏ hoàn toàn độ trễ giao tiếp mạng (network latency) so với việc hệ thống phải liên tục gọi API qua lại một máy chủ Judge0 độc lập, từ đó tối ưu hóa đáng kể tốc độ trả kết quả cho người dùng.

Để minh chứng cho tính hiệu quả và độ ổn định của kiến trúc này, em đã tiến hành kiểm thử sức chịu tải (Benchmark) trên thực tế. Khi giả lập hệ thống tiếp nhận từ 20 đến 50 bài nộp mã nguồn cùng một lúc, Sandbox đạt thời gian phản hồi trung bình chỉ 1.56 giây cho mỗi bài nộp với tỉ lệ hoàn thành 100%. Bên cạnh đó, Sandbox cũng chứng minh được khả năng chịu lỗi xuất sắc khi đối mặt với các đoạn mã độc hại hoặc lỗi logic từ sinh viên. Cụ thể, khi tiếp nhận đoạn mã lập vô hạn (Time Limit Exceeded) hay mã cố tình cấp phát mảng vượt ngưỡng 64MB (Memory Limit Exceeded), cơ chế cách ly của Docker lập tức bóp nghẹt tiến trình vi phạm, trả về mã lỗi chính xác mà không làm sập hay ảnh hưởng đến máy chủ chính. Những số liệu thực tế này đã khẳng định việc tự xây dựng Sandbox là một hướng tiếp cận hoàn toàn đúng đắn, đáp ứng trọn vẹn các yêu cầu khắt khe về kỹ thuật của đề án.

Ứng dụng: Đề án ứng dụng Docker để đóng gói các môi trường ngôn ngữ lập trình khác nhau thành các Sandbox độc lập. Khi sinh viên nộp bài, hệ thống sẽ kích hoạt một Docker Container riêng biệt để biên dịch mã nguồn, sau đó phá hủy container ngay khi hoàn thành để đảm bảo máy chủ luôn sạch sẽ.

1.7.3 Thư Viện Trình Soạn Thảo Monaco Editor

Monaco Editor là trình soạn thảo mã nguồn nền tảng web (web-based code editor) do Microsoft phát triển. Đây là thành phần lõi cung cấp năng lực phân tích cú pháp cho Visual Studio Code, hỗ trợ tính năng IntelliSense, đánh dấu cú pháp và tự động định dạng mã [8].

Lý do lựa chọn: So với các thư viện cũ hơn như CodeMirror hay Ace Editor, Monaco Editor vượt trội nhờ việc được tích hợp sẵn các worker phân tích ngôn ngữ (Language Servers) từ hệ sinh thái VS Code. Nó hỗ trợ IntelliSense (gợi ý mã thông

minh) mạnh mẽ hơn hẳn, giúp người học có cảm giác như đang sử dụng một IDE thực thụ cài đặt trên máy tính cá nhân thay vì một ô nhập liệu văn bản thông thường [9].

Ứng dụng: Việc tích hợp Monaco Editor giúp sinh viên có trải nghiệm gõ mã nguồn cực kỳ chuyên nghiệp ngay trên Website, bao gồm các tính năng tô màu cú pháp, tự động thụt lề, và gợi ý lệnh thông minh.

1.7.4 Thuật Toán Nhận Diện Vân Tay Mã Nguồn Winnowing

Winnowing là một thuật toán trích xuất dấu vân tay cục bộ (local fingerprinting) được thiết kế để đo lường độ tương đồng và phát hiện sao chép trong các tập dữ liệu văn bản lớn. Bằng cách sử dụng kỹ thuật k-gram kết hợp với hàm băm (hash function), thuật toán tạo ra các tập giá trị đại diện để đối chiếu văn bản hiệu quả mà không bị ảnh hưởng bởi những thay đổi nhỏ trong cấu trúc [10].

Lý do lựa chọn: Hệ thống MOSS (Measure of Software Similarity) nổi tiếng thực chất cũng sử dụng Winnowing làm lõi thuật toán. Tuy nhiên, MOSS là một dịch vụ đóng (phải gọi API bên ngoài), không cho phép tùy biến sâu. Nếu sử dụng các thuật toán so khớp Cây cú pháp trừu tượng (Abstract Syntax Tree - AST) hay Tree Edit Distance, hệ thống sẽ phải xây dựng bộ phân tích (parser) riêng cho từng ngôn ngữ lập trình rất nặng nề. Winnowing phân tích dựa trên k-gram chuỗi, giúp nó hoạt động nhanh, độc lập với ngôn ngữ (language-agnostic) và dễ dàng tích hợp trực tiếp vào nội bộ máy chủ của đồ án [11].

Ứng dụng: Bằng cách so sánh đối chiếu chéo các "dấu vân tay" này, hệ thống của đồ án có thể phát hiện chính xác mức độ sao chép giữa hai đoạn mã bất chấp việc người học cố tình thay đổi cấu trúc tên biến hay khoảng trắng.

1.7.5 Ngôn Ngữ Lập Trình Python

Python là ngôn ngữ lập trình bậc cao, đa mẫu hình (multi-paradigm), nổi bật với cú pháp tường minh và thư viện chuẩn phong phú. Theo triết lý thiết kế tập trung vào tính dễ đọc, Python hỗ trợ linh hoạt cho cả lập trình hướng đối tượng, lập trình hàm và xử lý dữ liệu [12].

Lý do lựa chọn: Python sở hữu hệ sinh thái cực kỳ mạnh mẽ để tương tác trực tiếp với hệ điều hành thông qua các thư viện subprocess (để gọi lệnh Docker từ Backend) và thao tác chuỗi hiệu năng cao (để băm dữ liệu cho thuật toán Winnowing). Việc dùng chung Python cho cả Backend (Django) lẫn thuật toán chấm điểm giúp kiến trúc hệ thống đồng nhất, dễ bảo trì hơn so với việc viết Backend bằng Node.js nhưng lại phải gọi kịch bản xử lý bằng Bash hay C++.

Ứng dụng: Trong đồ án này, Python là ngôn ngữ nền tảng xuyên suốt để xây dựng toàn bộ Logic ở phía máy chủ (Backend). Python được sử dụng để tương tác với cơ sở dữ liệu, điều khiển các luồng thực thi trong Sandbox (Docker), và đặc biệt đóng vai trò lõi trong việc bóc tách, chuẩn hóa mã nguồn của sinh viên trước khi đưa vào thuật toán phân tích đạo văn.

1.7.6 Các Công Nghệ Front-end (HTML, CSS, JavaScript, Bootstrap)

Lớp giao diện của hệ thống được cấu trúc dựa trên các tiêu chuẩn web cốt lõi. HTML định nghĩa bộ khung ngữ nghĩa, CSS đảm nhiệm việc trình bày trực quan, và JavaScript đóng vai trò xử lý các tương tác động phía máy khách [13].

Lý do lựa chọn: Thay vì sử dụng các framework UI phức tạp như ReactJS hay VueJS đòi hỏi thời gian biên dịch (build time) và thiết lập API phức tạp, việc kết hợp Vanilla JavaScript với Bootstrap giúp đồ án triển khai giao diện nhanh chóng, tương thích trực tiếp với hệ thống Template của Django. Bootstrap được chọn thay vì Tailwind CSS bởi nó cung cấp sẵn các Component (Nút bấm, Modal, Grid) chuẩn mực, giúp tiết kiệm thời gian thiết kế [14].

•

Ứng dụng: Trong dự án, các công nghệ Front-end kết hợp với Bootstrap tạo ra một giao diện học tập hiện đại, thân thiện. Đặc biệt, ngôn ngữ JavaScript đóng vai trò then chốt trong việc tạo ra các tương tác thời gian thực tại màn hình thi, xử lý đồng hồ đếm ngược, vô hiệu hóa các thao tác gian lận (Copy/Paste) và nhận diện các sự kiện gõ phím từ người học.

1.7.7 Nền Tảng Cơ Sở Dữ Liệu Đám Mây Supabase

Supabase là một giải pháp Backend-as-a-Service (BaaS) mã nguồn mở, cung cấp hạ tầng dữ liệu dựa trên hệ quản trị cơ sở dữ liệu quan hệ PostgreSQL [15].

Lý do lựa chọn: Đồ án không chọn Firebase vì Firebase sử dụng NoSQL (cơ sở dữ liệu phi quan hệ), rất khó để thiết kế các liên kết phức tạp như: Sinh viên -> Lớp học -> Bài tập -> Kết quả nộp bài. Supabase cung cấp sức mạnh của PostgreSQL (có quan hệ cấu trúc rõ ràng) kết hợp với các API thời gian thực. So với việc tự thuê VPS để triển

khai MySQL/PostgreSQL đòi hỏi cấu hình bảo mật và tối ưu máy chủ vất vả, Supabase giải quyết trọn gói bài toán lưu trữ chỉ bằng vài thao tác [16].

Ứng dụng: Thay vì lưu trữ dữ liệu cục bộ, đồ án sử dụng Supabase để đưa toàn bộ cơ sở dữ liệu lên đám mây. Hệ thống sẽ lưu trữ và truy xuất các thông tin quan trọng như: tài khoản người dùng, danh sách bài tập, bộ dữ liệu kiểm thử (Testcase), lịch sử nộp bài và điểm số của sinh viên một cách an toàn, bảo mật và luôn đảm bảo tốc độ truy cập cao.

1.7.8 Dịch Vụ Lưu Trữ Và Quản Lý Tài Nguyên Cloudinary

Cloudinary là một dịch vụ quản lý tài nguyên số đa phương tiện trên nền tảng đám mây. Hệ thống cung cấp giải pháp toàn diện cho việc lưu trữ, tự động hóa xử lý và phân phối tài nguyên tĩnh [17].

Lý do lựa chọn: Nếu lưu trữ ảnh ngay trên máy chủ (Local Storage), thư mục dự án sẽ ngày càng phình to và gây quá tải băng thông khi có nhiều sinh viên truy cập. Nếu dùng AWS S3, lập trình viên phải tự thiết lập CDN và mã hóa quy trình nén ảnh thủ công. Cloudinary được chọn vì tính năng tự động nén dung lượng thông minh và cung cấp sẵn CDN tối ưu hóa cho từng loại thiết bị truyền tải, tích hợp thẳng vào Django qua SDK [18].

Ứng dụng: Để giảm tải cho cơ sở dữ liệu và tối ưu tốc độ máy chủ, Cloudinary được tích hợp để chuyên xử lý các tệp tĩnh của hệ thống. Nó chịu trách nhiệm lưu trữ ảnh đại diện của người dùng, hình ảnh mô tả bài tập, hay các tệp tài liệu do giảng viên tải lên, đảm bảo các tài nguyên này luôn được hiển thị với tốc độ nhanh nhất.

1.7.9 Kỹ Thuật Xử Lý Bất Đồng Bộ AJAX (Asynchronous JavaScript And XML)

AJAX là cơ chế giao tiếp bất đồng bộ giữa máy khách và máy chủ, cho phép truy xuất và cập nhật dữ liệu nền [19].

Lý do lựa chọn: So với phương thức Form Submit truyền thống gây tải lại toàn bộ trang (gián đoạn việc code của sinh viên), AJAX giữ nguyên trạng thái màn hình. Dù công nghệ WebSockets (như Socket.io) hỗ trợ truyền tải thời gian thực nhanh hơn, nhưng nó tiêu tốn nhiều tài nguyên kết nối liên tục trên máy chủ và là sự dư thừa (overkill) đối với luồng nộp bài cơ bản (chỉ cần gửi HTTP request và chờ kết quả). AJAX thông qua Fetch API mang lại sự cân bằng hoàn hảo giữa hiệu năng và tính đơn giản [20].

Ứng dụng: AJAX được áp dụng mạnh mẽ vào tính năng cốt lõi nhất của đồ án: Nộp và chấm điểm mã nguồn. Khi sinh viên nhấn "Nộp bài", AJAX sẽ gửi mã nguồn ngầm lên máy chủ để chấm điểm qua Sandbox và trả về kết quả (Đúng/Sai) hiển thị ngay trên màn hình IDE mà không hề làm gián đoạn hay phải tải lại trang Web. Điều này mang lại trải nghiệm thao tác cực kỳ mượt mà cho người sử dụng.

CHƯƠNG 2: PHƯƠNG PHÁP NGHIÊN CỨU

2.1. Phương Pháp Nghiên Cứu

Để xây dựng hệ thống Website dạy và học lập trình trực tuyến, đồ án áp dụng phương pháp phân tích – thiết kế hệ thống thông tin theo hướng tiếp cận từ trên xuống (Top-down) kết hợp với mô hình Thác nước có lặp và tăng trưởng (Iterative and Incremental Waterfall).

Lý do lựa chọn phương pháp này thay vì các mô hình phát triển phần mềm Agile/Scrum phổ biến:

- + Thứ nhất: Các yêu cầu bài toán cốt lõi (xây dựng LMS, Sandbox chấm tự động, lỗi đạo văn Winnowing) đã được khảo sát và chốt chặt chẽ với Giảng viên hướng dẫn ngay từ đầu. Hệ thống không có sự thay đổi đột ngột về yêu cầu kinh doanh liên tục như các dự án thương mại, do đó không cần đến tính linh hoạt phân mảnh của Agile.

- + Thứ hai: Đây là một đồ án tốt nghiệp độc lập do cá nhân thực hiện với quỹ thời gian và giới hạn nguồn lực cố định. Việc áp dụng các nghi thức phức tạp của Scrum (chia Sprint, họp Daily Scrum) là không khả thi và gây lãng phí tài nguyên quản lý.

- + Thứ ba: Phương pháp thiết kế Top-down kết hợp Thác nước giúp định hình toàn bộ cấu trúc cơ sở dữ liệu và kiến trúc tổng thể ngay từ đầu. Tuy nhiên, ở giai đoạn Cài đặt và Kiểm thử, mô hình "Tăng trưởng và lặp" (Iterative & Incremental) được áp dụng: hệ thống được lập trình theo từng cụm module chức năng; mã nguồn sau khi lập trình sẽ được kiểm thử ngay lập tức. Bất kỳ lỗi phát sinh hoặc góp ý nào từ Giảng viên hướng dẫn sẽ tạo ra một vòng lặp phản hồi (feedback loop) để điều chỉnh lại CSDL và mã nguồn trước khi bước sang module khác. Quy trình nghiên cứu và xây dựng hệ thống được triển khai tuần tự nhưng có phản hồi qua 6 bước chính:

Bước 1: Phân tích yêu cầu

Tiến hành khảo sát và thu thập thông tin từ ba nhóm đối tượng: Quản trị viên (Admin), Giảng viên (Teacher) và Học viên (Student). Việc thu thập thông tin thông qua phỏng vấn và quan sát nhằm nhận diện khó khăn trong việc vận hành lớp học lập trình hiện tại (ra đề, chấm code thủ công, chống gian lận). Từ đó, xác định các nghiệp vụ trọng tâm: + Quản lý danh tính và phân quyền (Đăng nhập, đăng ký, xác thực, nâng cấp tài khoản). + Quản lý lớp học và môn học (Tạo lớp, tham gia lớp, import danh sách học viên, quản lý sổ điểm).

- + Quản lý bài tập và chống đạo văn (Tạo bài tập Lập trình, Trắc nghiệm, Tự luận; cấu hình testcase; quét gian lận).

- + Môi trường làm bài và chấm điểm (Web IDE, chấm tự động qua Sandbox, chấm thủ công bằng Rubric, bình luận trực tiếp).

- + Tương tác và thông báo (Diễn đàn thảo luận, thông báo đẩy).

Bước 2: Phân tích hệ thống

- + Xác định các tác nhân (actor) và phân quyền: Admin, Teacher, Student, Guest.

- + Xây dựng biểu đồ Use-case để mô tả và tài liệu hóa các chức năng của hệ thống theo từng góc độ người dùng.

Bước 3: Thiết kế hệ thống

- + Thiết kế kiến trúc tổng thể, phân tách rõ ràng giữa Web Server (Backend) và máy chấm tự động (Sandbox).

- + Thiết kế sơ đồ cơ sở dữ liệu (Class Diagram / ERD) nhằm liên kết tối ưu các thực thể: Người dùng, Bài tập, Lượt nộp bài, Testcase, Sổ điểm.

- + Thiết kế giao diện người dùng (UI/UX) đảm bảo trực quan, thân thiện (có chế độ Dark mode, code highlight).

Bước 4: Cài đặt và tích hợp

- + Lập trình Frontend xây dựng giao diện tương tác linh hoạt.

- + Xây dựng Backend bằng framework Django (Python), lưu trữ dữ liệu an toàn trên

PostgreSQL.

- + Xây dựng và cấu hình Sandbox (sử dụng Docker) để chạy mã nguồn một cách an toàn.

- + Tích hợp Third-party API như Google Login (OAuth 2.0) và API gửi Email tự động.

Bước 5: Kiểm thử (Có yếu tố Lặp - Iterative)

Kiểm thử tích hợp các quy trình nghiệp vụ cốt lõi theo từng cụm module. Nếu phát hiện lỗi, quy trình sẽ lặp lại Bước 3 hoặc Bước 4 để khắc phục:

- + Đăng nhập/Đăng ký - Quản lý tài khoản - Xác thực quyền.
- + Quá trình biên dịch và chấm điểm tự động (Xử lý các lỗi CE, TLE, MLE).
- + Tính chính xác của thuật toán quét tỷ lệ đạo văn Winnowing.
- + Sổ điểm, bảng xếp hạng và các tính năng tương tác.

Bước 6: Triển khai và đánh giá

Đưa hệ thống triển khai thực tế trên môi trường máy chủ (Server/Cloud).

Đánh giá hiệu quả dựa trên: Tốc độ xử lý của Sandbox khi có nhiều lượt nộp đồng thời, tính ổn định, độ bảo mật và mức độ đáp ứng nhu cầu giảng dạy thực tế.

Ghi nhận các hạn chế từ hội đồng và đề xuất cải tiến để hoàn thiện hệ thống trong tương lai.

2.2. Phân Tích Thiết Kế

2.2.1. Mô tả hệ thống

Hệ thống Website dạy và học lập trình được thiết kế với mục tiêu tạo ra một nền tảng học tập tập trung, hỗ trợ toàn diện quá trình giảng dạy và thực hành viết mã (code). Thay vì sử dụng các nền tảng rời rạc để giao bài, thu bài nén (zip) và chấm điểm thủ công, hệ thống cung cấp một môi trường tích hợp cho phép tự động hóa quy trình từ lúc ra đề, làm bài, đến chấm điểm thông qua máy chấm độc lập (Sandbox), giúp tiết kiệm thời gian, nhân lực và đảm bảo tính công bằng tuyệt đối.

Quản trị hệ thống & Người dùng: Quản lý toàn bộ thông tin tài khoản, xét duyệt quyền Giảng viên, theo dõi lịch sử hoạt động hệ thống (Audit logs) và cấu hình giới hạn biên dịch (Time Limit, Memory Limit) cho từng ngôn ngữ lập trình.

Quản lý lớp học & môn học: Cung cấp công cụ tổ chức lớp học, import danh sách sinh viên hàng loạt từ file Excel/CSV, quản lý các học kỳ, gán môn học vào lớp, quản lý thông báo, và thống kê sổ điểm (Gradebook).

Quản lý bài tập & bài thi: Hỗ trợ đa dạng thể loại ra đề (Lập trình, Trắc nghiệm, Tự luận/Nộp file). Nổi bật là khả năng cấu hình bộ dữ liệu kiểm thử (Testcase), thiết lập tiêu chí chấm điểm (Rubric) và tự động quét đạo văn (Plagiarism) bằng cách so sánh độ tương đồng mã nguồn.

Môi trường làm bài & chấm điểm (IDE & Sandbox): Cung cấp trình biên dịch trực tuyến (Web-based IDE) ngay trên trình duyệt. Sandbox sẽ tự động tiếp nhận, biên dịch và chạy thử code của sinh viên đối chiếu với Testcase, sau đó trả về kết quả (Pass, Fail, Time Limit Exceeded...) theo thời gian thực.

Tương tác & Thảo luận: Tích hợp diễn đàn hỏi đáp cho từng chủ đề, tính năng bình luận trực tiếp trên từng dòng code của bài nộp và hệ thống thông báo đẩy (Notifications) giúp kết nối chặt chẽ giữa học viên và giảng viên.

Hệ thống hướng đến 4 nhóm người dùng chính:

Quản trị viên (Admin): Nằm quyền hạn cao nhất, quản lý tài nguyên, thiết lập tham số hệ thống, giám sát cấu hình Sandbox và toàn bộ dữ liệu.

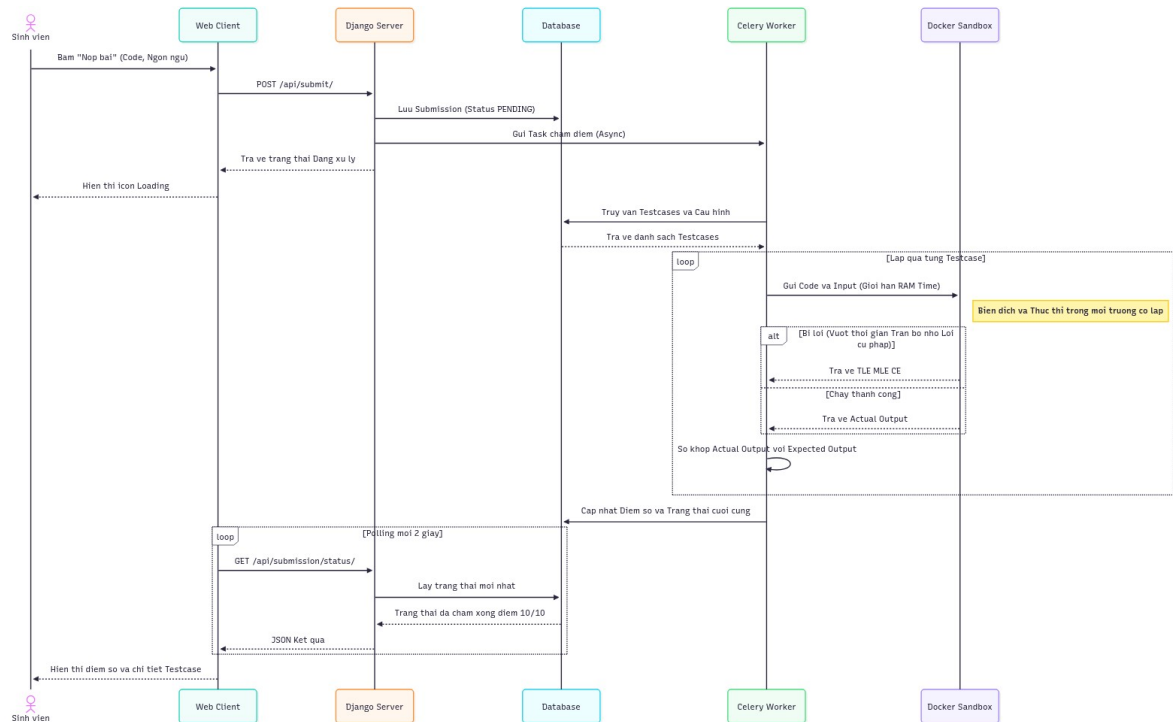
Giảng viên (Teacher): Trực tiếp điều hành lớp học, ra đề, cấu hình Testcase, chấm điểm bằng Rubric và giám sát tiến độ trong phòng thi.

Học viên (Student): Trực tiếp tham gia học tập, viết code trên giao diện IDE, làm bài trắc nghiệm và theo dõi bảng xếp hạng của lớp.

Khách (Guest): Người dùng chưa đăng nhập, sử dụng các tính năng cơ bản như xem giới thiệu, đăng ký và khôi phục mật khẩu.

2.2.2. Thiết kế hệ thống

2.2.2.1 Luồng nộp bài và chấm điểm tự động (Sandbox)



Hình 2.1 Sơ đồ tuần tự luồng nộp bài và chấm điểm tự động

Sơ đồ tuần tự (Sequence Diagram) trên mô tả luồng xử lý khi sinh viên nộp bài tập lập trình và hệ thống tiến hành chấm điểm tự động thông qua cơ chế xử lý bất đồng bộ (asynchronous processing).

Khi sinh viên hoàn thành mã nguồn trên giao diện Web IDE và thực hiện thao tác nộp bài, Web Client sẽ đóng gói mã nguồn cùng các thông tin liên quan (ngôn ngữ lập trình, ID bài tập) và gửi yêu cầu POST đến Django Server. Ngay khi tiếp nhận, Django Server lưu bản ghi Submission vào cơ sở dữ liệu với trạng thái khởi tạo là PENDING, đồng thời đẩy tác vụ chấm điểm vào hàng đợi của Celery. Việc tách tác vụ chấm điểm ra khỏi luồng xử lý request-response chính giúp hệ thống phản hồi ngay lập tức cho người dùng mà không bị nghẽn (blocking) trong lúc chờ quá trình biên dịch và thực thi mã nguồn, vốn có thể tốn nhiều thời gian.

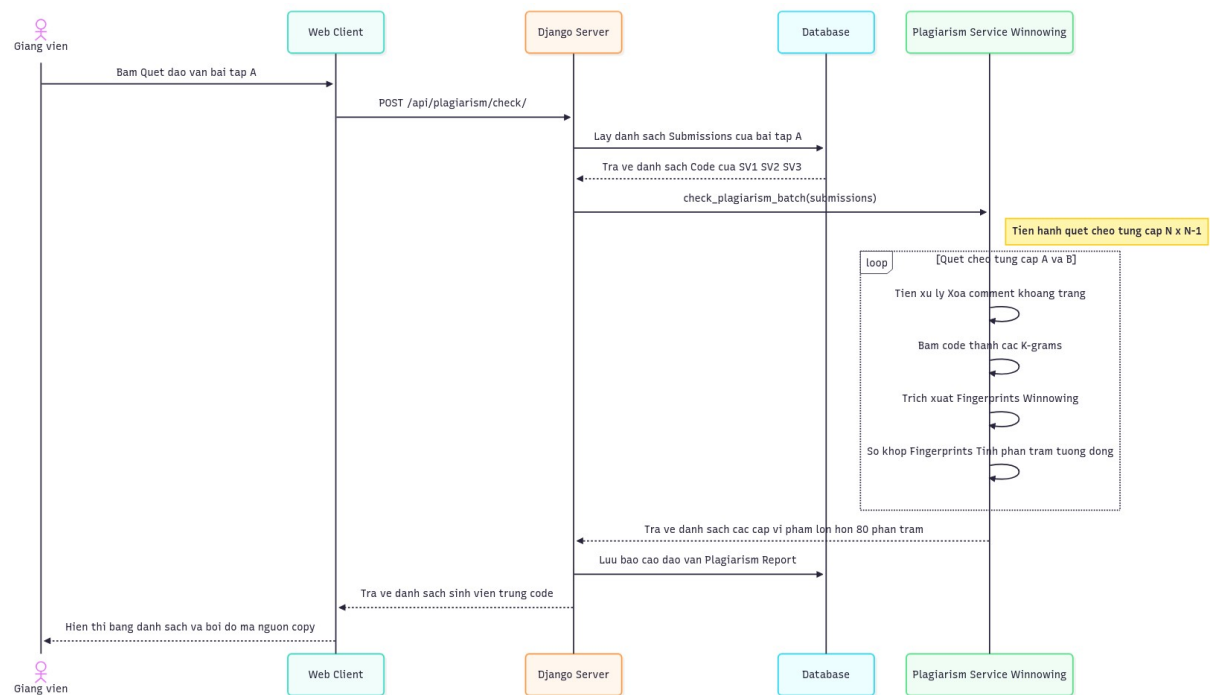
Celery Worker, khi nhận được tác vụ, sẽ truy vấn cơ sở dữ liệu để lấy bộ testcase tương ứng với bài tập, sau đó lần lượt gửi mã nguồn cùng dữ liệu đầu vào của từng testcase vào Docker Sandbox — một môi trường thực thi được cô lập (isolated environment) nhằm đảm bảo an toàn hệ thống trước các đoạn mã có khả năng gây hại hoặc tiêu tốn tài nguyên bất thường. Trong quá trình thực thi, các giới hạn về CPU, bộ nhớ (RAM) và thời gian chạy (timeout) được áp đặt nghiêm ngặt; nếu mã nguồn vi phạm các giới hạn này, Docker sẽ ngắt tiến trình và trả về lỗi tương ứng (Time Limit Exceeded/Memory Limit Exceeded), ngược lại kết quả đầu ra thực tế sẽ được trả về để Celery so khớp với kết quả kỳ vọng (Expected Output).

Sau khi hoàn tất việc chấm điểm trên toàn bộ testcase, Celery Worker cập nhật điểm số và trạng thái cuối cùng của bài nộp (ACCEPTED, WRONG_ANSWER, TIME_LIMIT...) vào cơ sở dữ liệu. Về phía giao diện người dùng, Web Client sử dụng cơ chế polling định kỳ (hoặc WebSocket trong các phiên bản mở rộng) để truy vấn trạng thái mới nhất từ server và cập nhật kết quả chấm điểm lên màn hình cho sinh viên theo thời gian thực.

Kiến trúc này mang lại các ưu điểm sau: Tách biệt tác vụ nặng (chấm điểm) khỏi luồng xử lý chính, tránh làm nghẽn server khi có nhiều sinh viên nộp bài đồng thời.

Đảm bảo an toàn hệ thống thông qua cơ chế sandbox hóa việc thực thi mã nguồn không tin cậy. Cho phép mở rộng theo chiều ngang (horizontal scaling) bằng cách tăng số lượng Celery Worker khi tải hệ thống tăng cao.

2.2.2.2 Luồng quét đạo văn hàng loạt (Winnowing Algorithm)



Hình 2.2 \ Sơ đồ tuần tự quy trình phát hiện đạo văn

Sơ đồ tuần tự trên minh họa quy trình phát hiện đạo văn (plagiarism detection) giữa các bài nộp của sinh viên trong cùng một bài tập, sử dụng thuật toán Winnowing một kỹ thuật băm cục bộ (local hashing) phổ biến trong các hệ thống chống đạo văn mã nguồn.

Quy trình được khởi động khi giảng viên chọn chức năng "Quét đạo văn toàn lớp" trên giao diện quản lý. Django Server tiếp nhận yêu cầu và truy vấn cơ sở dữ liệu để lấy toàn bộ các bài nộp hợp lệ (chỉ tính các bài nộp thành công) của bài tập được chỉ định, sau đó chuyển tập dữ liệu này cho Plagiarism Service — module xử lý logic thuật toán độc lập với tầng điều phối request.

Plagiarism Service thực hiện so sánh chéo từng cặp bài nộp (với độ phức tạp $O(N \times (N-1))$ trong trường hợp không tối ưu). Với mỗi cặp, hệ thống trải qua ba giai đoạn xử lý chính:

- + Tiền xử lý (Preprocessing): loại bỏ các yếu tố không mang ý nghĩa cấu trúc logic như comment, khoảng trắng, xuống dòng nhằm chuẩn hóa mã nguồn, tránh sai lệch do khác biệt định dạng.
- + Băm K-gram (K-gram Hashing): chia mã nguồn đã chuẩn hóa thành các chuỗi con liên tiếp có độ dài K (k-grams) và băm từng chuỗi con thành giá trị số.
- + Trích xuất vân tay (Fingerprinting): áp dụng thuật toán Winnowing để chọn ra tập giá trị băm đại diện (fingerprint) trong mỗi cửa sổ trượt (sliding window),

giúp giảm đáng kể khối lượng dữ liệu cần so sánh mà vẫn giữ được khả năng phát hiện các đoạn mã trùng lặp.

Sau khi có tập fingerprint của cả hai bài nộp trong từng cặp, hệ thống so khớp các giá trị trùng nhau để tính ra tỷ lệ phần trăm tương đồng. Các cặp có độ tương đồng vượt ngưỡng cho phép (ví dụ trên 80%) được xem là có dấu hiệu đạo văn và được Plagiarism Service trả về cho Django Server. Kết quả này được lưu lại dưới dạng báo cáo đạo văn (Plagiarism Report) trong cơ sở dữ liệu để phục vụ tra cứu về sau, đồng thời được hiển thị trực quan cho giảng viên dưới dạng danh sách các cặp sinh viên nghi vấn kèm phần mã nguồn trùng lặp được bôi đỏ để tiện đối chiếu.

Ưu điểm của phương pháp Winnowing so với so sánh trực tiếp toàn văn bản:

- + Có khả năng phát hiện đạo văn cục bộ (chỉ copy một phần đoạn code), không chỉ đạo văn toàn bộ.
- + Kháng được các kỹ thuật "né" đơn giản như đổi tên biến, thêm/bớt khoảng trắng, sắp xếp lại comment.
- + Hiệu suất tính toán tốt hơn so với việc so sánh ký tự trực tiếp (character-by-character) nhờ giảm khối lượng dữ liệu qua bước fingerprinting.

2.2.3. Mô tả chức năng

Yêu cầu chức năng:

Trong quá trình thiết kế và triển khai hệ thống, việc phân quyền người dùng đóng vai trò vô cùng quan trọng nhằm đảm bảo tính bảo mật, minh bạch và hiệu quả trong vận hành. Hệ thống được xây dựng trên nguyên tắc phân cấp quyền hạn rõ ràng, phù hợp với nghiệp vụ của từng đối tượng sử dụng.

Quản trị viên (Admin) là người nắm quyền cao nhất. Admin thực hiện các chức năng thiết lập lõi như phê duyệt các đơn xin cấp quyền Giảng viên, khóa/mở khóa tài khoản người dùng, tạo mới môn học và thiết lập học kỳ hệ thống. Đặc biệt, Admin chịu trách nhiệm giám sát hệ thống máy chấm (Sandbox), xử lý các tiến trình biên dịch bị treo (zombie tasks) và kiểm tra log hoạt động bảo mật của hệ thống.

Giảng viên (Teacher) đóng vai trò trung tâm trong quá trình đào tạo. Giảng viên được cấp quyền tạo lớp học mới, tạo hoặc nhân bản bài tập, tải lên các file tài liệu và cấu hình Testcase hàng loạt. Giảng viên có công cụ giám sát phòng thi theo thời gian thực, có quyền ép nộp bài ngay lập tức (Force submit) hoặc cộng thêm thời gian thi cho một cá nhân. Sau khi thi, giảng viên thực hiện chấm điểm tự luận bằng Rubric, để lại nhận xét ngay trên dòng code bị lỗi của sinh viên, và xem báo cáo tỷ lệ đạo văn.

Học viên (Student) là đối tượng thụ hưởng trực tiếp các dịch vụ của nền tảng. Học viên truy cập để làm bài tập thông qua giao diện trình biên dịch trực tuyến (IDE có

hỗ trợ tô màu cú pháp và Dark mode). Khi nhấn nút "Chạy thử", học viên nhận ngay phản hồi từ máy chấm. Ngoài ra, học viên làm bài trắc nghiệm, nộp file báo cáo, theo dõi Bảng xếp hạng (Leaderboard) thi đua cùng lớp, tra cứu sổ điểm và tham gia bình chọn (Vote) trên diễn đàn hỏi đáp.

Khách (Guest) là lớp người dùng cơ bản nhất, được cung cấp tính năng tạo tài khoản (mặc định vào nhóm Học viên), đăng nhập tiện lợi qua Google Login API (OAuth 2.0) và thực hiện quy trình khôi phục mật khẩu qua Email tự động.

Yêu cầu phi chức năng:

Bảo mật: Mật khẩu người dùng được băm (hash) an toàn trong cơ sở dữ liệu. Hệ thống áp dụng cơ chế xác thực phiên làm việc, chống các cuộc tấn công SQL Injection và Cross-Site Scripting (XSS). Dữ liệu mã nguồn tải lên hoặc nộp file được kiểm tra chặt chẽ về định dạng và dung lượng.

Hiệu năng và khả năng mở rộng: Kiến trúc hệ thống tách biệt rõ ràng giữa Web Server (chạy website) và Sandbox Server (chấm bài). Sandbox ứng dụng công nghệ ảo hóa (Docker) để cách ly môi trường thực thi, chống mã độc, đồng thời giúp hệ thống xử lý mượt mà hàng trăm lượt nộp bài đồng thời thông qua cơ chế hàng đợi (Queue).

Tính tương thích: Website được tối ưu hóa để hoạt động ổn định trên các trình duyệt phổ biến (Chrome, Edge, Safari). Riêng giao diện làm bài (IDE) được thiết kế ưu tiên hiển thị trên màn hình máy tính (PC/Laptop) để đảm bảo không gian thao tác viết code tốt nhất.

Trải nghiệm người dùng (UX): Giao diện trực quan, tối giản. Các bài thi trắc nghiệm hoặc viết code đều có tính năng lưu nháp tự động (Auto-save) nhằm ngăn chặn việc mất dữ liệu khi gặp sự cố ngắt kết nối mạng.

2.2.4. Sơ đồ Usecase

Các tác nhân (Actors):

Admin (Quản trị viên): Có quyền hạn cao nhất, quản lý toàn bộ hệ thống, phê duyệt các yêu cầu tạo tài khoản Giảng viên, Lớp học, Môn học, cấu hình máy chủ chấm code (Sandbox) và theo dõi lịch sử thao tác hệ thống (Logs).

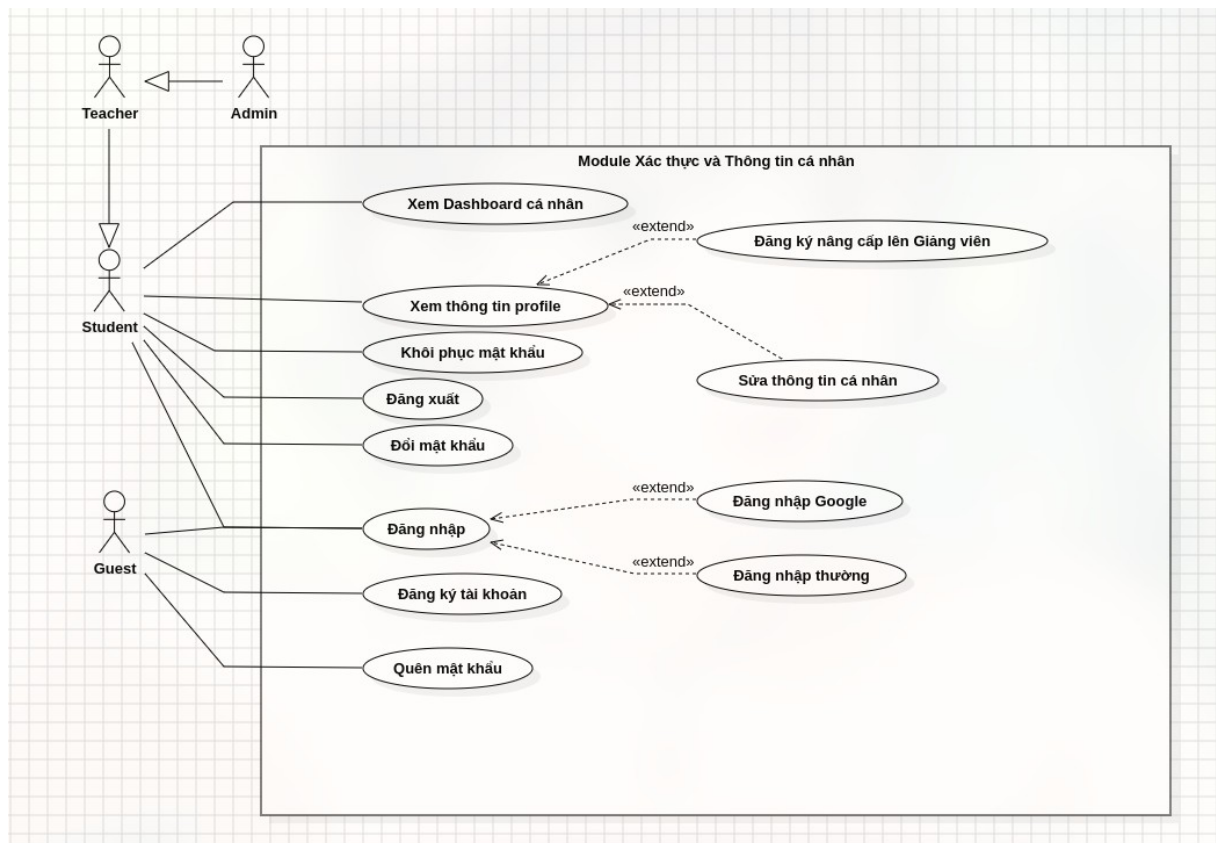
Teacher (Giảng viên): Quản lý các lớp học do mình phụ trách, tạo bài tập/bài thi (Code, Trắc nghiệm, Tự luận), import testcase, thiết lập tiêu chí chấm điểm (Rubric), giám sát phòng thi và kiểm tra đạo văn.

Student (Học viên): Người dùng đã đăng ký thành công. Có quyền tham gia lớp học (thông qua mã code hoặc link), làm bài trên IDE trực tuyến, làm bài trắc nghiệm, nộp file bài tập, xem lịch sử nộp bài, xem thống kê điểm số và tham gia thảo luận.

Guest (Khách): Tác nhân chưa đăng nhập, chỉ có thể thực hiện chức năng đăng nhập, đăng ký tài khoản (hệ thống mặc định cấp quyền Student) và yêu cầu khôi phục mật khẩu.

2.2.4.1. Biểu đồ Use Case Module Xác thực và Thông tin cá nhân

Mô tả: Module này cho phép tất cả các tác nhân thao tác với tài khoản và phiên làm việc của mình. Guest có thể đăng ký tài khoản mới (mặc định là Student). Các tác nhân đã có tài khoản hợp lệ có thể đăng nhập (bao gồm đăng nhập Google), đăng xuất, quên mật khẩu và cập nhật thông tin cá nhân. Học viên có thể gửi yêu cầu nâng cấp lên quyền Giảng viên (trạng thái chờ duyệt).



Hình 2.3. Biểu đồ Use Case Module Xác thực và Thông tin cá nhân

Danh sách chức năng chi tiết:

Đăng nhập (Login)

Đăng ký (Register)

Khôi phục mật khẩu (Forgot Password)

Đổi mật khẩu (Change Password)

Quản lý hồ sơ (Profile)

Đăng ký Giảng viên (Teacher Registration)

Xem Dashboard

Đặc tả chi tiết các Use Case:

Bảng 2.1. Đặc tả Use Case "Đăng nhập"

Mục	Nội dung
Tên Use Case	Đăng nhập hệ thống
Tác nhân	Admin, Teacher, Student
Mô tả	Cho phép người dùng đã có tài khoản truy cập vào hệ thống.
Tiền điều kiện	Người dùng đã tạo tài khoản và không bị khóa.
Hậu điều kiện	Đăng nhập thành công, khởi tạo session, chuyển hướng tới Dashboard.
Luồng sự kiện chính	1. Chọn Đăng nhập. 2. Nhập Username/Password hoặc chọn Đăng nhập Google. 3. Nhấn "Đăng nhập". 4. Hệ thống kiểm tra. 5. Chuyển hướng Dashboard.
Luồng rẽ nhánh	- Sai thông tin → Thông báo lỗi. - Tài khoản bị khóa → Cảnh báo.

Bảng 2.1. Đặc tả Use Case "Đăng nhập"

Bảng 2.2. Đặc tả Use Case "Đăng ký tài khoản"

Mục	Nội dung
Tên Use Case	Đăng ký tài khoản
Tác nhân	Guest
Mô tả	Cho phép người dùng tạo tài khoản mới.
Tiền điều kiện	Email chưa tồn tại trong hệ thống.

Mục	Nội dung
Hậu điều kiện	Tài khoản được tạo với quyền Student.
Luồng sự kiện chính	1. Chọn Đăng ký. 2. Nhập Họ tên, Username, Email, Mật khẩu. 3. Hệ thống validate. 4. Lưu CSDL và tự động đăng nhập.
Luồng rẽ nhánh	- Email/Username đã tồn tại → Thông báo lỗi.

Bảng 2.2. Đặc tả Use Case "Đăng ký tài khoản"

Bảng 2.3. Đặc tả Use Case "Khôi phục mật khẩu"

Mục	Nội dung
Tên Use Case	Khôi phục mật khẩu (Forgot Password)
Tác nhân	Guest
Mô tả	Khôi phục mật khẩu qua Email khi người dùng quên.
Tiền điều kiện	Người dùng có Email hợp lệ đã đăng ký.
Hậu điều kiện	Mật khẩu mới được thiết lập.
Luồng sự kiện chính	1. Chọn Quên mật khẩu. 2. Nhập Email. 3. Hệ thống gửi link khôi phục. 4. Người dùng bấm link, nhập mật khẩu mới. 5. Lưu CSDL.
Luồng rẽ nhánh	- Email không tồn tại → Thông báo không tìm thấy.

Bảng 2.3. Đặc tả Use Case "Khôi phục mật khẩu"

Bảng 2.4. Đặc tả Use Case "Đổi mật khẩu"

Mục	Nội dung
Tên Use Case	Đổi mật khẩu
Tác nhân	Admin, Teacher, Student
Mô tả	Người dùng đổi mật khẩu khi đang đăng nhập để bảo mật.
Tiền điều kiện	Đã đăng nhập.

Mục	Nội dung
Hậu điều kiện	Mật khẩu được cập nhật.
Luồng sự kiện chính	1. Vào Profile. 2. Nhập mật khẩu cũ, mật khẩu mới, xác nhận mật khẩu. 3. Nhấn Lưu. 4. Hệ thống cập nhật.
Luồng rẽ nhánh	- Mật khẩu cũ sai → Thông báo lỗi.

Bảng 2.4. Đặc tả Use Case "Đổi mật khẩu"

Bảng 2.5. Đặc tả Use Case "Quản lý hồ sơ"

Mục	Nội dung
Tên Use Case	Quản lý hồ sơ (Profile)
Tác nhân	Admin, Teacher, Student
Mô tả	Chỉnh sửa thông tin cá nhân và Avatar.
Tiền điều kiện	Đã đăng nhập.
Hậu điều kiện	Thông tin/Avatar mới được lưu lên DB/Cloudinary.
Luồng sự kiện chính	1. Vào trang Cá nhân. 2. Đổi Họ tên, SĐT, up Avatar. 3. Nhấn Lưu. 4. Hệ thống cập nhật.
Luồng rẽ nhánh	- File ảnh quá lớn/sai định dạng → Báo lỗi upload.

Bảng 2.5. Đặc tả Use Case "Quản lý hồ sơ"

Bảng 2.6. Đặc tả Use Case "Đăng ký Giảng viên"

Mục	Nội dung
Tên Use Case	Đăng ký Giảng viên
Tác nhân	Student
Mô tả	Nộp đơn yêu cầu cấp quyền Giảng viên.
Tiền điều kiện	Quyền hiện tại là Student.

Mục	Nội dung
Hậu điều kiện	Đơn được gửi cho Admin chờ duyệt.
Luồng sự kiện chính	1. Bấm Đăng ký Giảng viên. 2. Điền thông tin chuyên môn. 3. Gửi đơn. 4. Trạng thái chuyển sang "Chờ duyệt".
Luồng rẽ nhánh	- Đã gửi đơn trước đó → Hệ thống báo đơn đang chờ duyệt.

Bảng 2.6. Đặc tả Use Case "Đăng ký Giảng viên"

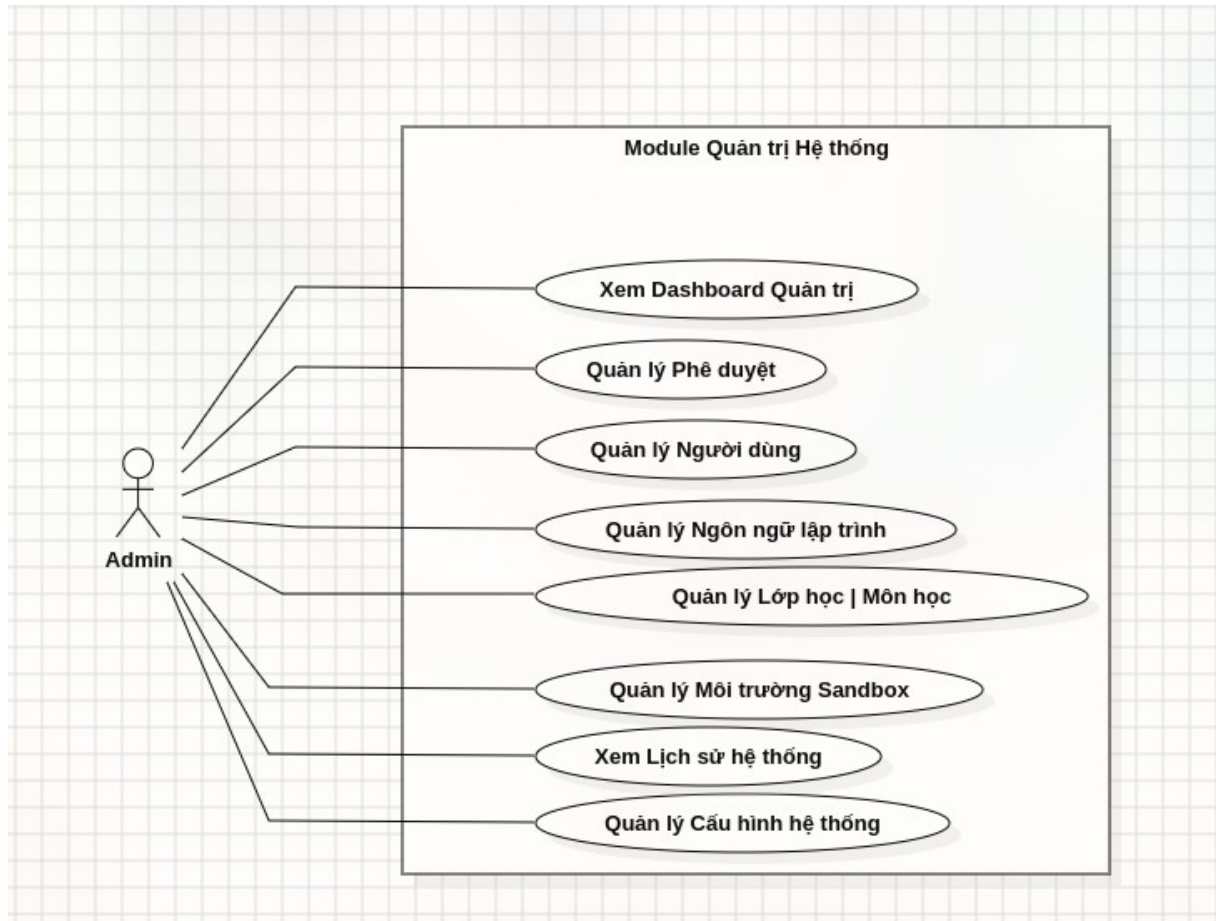
Bảng 2.7. Đặc tả Use Case "Xem Dashboard"

Mục	Nội dung
Tên Use Case	Xem Dashboard
Tác nhân	Teacher, Student
Mô tả	Xem thống kê cá nhân (Tiến độ học tập, lớp quản lý).
Tiền điều kiện	Đã đăng nhập.
Hậu điều kiện	Giao diện Dashboard được hiển thị.
Luồng sự kiện chính	1. Truy cập trang chủ. 2. Hệ thống query dữ liệu thống kê. 3. Render biểu đồ và danh sách.
Luồng rẽ nhánh	- Lỗi query DB → Thông báo hệ thống bận.

Bảng 2.7. Đặc tả Use Case "Xem Dashboard"

2.2.4.2. Biểu đồ Use Case Module Quản trị Hệ thống (Chỉ dành cho Admin)

Mô tả: Chức năng dành riêng cho Admin nhằm kiểm soát quyền truy cập và tài nguyên. Admin có thể xem Dashboard, phê duyệt yêu cầu, quản lý người dùng, lớp học, cấu hình Sandbox và xem Logs.



Hình 2.4. Biểu đồ Use Case Module Quản trị Hệ thống

Danh sách chức năng chi tiết:

Xem Dashboard Quản trị

Quản lý Phê duyệt (Approvals)

Quản lý Người dùng toàn cục

Quản lý Lớp học/Môn học toàn cục

Quản lý Ngôn ngữ lập trình

Quản lý Sandbox

Quản lý Cấu hình (Settings) & Lịch sử (Logs)

Đặc tả chi tiết các Use Case:

Bảng 2.8. Đặc tả Use Case "Xem Dashboard Quản trị"

Mục	Nội dung
Tên Use Case	Xem Dashboard Quản trị
Tác nhân	Admin
Mô tả	Trực quan hóa số liệu hệ thống.
Tiền điều kiện	Đăng nhập bằng quyền Admin.
Hậu điều kiện	Hiển thị biểu đồ thống kê.
Luồng sự kiện chính	1. Vào Dashboard. 2. Xem các chỉ số: tổng user, lớp học, tải server Sandbox.
Luồng rẽ nhánh	- Không có quyền Admin → Từ chối truy cập (403).

Bảng 2.8. Đặc tả Use Case "Xem Dashboard Quản trị"

Bảng 2.9. Đặc tả Use Case "Quản lý Phê duyệt"

Mục	Nội dung
Tên Use Case	Quản lý Phê duyệt
Tác nhân	Admin
Mô tả	Duyệt/Từ chối cấp quyền Giảng viên, tạo Lớp.
Tiền điều kiện	Có yêu cầu chờ duyệt.
Hậu điều kiện	Yêu cầu được chuyển trạng thái.
Luồng sự kiện chính	1. Vào danh sách chờ duyệt. 2. Bấm "Duyệt" hoặc "Từ chối". 3. Hệ thống đổi Role/Tạo lớp. 4. Gửi thông báo cho người yêu cầu.
Luồng rẽ nhánh	- Không có dữ liệu → Hiển thị danh sách rỗng.

Bảng 2.9. Đặc tả Use Case "Quản lý Phê duyệt"

Bảng 2.10. Đặc tả Use Case "Quản lý Người dùng toàn cục"

Mục	Nội dung
Tên Use Case	Quản lý Người dùng toàn cục
Tác nhân	Admin
Mô tả	Thêm/sửa/xóa/khóa tài khoản, reset mật khẩu người dùng.
Tiền điều kiện	Đăng nhập Admin.
Hậu điều kiện	Dữ liệu người dùng được cập nhật.
Luồng sự kiện chính	1. Vào Quản lý User. 2. Tìm kiếm User. 3. Click nút Khóa/Reset Pass. 4. Xác nhận. 5. Hệ thống thực thi.
Luồng rẽ nhánh	- Khóa chính mình → Hệ thống chặn thao tác.

*Bảng 2.10. Đặc tả Use Case "Quản lý Người dùng toàn cục"***Bảng 2.11. Đặc tả Use Case "Quản lý Lớp học/Môn học toàn cục"**

Mục	Nội dung
Tên Use Case	Quản lý Lớp học/Môn học toàn cục
Tác nhân	Admin
Mô tả	Khóa, xóa hoặc xem bất kỳ lớp học/môn học nào.
Tiền điều kiện	Đăng nhập Admin.
Hậu điều kiện	Lớp/Môn bị tác động.
Luồng sự kiện chính	1. Vào DS Lớp/Môn. 2. Bấm Khóa/Xóa. 3. Hệ thống kiểm tra ràng buộc. 4. Xóa/Khóa thành công.
Luồng rẽ nhánh	- Lớp đang có thi → Không cho phép xóa.

Bảng 2.11. Đặc tả Use Case "Quản lý Lớp học/Môn học toàn cục"

Bảng 2.12. Đặc tả Use Case "Quản lý Ngôn ngữ lập trình"

Mục	Nội dung
Tên Use Case	Quản lý Ngôn ngữ lập trình
Tác nhân	Admin
Mô tả	Bật/tắt ngôn ngữ hỗ trợ biên dịch, set Time/Memory Limit.
Tiền điều kiện	Đăng nhập Admin.
Hậu điều kiện	Giới hạn ngôn ngữ được cập nhật vào Sandbox.
Luồng sự kiện chính	1. Vào Cấu hình Ngôn ngữ. 2. Đổi Time Limit (vd Python: 2s, C++: 1s). 3. Nhấn Lưu.
Luồng rẽ nhánh	- Nhập số âm → Cảnh báo lỗi validate.

Bảng 2.12. Đặc tả Use Case "Quản lý Ngôn ngữ lập trình"

Bảng 2.13. Đặc tả Use Case "Quản lý Sandbox"

Mục	Nội dung
Tên Use Case	Quản lý Sandbox
Tác nhân	Admin
Mô tả	Theo dõi, test kết nối, kill tiến trình Sandbox bị treo.
Tiền điều kiện	Sandbox Server đang bật.
Hậu điều kiện	Tiến trình zombie bị diệt, hàng đợi được thông.
Luồng sự kiện chính	1. Vào Quản lý Sandbox. 2. Bấm Test Connection. 3. Xem danh sách tiến trình. 4. Bấm "Kill" tiến trình treo.
Luồng rẽ nhánh	- Sandbox mất kết nối → Báo lỗi offline.

Bảng 2.13. Đặc tả Use Case "Quản lý Sandbox"

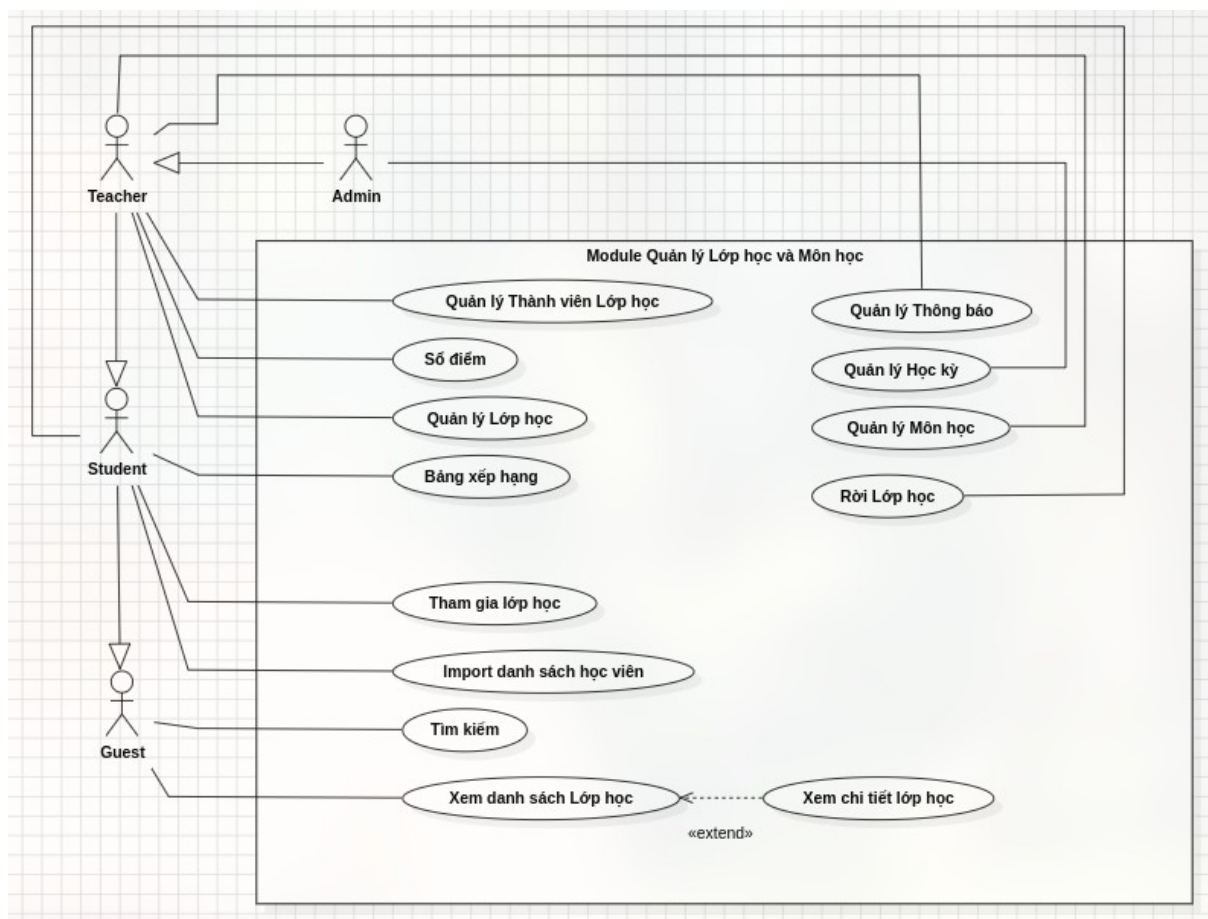
Bảng 2.14. Đặc tả Use Case "Quản lý Cấu hình & Logs"

Mục	Nội dung
Tên Use Case	Quản lý Cấu hình & Lịch sử (Logs)
Tác nhân	Admin
Mô tả	Xem Audit logs và cài đặt chung.
Tiền điều kiện	Đăng nhập Admin.
Hậu điều kiện	N/A (Chủ yếu là thao tác xem/read).
Luồng sự kiện chính	1. Vào System Logs. 2. Lọc log theo ngày, hành động, mức độ.
Luồng rẽ nhánh	- Log quá lớn → Tự động phân trang hoặc tải báo cáo CSV.

Bảng 2.14. Đặc tả Use Case "Quản lý Cấu hình & Logs"

2.2.4.3. Biểu đồ Use Case Module Quản lý Lớp học và Môn học

Mô tả: Vận hành và tổ chức lớp. Teacher tạo lớp, quản lý thành viên, đăng thông báo, sổ điểm. Student tham gia lớp và xem điểm.



Hình 2.5. Biểu đồ Use Case Module Quản lý Lớp học và Môn học

Danh sách chức năng chi tiết:

Tìm kiếm Lớp học

Tham gia Lớp học (Join)

Quản lý Lớp học (Tạo, sửa, xóa)

Quản lý Thành viên (Duyệt, xóa)

Quản lý Môn học & Học kỳ

Bảng tin (Stream) & Thông báo

Sổ điểm & Bảng xếp hạng

Đặc tả chi tiết các Use Case:

Bảng 2.15. Đặc tả Use Case "Tìm kiếm Lớp học"

Mục	Nội dung
Tên Use Case	Tìm kiếm Lớp học

Mục	Nội dung
Tác nhân	Student
Mô tả	Tìm lớp bằng Auto-suggest.
Tiền điều kiện	Đã đăng nhập.
Hậu điều kiện	Hiển thị kết quả lớp học.
Luồng sự kiện chính	1. Nhập từ khóa vào ô tìm kiếm. 2. Hệ thống gọi API auto-suggest. 3. Hiển thị danh sách khớp.
Luồng rẽ nhánh	- Không tìm thấy → Báo "Không có kết quả".

Bảng 2.15. Đặc tả Use Case "Tìm kiếm Lớp học"

Bảng 2.16. Đặc tả Use Case "Tham gia Lớp học"

Mục	Nội dung
Tên Use Case	Tham gia Lớp học (Join)
Tác nhân	Student
Mô tả	Tham gia qua Mã Code hoặc Link mời.
Tiền điều kiện	Mã/Link hợp lệ.
Hậu điều kiện	Student trở thành thành viên (Member) của lớp.
Luồng sự kiện chính	1. Nhập Mã lớp hoặc click Link mời. 2. Hệ thống xác thực mã. 3. Thêm bản ghi vào `classroom_members`. 4. Chuyển vào trang Bảng tin lớp.
Luồng rẽ nhánh	- Mã sai/hết hạn → Báo lỗi mã không hợp lệ.

Bảng 2.16. Đặc tả Use Case "Tham gia Lớp học"

Bảng 2.17. Đặc tả Use Case "Quản lý Lớp học"

Mục	Nội dung
Tên Use Case	Quản lý Lớp học
Tác nhân	Teacher
Mô tả	Tạo, sửa, xóa lớp học do mình phụ trách.
Tiền điều kiện	Có quyền Teacher.
Hậu điều kiện	Lớp được cập nhật trong DB.
Luồng sự kiện chính	1. Bấm Tạo lớp. 2. Nhập Tên lớp, Mô tả. 3. Nhấn Lưu. 4. Hệ thống sinh mã Invite Code và lưu.
Luồng rẽ nhánh	- Thiếu Tên lớp → Form yêu cầu nhập lại.

Bảng 2.17. Đặc tả Use Case "Quản lý Lớp học"

Bảng 2.18. Đặc tả Use Case "Quản lý Thành viên"

Mục	Nội dung
Tên Use Case	Quản lý Thành viên
Tác nhân	Teacher
Mô tả	Duyệt, xóa sinh viên khỏi lớp.
Tiền điều kiện	Lớp đang có sinh viên tham gia.
Hậu điều kiện	Danh sách sinh viên được cập nhật.
Luồng sự kiện chính	1. Vào Tab Thành viên. 2. Bấm Xóa/Duyệt user. 3. Hệ thống đổi trạng thái `status` trong DB.
Luồng rẽ nhánh	- N/A

Bảng 2.18. Đặc tả Use Case "Quản lý Thành viên"

Bảng 2.19. Đặc tả Use Case "Import danh sách học viên"

Mục	Nội dung
Tên Use Case	Import danh sách học viên
Tác nhân	Teacher, Admin
Mô tả	Thêm hàng loạt học viên qua file Excel/CSV.
Tiền điều kiện	File đúng định dạng.
Hậu điều kiện	Các học viên được tự động gán vào lớp.
Luồng sự kiện chính	1. Chọn Import. 2. Upload file CSV. 3. Hệ thống đọc và đối chiếu Email. 4. Lưu vào CSDL và báo số lượng thành công.
Luồng rẽ nhánh	- Lỗi sai định dạng → Từ chối file. - User chưa đăng ký tài khoản → List ra danh sách lỗi.

Bảng 2.19. Đặc tả Use Case "Import danh sách học viên"

Bảng 2.20. Đặc tả Use Case "Quản lý Môn học & Học kỳ"

Mục	Nội dung
Tên Use Case	Quản lý Môn học & Học kỳ
Tác nhân	Admin, Teacher
Mô tả	Thêm môn, gán môn vào học kỳ.
Tiền điều kiện	Đăng nhập quyền hợp lệ.
Hậu điều kiện	Môn/Học kỳ được lưu.
Luồng sự kiện chính	1. Vào Quản lý Môn. 2. Khai báo Môn mới. 3. Gán Môn vào Lớp học. 4. Lưu.
Luồng rẽ nhánh	- Trùng mã môn → Báo lỗi.

Bảng 2.20. Đặc tả Use Case "Quản lý Môn học & Học kỳ"

Bảng 2.21. Đặc tả Use Case "Bảng tin & Thông báo"

Mục	Nội dung
Tên Use Case	Quản lý Bảng tin
Tác nhân	Teacher
Mô tả	Đăng bài, ghim bài quan trọng trong lớp.
Tiền điều kiện	Lớp học đã được tạo.
Hậu điều kiện	Bài viết xuất hiện trên luồng sự kiện (Stream).
Luồng sự kiện chính	1. Ở trang chủ lớp, nhập thông báo. 2. Đính kèm file (nếu có). 3. Bấm Đăng. 4. Hệ thống đẩy Post lên Stream.
Luồng rẽ nhánh	- Nội dung rỗng → Không cho bấm Đăng.

Bảng 2.21. Đặc tả Use Case "Bảng tin & Thông báo"

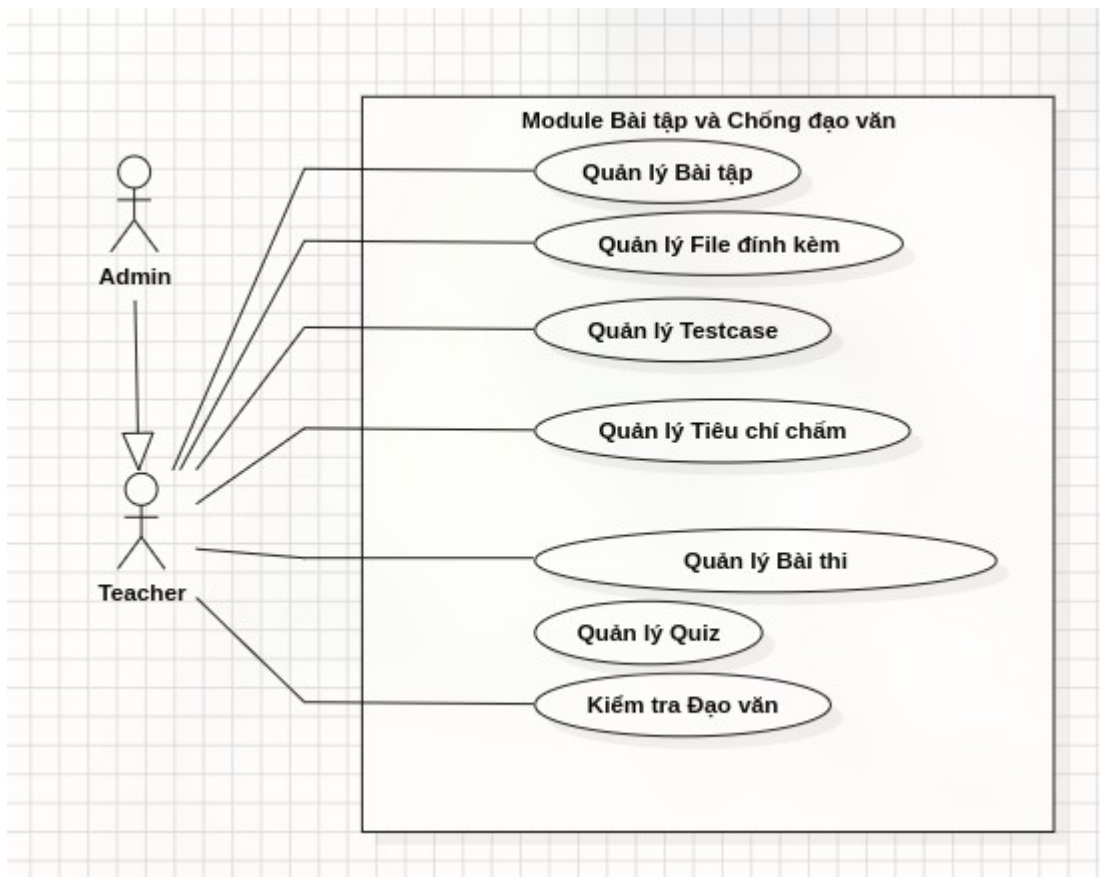
Bảng 2.22. Đặc tả Use Case "Sổ điểm & Bảng xếp hạng"

Mục	Nội dung
Tên Use Case	Sổ điểm & Bảng xếp hạng
Tác nhân	Teacher, Student
Mô tả	Hiển thị điểm thi và thứ hạng. Teacher có thể Export CSV.
Tiền điều kiện	Đã có bài thi được chấm điểm.
Hậu điều kiện	Hiển thị Grid điểm số trực quan.
Luồng sự kiện chính	1. Vào Tab Sổ điểm. 2. Hệ thống tính toán tổng điểm từ các Assignments. 3. Xếp hạng giảm dần. 4. Teacher bấm "Xuất Excel".
Luồng rẽ nhánh	- Lớp chưa có bài tập → Bảng rỗng.

Bảng 2.22. Đặc tả Use Case "Sổ điểm & Bảng xếp hạng"

2.2.4.4. Biểu đồ Use Case Module Bài tập và Chồng đạo văn

Mô tả: Hỗ trợ ra đa dạng các loại bài tập (Lập trình, Trắc nghiệm, Upload file), thiết lập Testcase, tiêu chí Rubric và quét đạo văn mã nguồn.



Hình 2.6. Biểu đồ Use Case Module Bài tập và Chồng đạo văn

Danh sách chức năng chi tiết:

Tạo bài tập/bài thi

Quản lý Bài tập/Bài thi (Sửa, xóa, nhân bản)

Quản lý File đính kèm

Quản lý Testcase

Quản lý Tiêu chí chấm (Rubric)

Quản lý Quiz

Kiểm tra Đạo văn (Plagiarism)

Đặc tả chi tiết các Use Case:

Bảng 2.23. Đặc tả Use Case "Tạo bài tập/bài thi"

Mục	Nội dung
Tên Use Case	Tạo bài tập/bài thi
Tác nhân	Teacher
Mô tả	Tạo đề bài (Code/Quiz/File) với thời gian và thang điểm.
Tiền điều kiện	Đang quản lý lớp học.
Hậu điều kiện	Bài tập được tạo và hiển thị lên Stream lớp.
Luồng sự kiện chính	1. Chọn Tạo bài tập. 2. Nhập Tiêu đề, Mô tả (Markdown), Hạn nộp. 3. Lưu và chuyển sang cấu hình Testcase/Quiz.
Luồng rẽ nhánh	- Thiếu tiêu đề → Cảnh báo.

Bảng 2.23. Đặc tả Use Case "Tạo bài tập/bài thi"

Bảng 2.24. Đặc tả Use Case "Quản lý Bài tập/Bài thi"

Mục	Nội dung
Tên Use Case	Quản lý Bài tập/Bài thi
Tác nhân	Teacher
Mô tả	Sửa, Xóa, Nhân bản (Clone) bài tập.
Tiền điều kiện	Bài tập đã được tạo.
Hậu điều kiện	Bài tập được cập nhật dữ liệu.

Mục	Nội dung
Luồng sự kiện chính	1. Chọn Menu thao tác ở góc bài tập. 2. Chọn Clone/Sửa. 3. Lưu.
Luồng rẽ nhánh	- Đang có học viên thi → Chặn thao tác Sửa/Xóa.

Bảng 2.24. Đặc tả Use Case "Quản lý Bài tập/Bài thi"

Bảng 2.25. Đặc tả Use Case "Quản lý File đính kèm"

Mục	Nội dung
Tên Use Case	Quản lý File đính kèm
Tác nhân	Teacher, Student
Mô tả	Teacher up tài liệu, Student tải xuống.
Tiền điều kiện	Có bài tập.
Hậu điều kiện	File được lưu Storage và gán vào ID bài tập.
Luồng sự kiện chính	1. Teacher chọn Upload file cho bài tập. 2. Sinh viên vào bài tập, nhấn Download.
Luồng rẽ nhánh	- Vượt dung lượng cho phép → Lỗi upload.

Bảng 2.25. Đặc tả Use Case "Quản lý File đính kèm"

Bảng 2.26. Đặc tả Use Case "Quản lý Testcase"

Mục	Nội dung
Tên Use Case	Quản lý Testcase
Tác nhân	Teacher
Mô tả	Thêm, sửa, xóa, import hàng loạt Input/Output mẫu.

Mục	Nội dung
Tiền điều kiện	Bài tập loại Lập trình (Code).
Hậu điều kiện	Bộ Testcase được gắn vào Bài tập.
Luồng sự kiện chính	1. Mở Cấu hình Testcase. 2. Nhập thủ công hoặc Import file ZIP. 3. Lưu hệ số điểm (weight) từng case. 4. Đánh dấu Ẩn/Hiện.
Luồng rẽ nhánh	- File ZIP không đúng cấu trúc thư mục → Báo lỗi định dạng.

Bảng 2.26. Đặc tả Use Case "Quản lý Testcase"

Bảng 2.27. Đặc tả Use Case "Quản lý Tiêu chí chấm (Rubric)"

Mục	Nội dung
Tên Use Case	Quản lý Rubric
Tác nhân	Teacher
Mô tả	Tạo các thang điểm chấm bài Tự luận.
Tiền điều kiện	Bài tập chưa cấu hình Rubric.
Hậu điều kiện	Rubric được lưu và phục vụ màn hình chấm tay.
Luồng sự kiện chính	1. Tạo Tiêu chí (vd: Format Code, Thuật toán). 2. Định mức điểm tối đa. 3. Lưu lại.
Luồng rẽ nhánh	- Tổng điểm Rubric khác điểm bài tập → Hiện cảnh báo.

Bảng 2.27. Đặc tả Use Case "Quản lý Tiêu chí chấm (Rubric)"

Bảng 2.28. Đặc tả Use Case "Quản lý Quiz"

Mục	Nội dung
Tên Use Case	Quản lý Quiz
Tác nhân	Teacher
Mô tả	Tạo câu hỏi, đáp án, Import ngân hàng câu hỏi.

Mục	Nội dung
Tiền điều kiện	Bài tập loại Quiz.
Hậu điều kiện	Bộ câu hỏi được tạo.
Luồng sự kiện chính	1. Nhập câu hỏi, Thêm lựa chọn (A,B,C,D). 2. Tick chọn đáp án Đúng. 3. Lưu lại.
Luồng rẽ nhánh	- Chưa chọn đáp án đúng nào → Không cho lưu.

Bảng 2.28. Đặc tả Use Case "Quản lý Quiz"

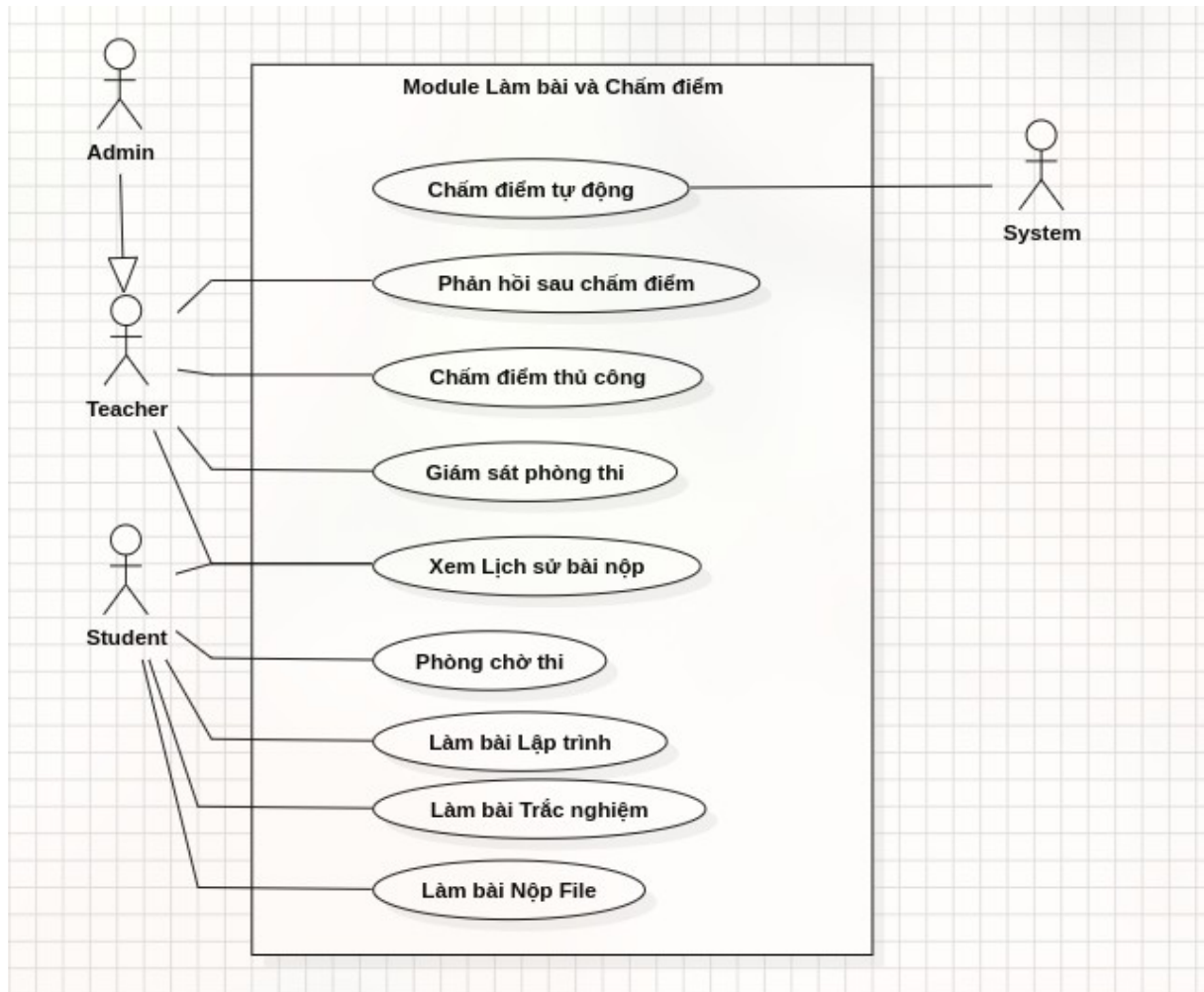
Bảng 2.29. Đặc tả Use Case "Kiểm tra Đạo văn"

Mục	Nội dung
Tên Use Case	Kiểm tra Đạo văn (Plagiarism)
Tác nhân	Teacher
Mô tả	Tự động quét và đối chiếu toàn bộ các bài nộp code.
Tiền điều kiện	Có từ 2 sinh viên nộp bài trở lên.
Hậu điều kiện	Hiển thị bảng tỷ lệ % gian lận và Highlight code.
Luồng sự kiện chính	1. Nhấn nút Quét Đạo Văn. 2. Hệ thống gọi thuật toán so sánh chuỗi (Moss). 3. Trả về kết quả các cặp sinh viên giống nhau. 4. Click vào để xem code đối chiếu 2 cột.
Luồng rẽ nhánh	- Chưa có bài nộp → Nút bị mờ (Disable).

Bảng 2.29. Đặc tả Use Case "Kiểm tra Đạo văn"

2.2.4.5. Biểu đồ Use Case Module Làm bài và Chấm điểm (Submissions & Grading)

Mô tả: Trải nghiệm thi, biên dịch mã, chấm tự động qua Sandbox hoặc chấm tay qua Rubric.



Hình 2.7. Biểu đồ Use Case Module Làm bài và Chấm điểm

Danh sách chức năng chi tiết:

Làm bài Lập trình (IDE)

Làm bài Trắc nghiệm (Quiz)

Làm bài Nộp File (Upload)

Phòng chờ thi (Exam Lobby)

Giám sát phòng thi (Exam Monitor)

Chấm điểm tự động (Auto-grade)

Chấm điểm thủ công (Manual-grade)

Xem Lịch sử bài nộp

Đặc tả chi tiết các Use Case:

Bảng 2.30. Đặc tả Use Case "Làm bài Lập trình (IDE)"

Mục	Nội dung
Tên Use Case	Làm bài trên IDE trực tuyến
Tác nhân	Student
Mô tả	Viết code, chạy thử và nộp bài.
Tiền điều kiện	Đang trong hạn nộp bài.
Hậu điều kiện	Mã nguồn gửi cho Sandbox chấm và lưu DB.
Luồng sự kiện chính	1. Mở IDE trên web. 2. Viết Code. 3. Bấm "Run" để test nhanh nghiệm. 4. Bấm "Submit" để nộp. 5. Hệ thống khóa IDE, đợi kết quả.
Luồng rẽ nhánh	- Bấm Submit khi quá hạn → Báo hết giờ nộp bài.

Bảng 2.30. Đặc tả Use Case "Làm bài Lập trình (IDE)"

Bảng 2.31. Đặc tả Use Case "Làm bài Trắc nghiệm"

Mục	Nội dung
Tên Use Case	Làm bài Trắc nghiệm (Quiz)
Tác nhân	Student
Mô tả	Trả lời trắc nghiệm, có auto-save.
Tiền điều kiện	Thời gian mở Quiz bắt đầu.
Hậu điều kiện	Lưu câu trả lời, tự động chấm điểm ngay sau khi nộp.

Luồng sự kiện chính	1. Tick chọn đáp án. 2. Hệ thống Auto-save mỗi 5s. 3. Bấm Nộp bài (hoặc hết giờ tự nộp). 4. Tính điểm.
Luồng rẽ nhánh	- Mất mạng → Auto-save lưu lại local, tự đồng bộ khi có mạng lại.

Bảng 2.31. Đặc tả Use Case "Làm bài Trắc nghiệm"

Bảng 2.32. Đặc tả Use Case "Làm bài Nộp File"

Mục	Nội dung
Tên Use Case	Làm bài Nộp File
Tác nhân	Student
Mô tả	Nộp báo cáo PDF/Word/Zip.
Tiền điều kiện	Đang trong hạn nộp.
Hậu điều kiện	File được Upload thành công.
Luồng sự kiện chính	1. Bấm Chọn file. 2. Chọn file đúng định dạng. 3. Bấm Nộp. 4. Hiện thị link file nộp.
Luồng rẽ nhánh	- File sai định dạng cấu hình bài tập → Báo lỗi không cho tải lên.

Bảng 2.32. Đặc tả Use Case "Làm bài Nộp File"

Bảng 2.33. Đặc tả Use Case "Phòng chờ thi"

Mục	Nội dung
Tên Use Case	Phòng chờ thi (Exam Lobby)
Tác nhân	Student
Mô tả	Màn hình đếm ngược chờ đến giờ thi.
Tiền điều kiện	Đã vào link bài thi nhưng chưa đến giờ mở.
Hậu điều kiện	Hết giờ đếm ngược sẽ tự mở đề.

Luồng sự kiện chính	1. Vào link. 2. Xem đồng hồ đếm ngược. 3. Hết giờ, trang tự Refresh và hiện nút "Bắt đầu làm bài".
Luồng rẽ nhánh	- N/A

Bảng 2.33. Đặc tả Use Case "Phòng chờ thi"

Bảng 2.34. Đặc tả Use Case "Giám sát phòng thi"

Mục	Nội dung
Tên Use Case	Giám sát phòng thi (Exam Monitor)
Tác nhân	Teacher
Mô tả	Theo dõi trạng thái thi realtime, ép nộp/cộng giờ.
Tiền điều kiện	Bài thi đang diễn ra.
Hậu điều kiện	Áp dụng thay đổi thời gian/trạng thái cho sinh viên.
Luồng sự kiện chính	1. Mở trang Giám sát. 2. Xem các chấm màu (Đang làm/Đã nộp/Offline). 3. Click vào user, chọn "Force Submit" hoặc "Add Time".
Luồng rẽ nhánh	- Sinh viên đã nộp bài → Ấn nút ép nộp.

Bảng 2.34. Đặc tả Use Case "Giám sát phòng thi"

Bảng 2.35. Đặc tả Use Case "Chấm điểm tự động"

Mục	Nội dung
Tên Use Case	Chấm điểm tự động (Auto-grade)
Tác nhân	Hệ thống (Sandbox)
Mô tả	Biên dịch và chạy code để chấm điểm.
Tiền điều kiện	Message queue nhận lệnh chấm bài.

Mục	Nội dung
Hậu điều kiện	Cập nhật điểm, memory, runtime từng testcase vào DB.
Luồng sự kiện chính	1. Daemon kéo bài từ Queue. 2. Đẩy vào Docker biên dịch. 3. Chạy các file Testcase. 4. Compare output. 5. Lưu kết quả Pass/Fail.
Luồng rẽ nhánh	- Code dính Infinite loop → Báo Time Limit Exceeded (TLE). - Compile lỗi → Báo Compile Error (CE).

Bảng 2.35. Đặc tả Use Case "Chấm điểm tự động"

Bảng 2.36. Đặc tả Use Case "Chấm điểm thủ công"

Mục	Nội dung
Tên Use Case	Chấm điểm thủ công (Manual-grade)
Tác nhân	Teacher
Mô tả	Chấm bằng Rubric, comment trực tiếp vào code.
Tiền điều kiện	Có bài nộp.
Hậu điều kiện	Điểm được lưu, sinh viên thấy nhận xét.
Luồng sự kiện chính	1. Mở bài sinh viên. 2. Click Rubric chấm điểm. 3. Bôi đen dòng code, gõ Comment. 4. Nhấn Hoàn tất chấm.
Luồng rẽ nhánh	- N/A

Bảng 2.36. Đặc tả Use Case "Chấm điểm thủ công"

Bảng 2.37. Đặc tả Use Case "Xem Lịch sử bài nộp"

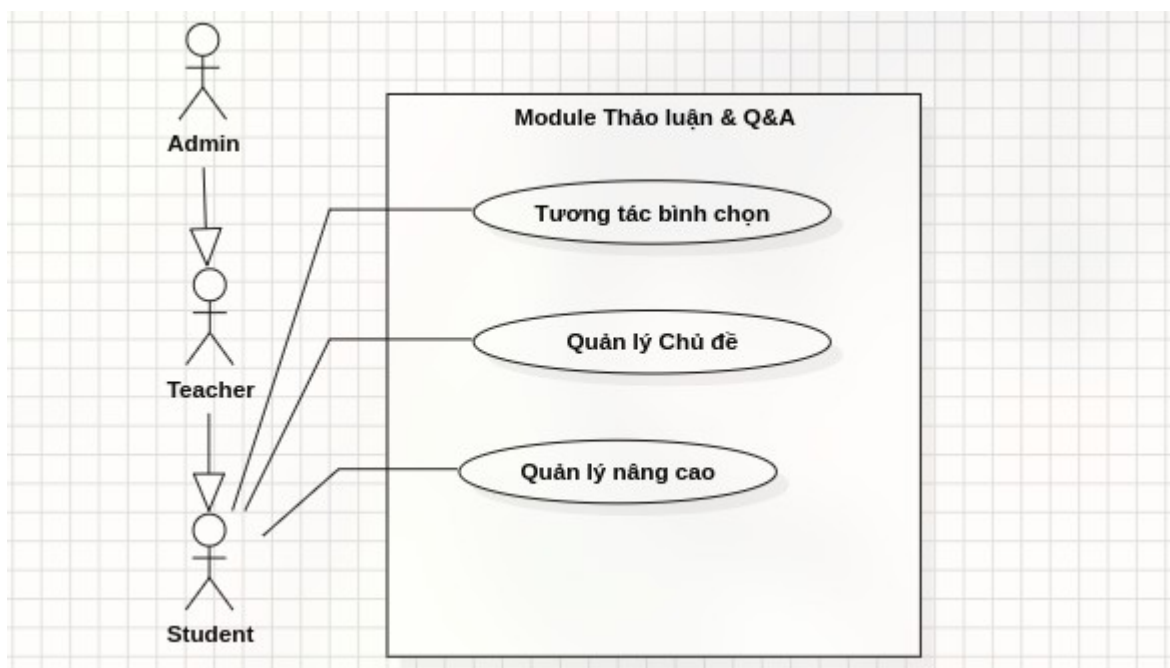
Mục	Nội dung
Tên Use Case	Xem Lịch sử bài nộp
Tác nhân	Student, Teacher

Mục	Nội dung
Mô tả	Xem lại toàn bộ các lần đã Submit.
Tiền điều kiện	Có ít nhất 1 lần Submit.
Hậu điều kiện	Hiển thị bảng chi tiết các lần nộp.
Luồng sự kiện chính	1. Nhấn tab Lịch sử. 2. Xem thời gian nộp, tổng điểm. 3. Click chi tiết để xem code đã nộp.
Luồng rẽ nhánh	- N/A

Bảng 2.37. Đặc tả Use Case "Xem Lịch sử bài nộp"

2.2.4.6. Biểu đồ Use Case Module Thảo luận & Q&A

Mô tả: Không gian tương tác giải đáp thắc mắc chuyên sâu cho từng chủ đề bài tập/môn học.



Hình 2.8. Biểu đồ Use Case Module Thảo luận & Q&A

Danh sách chức năng chi tiết:

Quản lý Chủ đề

Tương tác bình chọn (Voting)

Quản lý nâng cao

Đặc tả chi tiết các Use Case:

Bảng 2.38. Đặc tả Use Case "Quản lý Chủ đề"

Mục	Nội dung
Tên Use Case	Quản lý Chủ đề
Tác nhân	Teacher, Student
Mô tả	Tạo bài đăng hỏi đáp, sửa, xóa.
Tiền điều kiện	Đã đăng nhập.
Hậu điều kiện	Bài viết hiển thị trên Forum.
Luồng sự kiện chính	1. Chọn Tạo Thảo luận. 2. Gõ câu hỏi. 3. Bấm Đăng.
Luồng rẽ nhánh	- Xóa bài người khác (Student) → Không có quyền.

Bảng 2.38. Đặc tả Use Case "Quản lý Chủ đề"

Bảng 2.39. Đặc tả Use Case "Tương tác bình chọn"

Mục	Nội dung
Tên Use Case	Tương tác bình chọn (Voting)
Tác nhân	Teacher, Student
Mô tả	Vote Up/Down bài viết.
Tiền điều kiện	Có bài viết tồn tại.
Hậu điều kiện	Điểm Vote thay đổi trong DB.
Luồng sự kiện chính	1. Click biểu tượng mũi tên lên/xuống. 2. Hệ thống cộng/trừ 1 điểm.
Luồng rẽ nhánh	- Click lần 2 vào nút đã vote → Hủy vote.

Bảng 2.39. Đặc tả Use Case "Tương tác bình chọn"

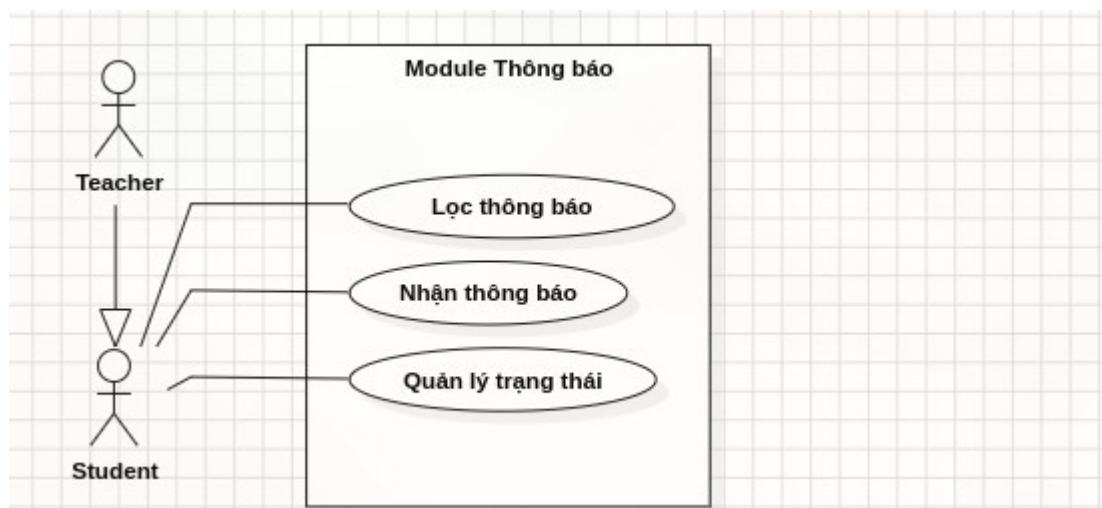
Bảng 2.40. Đặc tả Use Case "Quản lý nâng cao"

Mục	Nội dung
Tên Use Case	Quản lý nâng cao
Tác nhân	Teacher, Tác giả bài đăng
Mô tả	Mark as answer, Ghim bài.
Tiền điều kiện	Bài viết có câu trả lời.
Hậu điều kiện	Bài được ghim lên Top, hoặc câu trả lời hiện màu xanh (Accepted).
Luồng sự kiện chính	1. Tác giả câu hỏi nhấn dấu tick vào reply đúng. 2. Teacher nhấn "Pin" để ghim bài hỏi lên đầu forum.
Luồng rẽ nhánh	- N/A

Bảng 2.40. Đặc tả Use Case "Quản lý nâng cao"

2.2.4.7. Biểu đồ Use Case Module Thông báo

Mô tả: Hệ thống Alert đẩy thông tin các sự kiện quan trọng đến người dùng kịp thời.



Hình 2.9. Biểu đồ Use Case Module Thông báo

Danh sách chức năng chi tiết:

Nhận thông báo

Lọc thông báo

Quản lý trạng thái

Đặc tả chi tiết các Use Case:

Bảng 2.41. Đặc tả Use Case "Nhận thông báo"

Mục	Nội dung
Tên Use Case	Nhận thông báo
Tác nhân	Teacher, Student
Mô tả	Tự động nhận Alert khi có sự kiện (Có điểm, có comment...).
Tiền điều kiện	Sự kiện phát sinh.
Hậu điều kiện	Icon Quả chuông hiển thị badge màu đỏ (số lượng).
Luồng sự kiện chính	1. Sự kiện xảy ra (vd có người comment). 2. Hệ thống tạo record Notification. 3. Giao diện fetch và hiện chấm đỏ.
Luồng rẽ nhánh	- N/A

Bảng 2.41. Đặc tả Use Case "Nhận thông báo"

Bảng 2.42. Đặc tả Use Case "Lọc thông báo"

Mục	Nội dung
Tên Use Case	Lọc thông báo
Tác nhân	Teacher, Student
Mô tả	Xem theo Tab Tất cả / Chưa đọc.
Tiền điều kiện	Đã mở Panel thông báo.
Hậu điều kiện	Danh sách hiển thị theo bộ lọc.
Luồng sự kiện chính	1. Click icon chuông. 2. Chọn Tab "Chưa đọc". 3. Hệ thống render danh sách `is_read = false`.

Mục	Nội dung
Luồng rẽ nhánh	- Hết thông báo chưa đọc → Hiện "Bạn không có thông báo mới".

Bảng 2.42. Đặc tả Use Case "Lọc thông báo"

Bảng 2.43. Đặc tả Use Case "Quản lý trạng thái"

Mục	Nội dung
Tên Use Case	Quản lý trạng thái thông báo
Tác nhân	Teacher, Student
Mô tả	Đánh dấu đã đọc.
Tiền điều kiện	Có thông báo chưa đọc.
Hậu điều kiện	Trạng thái chuyển thành `is_read = true`, icon chuông giảm số lượng.
Luồng sự kiện chính	1. Click vào thông báo cụ thể hoặc nhấn "Đánh dấu đã đọc tất cả". 2. Giao diện update trạng thái, tắt nền xám của thông báo.
Luồng rẽ nhánh	- N/A

Bảng 2.43. Đặc tả Use Case "Quản lý trạng thái"

2.2.5. Xây dựng sơ đồ Class (Class Diagram)

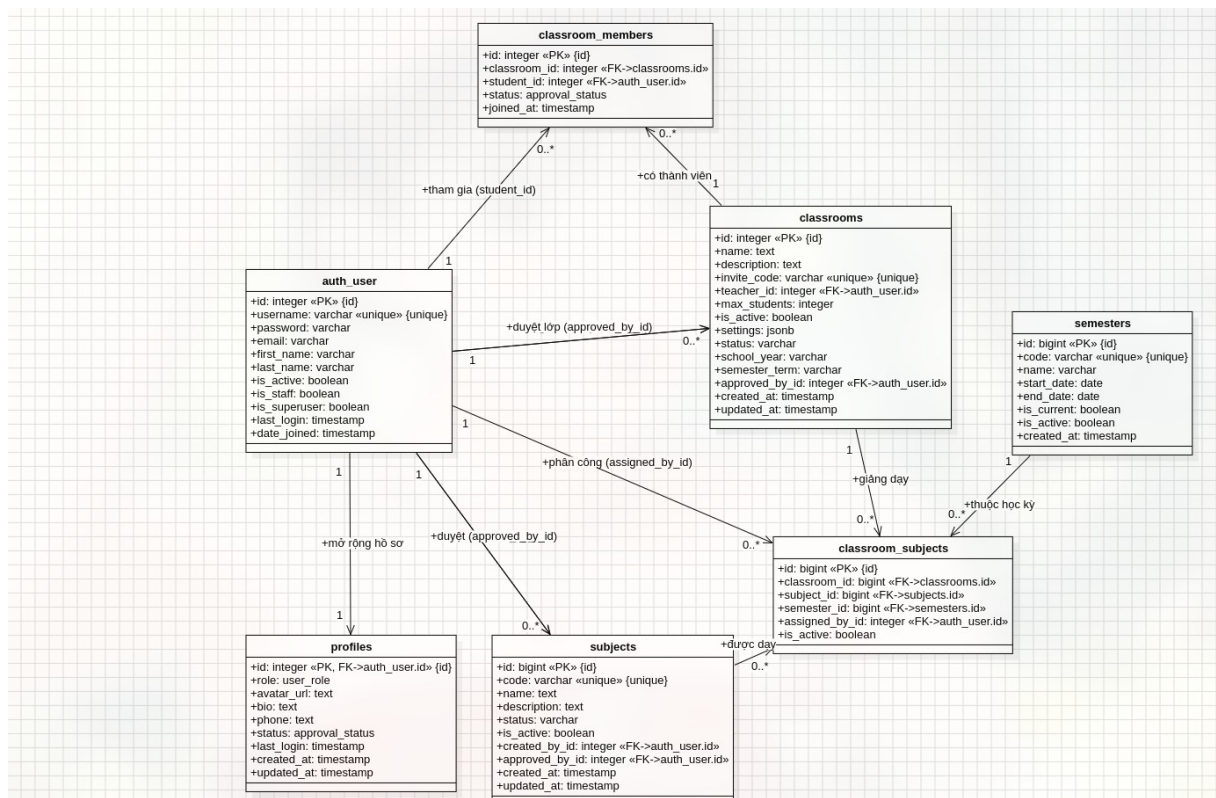
Do hệ thống Website dạy và học lập trình có quy mô dữ liệu lớn, sơ đồ lớp được chia thành 5 module chính để đảm bảo tính trực quan và dễ theo dõi. Sơ đồ mô tả cấu trúc dữ liệu, các thuộc tính, phương thức và mối quan hệ (kế thừa, liên kết 1-N, N-N) giữa các thực thể. Trước khi đi vào từng module chi tiết, hệ thống có hai lớp (class) cốt lõi đóng vai trò trung tâm và liên kết với hầu hết các phân hệ khác.

Nhóm các Lớp (Class) nền tảng dùng chung:

Lớp `auth\_user`: Là trung tâm quản lý danh tính người dùng (Admin, Teacher, Student...). Lớp này lưu trữ các thông tin cơ bản như tên đăng nhập, mật khẩu, email và đóng vai trò cấp quyền truy cập. Nó xuất hiện ở mọi module để xác định ai là người tạo lớp, ai làm bài, ai gửi thảo luận.

Lớp `assignments`: Là cốt lõi của luồng học tập. Lớp này lưu thông tin về bài tập, bài thi, kỳ hạn, thang điểm và các ràng buộc làm bài. Từ lớp này sẽ phân nhánh ra các bảng testcase, quiz, bài nộp, và chấm điểm.

2.2.5.1. Sơ đồ lớp Module Lớp học & Môn học



Hình 2.10. Sơ đồ Class Module Lớp học & Môn học

Mô tả các lớp trong module:

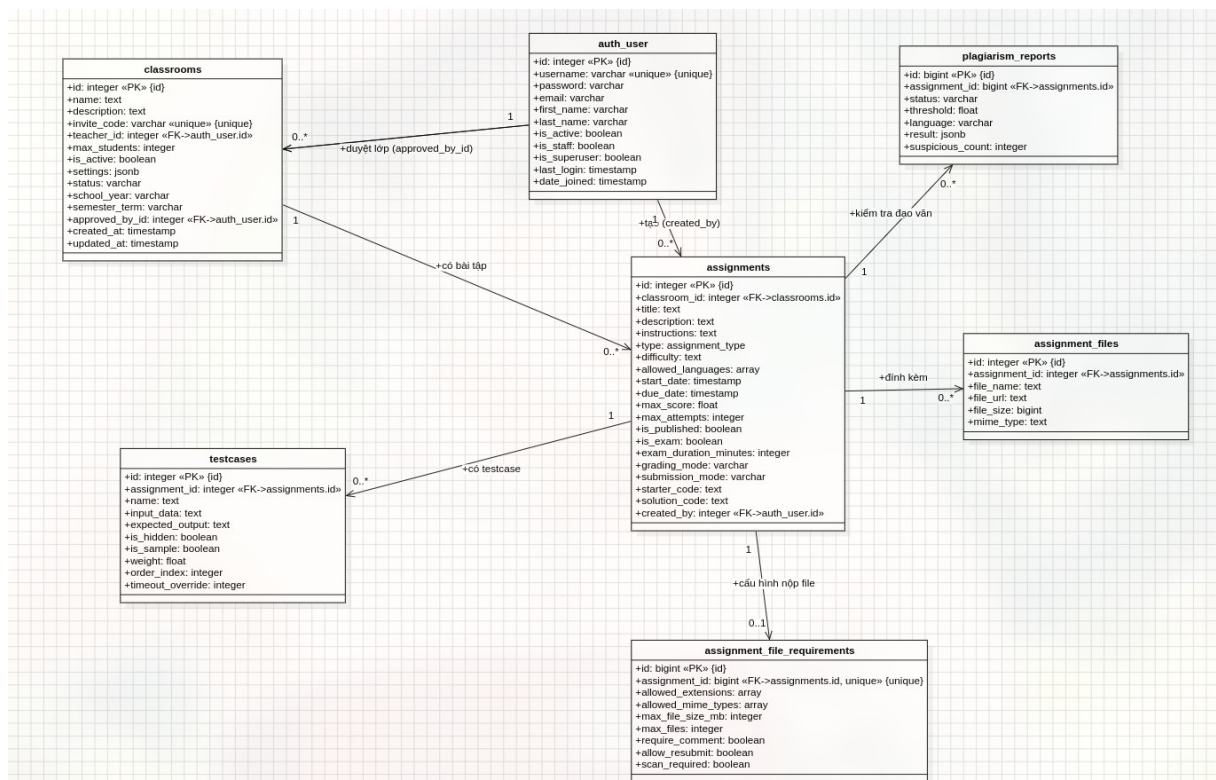
Lớp `profiles`: Mở rộng từ auth_user, lưu trữ thông tin cá nhân bổ sung của người dùng như ảnh đại diện (avatar_url), tiểu sử (bio), số điện thoại và trạng thái phê duyệt.

Lớp `classrooms`: Biểu diễn thực thể lớp học. Lưu trữ mã mời (invite_code), giáo viên phụ trách, số lượng sinh viên tối đa và năm học. Lớp này liên kết 1-N với classroom_members để quản lý danh sách sinh viên.

Lớp `classroom\_members`: Lớp trung gian thể hiện quan hệ N-N giữa sinh viên và lớp học. Lưu trữ trạng thái tham gia và thời gian gia nhập.

Lớp `subjects` & `semesters`: Quản lý danh mục môn học (mã môn, tên môn) và học kỳ (ngày bắt đầu, ngày kết thúc). Các môn học được gán vào lớp thông qua lớp trung gian classroom_subjects.

2.2.5.2 Sơ đồ lớp Module Bài tập & Chống đạo văn



Hình 2.11. Sơ đồ Class Module Bài tập & Chống đạo văn

Mô tả các lớp trong module:

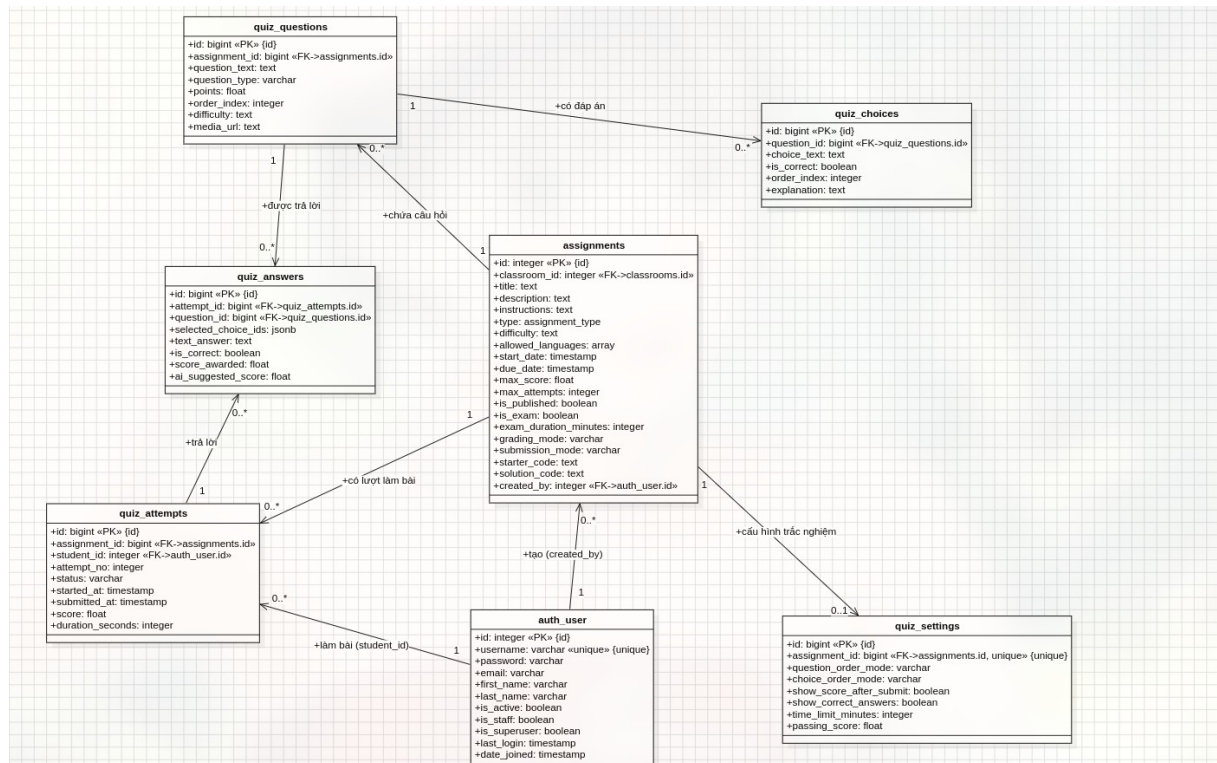
Lớp `testcases`: Lưu trữ bộ dữ liệu kiểm thử (Input/Output chuẩn) để chấm code tự động. Liên kết 1-N với assignments, cho phép phân bổ trọng số (weight) và tài nguyên (RAM, Timeout) cho từng testcase.

Lớp `assignment\_files`: Quản lý các file tài liệu đính kèm của đề bài mà giáo viên tải lên.

Lớp `assignment\_file\_requirements`: Lớp định nghĩa cấu hình ràng buộc khi sinh viên nộp bài dạng file, bao gồm kích thước file tối đa, định dạng cho phép và số lượng file.

Lớp `plagiarism\_reports`: Ghi nhận kết quả quét đạo văn của hệ thống. Lưu trữ tỷ lệ % đạo văn, số lượng bài nghi ngờ và nội dung chi tiết các đoạn mã nguồn trùng lặp giữa các sinh viên.

2.2.5.3. Sơ đồ lớp Module Thi Trắc nghiệm (Quiz)



Hình 2.12. Sơ đồ Class Module Thi Trắc nghiệm

Mô tả các lớp trong module:

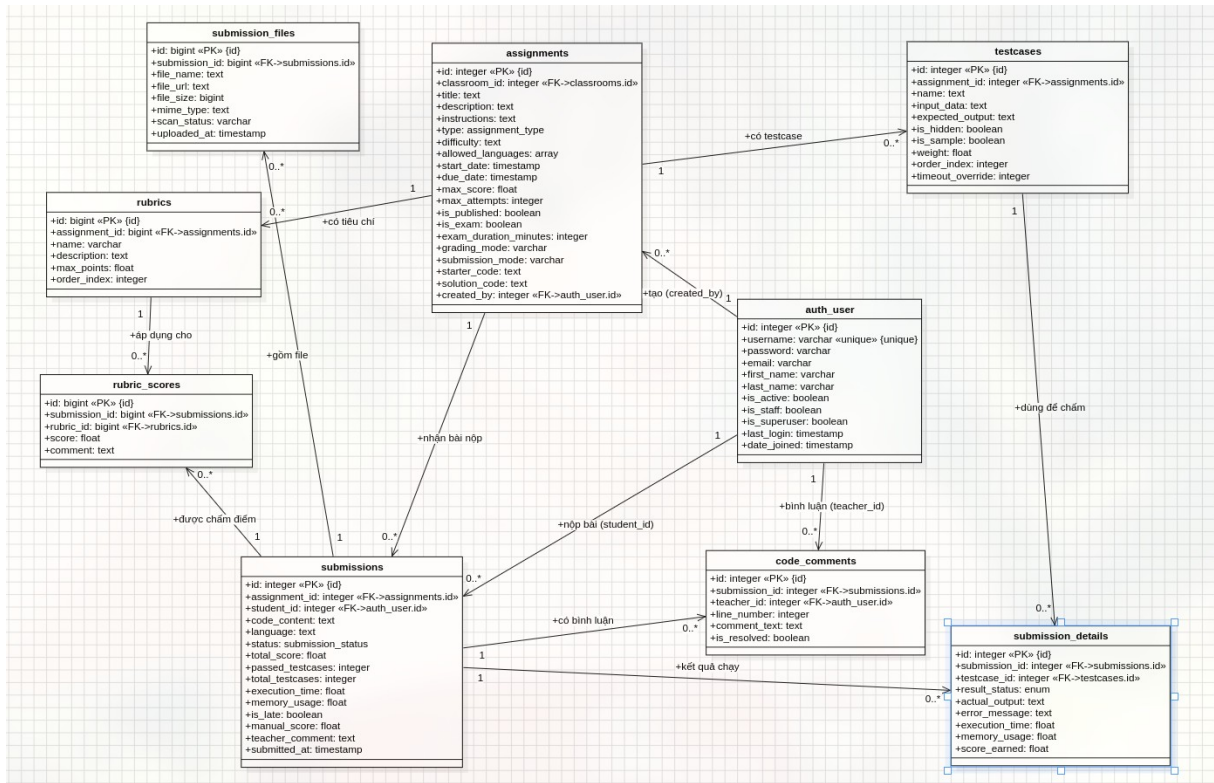
Lớp `quiz\_settings`: Lưu trữ cấu hình đặc thù của bài thi trắc nghiệm như chế độ đảo câu hỏi, hiển thị đáp án sau khi thi, và điểm chuẩn (passing score).

Lớp `quiz\_questions` & `quiz\_choices`: quiz_questions lưu trữ nội dung câu hỏi, hình ảnh đính kèm và điểm số. Mỗi câu hỏi liên kết 1-N với quiz_choices để định nghĩa các phương án lựa chọn (A, B, C, D) và đánh dấu phương án đúng.

Lớp `quiz\_attempts`: Ghi nhận một phiên thi trắc nghiệm của sinh viên, bao gồm thời gian bắt đầu, nộp bài, điểm số đạt được và IP thiết bị thi.

Lớp `quiz\_answers`: Lịch sử làm bài chi tiết. Ghi nhận chính xác sinh viên đã tick chọn những đáp án nào cho từng câu hỏi thông qua liên kết N-N quiz_answers_selected_choices.

2.2.5.4. Sơ đồ lớp Module Làm bài & Chấm điểm (Submissions)



Hình 2.13. Sơ đồ Class Module Làm bài & Chấm điểm

Mô tả các lớp trong module:

Lớp `submissions`: Lưu trữ phiên nộp bài của sinh viên, chứa mã nguồn (code_content), ngôn ngữ lập trình sử dụng, tổng điểm, và trạng thái chấm (Pending/Graded).

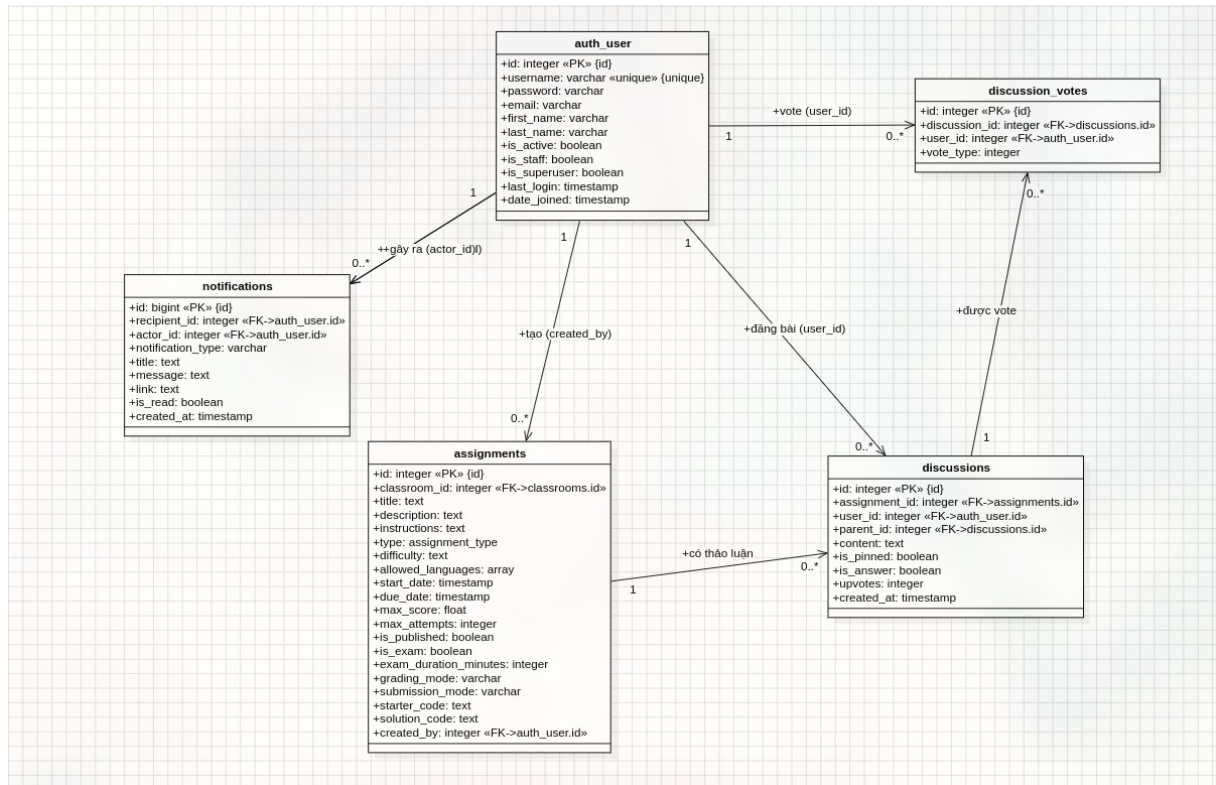
Lớp `submission\_details`: Chi tiết kết quả chạy máy chấm (Sandbox). Mỗi bản ghi tương ứng với một testcase, ghi nhận kết quả Pass/Fail, bộ nhớ RAM tiêu thụ, thời gian thực thi (execution_time) và log lỗi.

Lớp `submission\_files`: Quản lý các file thực tế mà sinh viên đã upload lên hệ thống đối với bài tập dạng nộp file.

Lớp `rubrics` & `rubric\_scores`: rubrics định nghĩa các tiêu chí chấm điểm tự luận (ví dụ: Hình thức 2đ, Nội dung 8đ). rubric_scores lưu điểm thực tế giáo viên đã chấm cho từng tiêu chí trên bài nộp.

Lớp `code\_comments`: Lưu nội dung mà giảng viên bôi đen và để lại bình luận trực tiếp trên dòng code (line_number) của sinh viên.

2.2.5.5. Sơ đồ lớp Module Tương tác (Discussions)



Hình 2.14. Sơ đồ Class Module Tương tác & Thông báo

Mô tả các lớp trong module:

Lớp `discussions`: Lưu trữ các bài đăng hỏi đáp của sinh viên và giảng viên. Lớp này hỗ trợ liên kết đệ quy (parent_id) để tạo thành các luồng bình luận (reply) xếp lồng vào nhau. Ghi nhận bài viết đã ghim (is_pinned) hoặc câu trả lời đúng (is_answer).

Lớp `discussion\_votes`: Ghi nhận lượt Upvote (+1) hoặc Downvote (-1) của người dùng để xếp hạng bài viết.

Lớp `notifications`: Quản lý luồng thông báo đẩy của hệ thống. Chứa nội dung, tiêu đề, liên kết, và trạng thái đã đọc (is_read) của thông báo được gửi đến từng user.

CHƯƠNG 3: KẾT QUẢ VÀ THẢO LUẬN

3.1. Kết Quả Triển Khai Giao Diện Và Chức Năng

Sau quá trình phân tích, thiết kế và lập trình, hệ thống "Website dạy và học lập trình trực tuyến" đã được hoàn thiện và triển khai thử nghiệm. Dưới đây là các kết quả đạt được, minh họa thông qua các giao diện chức năng cốt lõi của hệ thống, tập trung vào trải nghiệm người dùng và tính năng chuyên sâu.

Hệ thống hiện đã vận hành ổn định trên môi trường máy chủ, đảm bảo khả năng đáp ứng tốt và xử lý đồng thời nhiều tác vụ phức tạp như biên dịch mã nguồn cách ly (Sandbox) hay theo dõi thời gian thực (Real-time). Toàn bộ các luồng nghiệp vụ được xây dựng theo cấu trúc phân quyền chặt chẽ, tối ưu hóa giao diện cho ba nhóm người dùng chính: Quản trị viên, Giảng viên và Học viên.

Không chỉ đơn thuần là số hóa quy trình giao - nộp bài, ứng dụng còn tích hợp sâu các công cụ hỗ trợ thông minh nhằm mang lại một môi trường giáo dục tương tác, an toàn, công bằng và tiện lợi nhất.

3.1.1 Chức năng đăng nhập và xác thực

- Để có thể truy cập và sử dụng các dịch vụ của hệ thống dưới tư cách là Học viên hay Giảng viên, người dùng bắt buộc phải trải qua bước xác thực danh tính. Hệ thống cung cấp hai phương thức linh hoạt: tạo tài khoản mới thông qua biểu mẫu đăng ký truyền thống hoặc sử dụng tính năng đăng nhập nhanh bằng tài khoản Google.

The image shows a login form titled "Chào mừng trở lại" (Welcome back) with the subtitle "Đăng nhập để tiếp tục học lập trình" (Log in to continue learning programming). At the top is a blue icon with a white code symbol. Below the title is a button labeled "Đăng nhập với Google" (Log in with Google). A horizontal line with the word "hoặc" (or) follows. The form then asks for "Tên đăng nhập" (Username) and "Mật khẩu" (Password). There is a "Ghi nhớ đăng nhập" (Remember me) checkbox and a "Quên mật khẩu?" (Forgot password?) link. A large blue "Đăng nhập" (Log in) button is at the bottom, with a link "Chưa có tài khoản? Đăng ký" (Don't have an account? Sign up) below it.

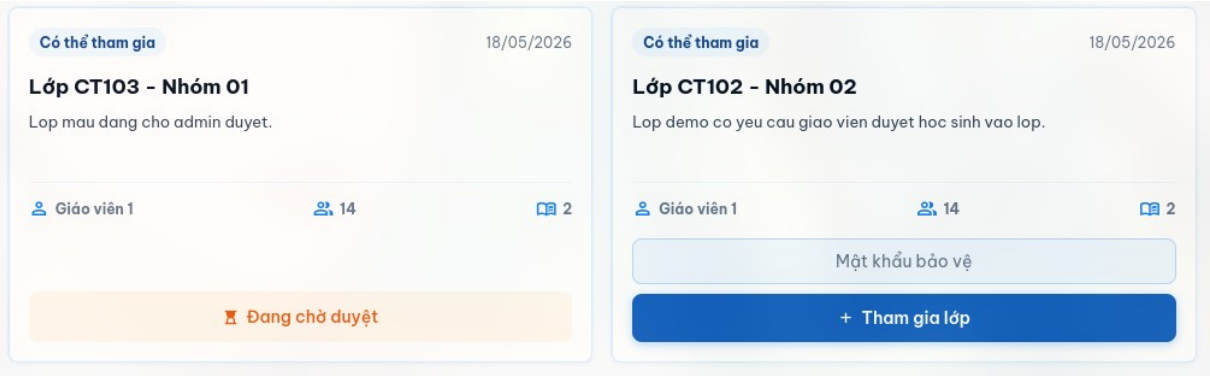
Hình 3.1. Giao diện form đăng nhập

- Việc yêu cầu đăng nhập(Hình 3.1) không chỉ giúp bảo mật dữ liệu mà còn là cơ sở để hệ thống tiến hành phân quyền. Chỉ khi cung cấp đúng thông tin tài khoản hợp lệ, người dùng mới được cấp phép tiến vào không gian làm việc chính, từ đó truy cập vào các chức năng cốt lõi như tham gia lớp học, làm bài tập, chấm điểm hay theo dõi tiến độ học tập dựa trên đúng vai trò mà mình được cấp.

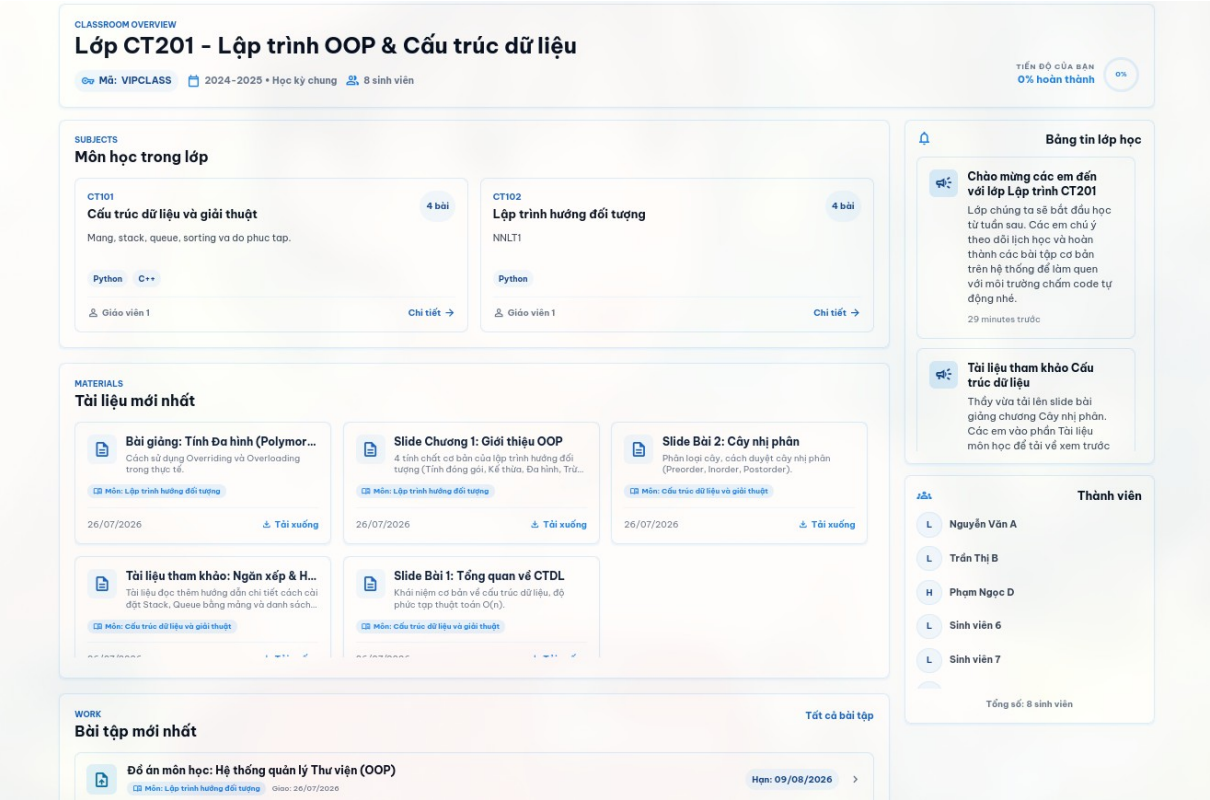
-

3.1.2 Tham gia lớp học đối với học sinh

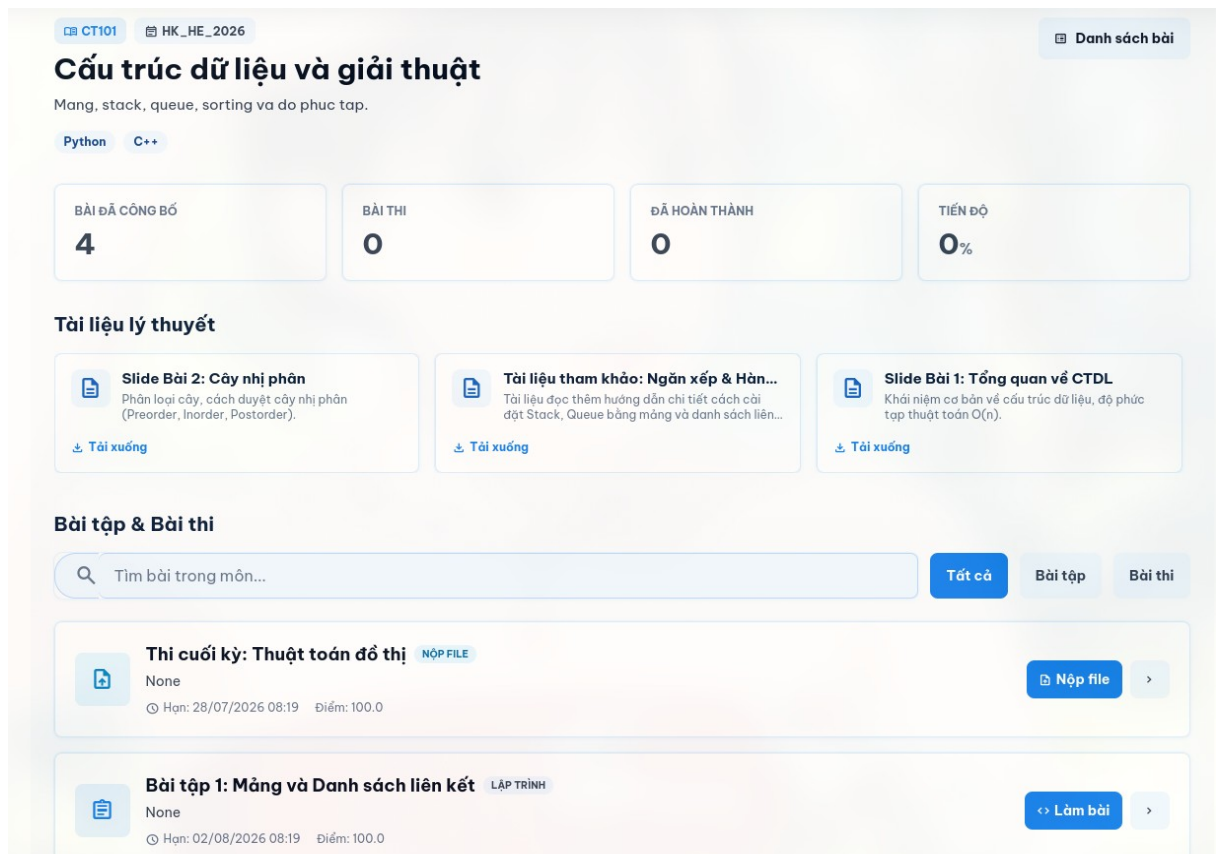
- Học sinh khi là người dùng mới thì mặc định chưa tham gia lớp học nào, do đó học sinh có thể tham gia lớp học thông qua việc nhập mã lớp hoặc gửi yêu cầu xin vào lớp từ danh sách các lớp học hiện có. Trong mỗi lớp học sẽ có sẵn các môn học do giảng viên tạo từ trước. Kèm theo đó là các bài tập được phân bổ theo từng môn để học sinh làm bài, qua đó giảng viên có thể theo dõi, đánh giá năng lực của từng cá nhân và cuối cùng là cho học sinh làm bài thi để đánh giá khi kết thúc mỗi môn học.



Hình 3.2. Tham gia lớp học với mật khẩu



Hình 3.3. Giao diện lớp học



Hình 3.4. Giao diện môn học

- Khi muốn tham gia vào một lớp học, học sinh cần phải nhập mật khẩu bảo vệ. Nếu nhập đúng mật khẩu, tài khoản học sinh sẽ được đưa vào trạng thái chờ duyệt (Hình 3.2). Ở trang chính của lớp học (Hình 3.3), học sinh có thể xem được các môn học thuộc lớp, các tài liệu mới nhất của môn học, bài tập mới nhất, bảng tin lớp học (nơi giảng viên đăng các thông báo), và cuối cùng là danh sách thành viên của lớp học.

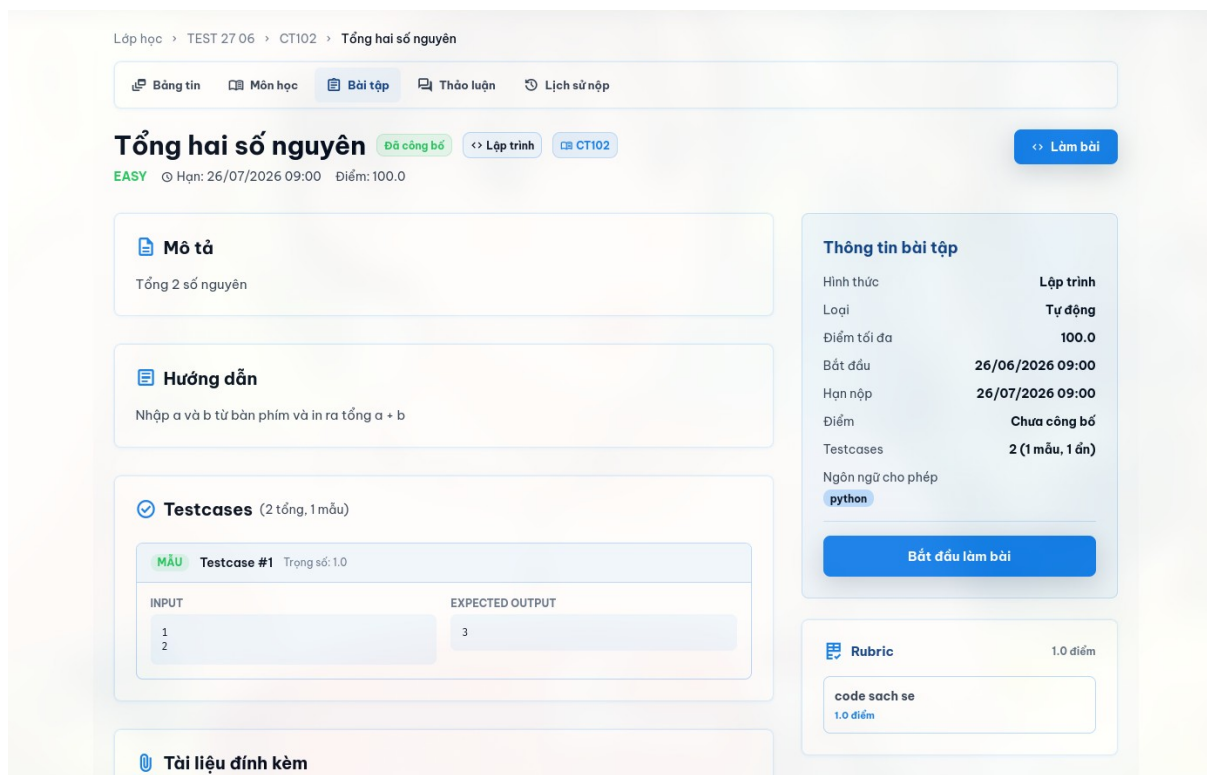
- Khi truy cập vào một môn học cụ thể (Hình 3.4) trong lớp, hệ thống sẽ hiển thị danh sách các file lý thuyết và các bài tập. Học sinh có thể dễ dàng tải các file tài liệu này xuống thiết bị cá nhân hoặc nhấn xem trực tiếp ngay trên website.

3.1.3 Làm bài tập

- Sau khi đã trở thành thành viên chính thức của lớp học, học sinh sẽ có quyền truy cập vào danh sách các bài tập và bài kiểm tra do giảng viên giao phó theo từng môn học. Nhằm đáp ứng đa dạng các phương pháp kiểm tra đánh giá, hệ thống hiện tại hỗ trợ ba hình thức làm bài chính: thực hành viết mã nguồn, làm bài trắc nghiệm và nộp tệp tin.

Tương ứng với mỗi hình thức, hệ thống cung cấp một không gian làm bài chuyên biệt, được thiết kế tối ưu với các công cụ tự động hóa (như trình soạn thảo Web IDE tích hợp sẵn, đồng hồ đếm ngược, cơ chế chấm điểm tức thì) để mang lại trải nghiệm tốt nhất. Thông qua quá trình làm bài và nhận phản hồi trực tiếp từ hệ thống, học sinh có

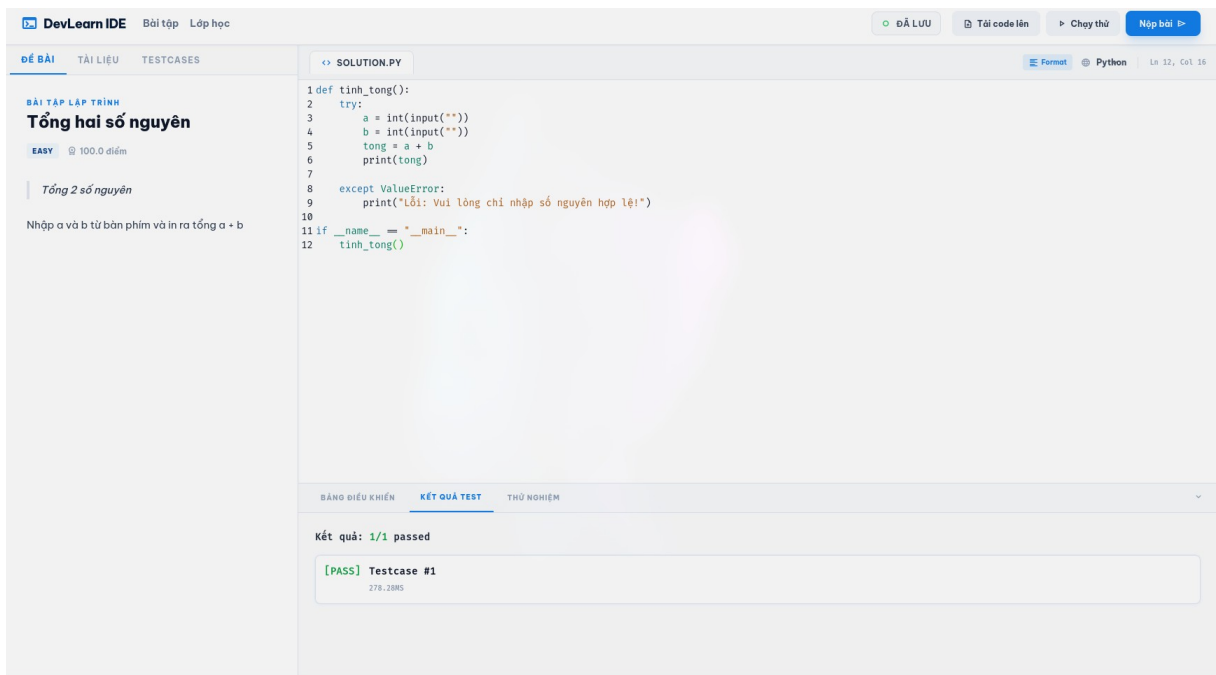
thể chủ động củng cố kiến thức, đồng thời giúp giảng viên có được cơ sở dữ liệu trực quan để theo dõi tiến độ và đánh giá chính xác năng lực của từng cá nhân.



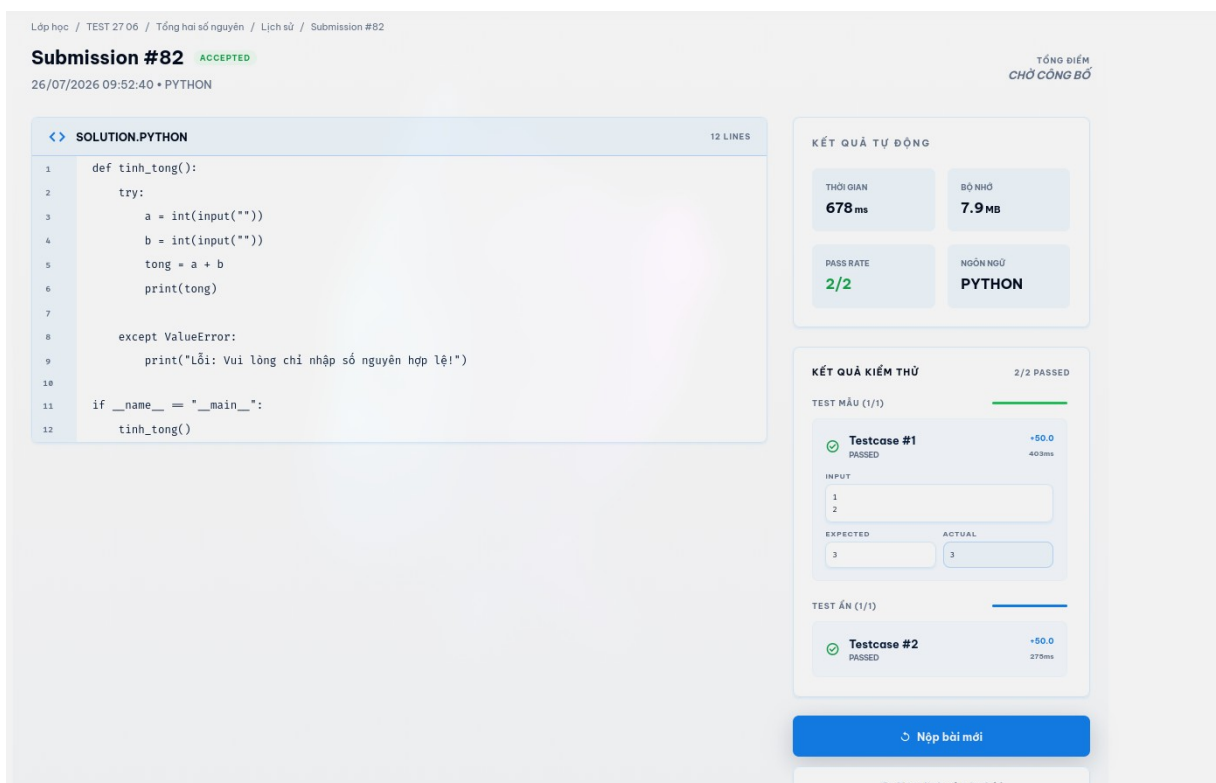
Hình 3.5. Giao diện bài tập

- Khi học sinh nhấn chọn một bài tập cụ thể, hệ thống sẽ chuyển hướng đến trang giao diện Chi tiết bài tập (Hình 3.5). Tại đây, học sinh có thể theo dõi trực quan toàn bộ các thông tin quan trọng nhất để chuẩn bị cho việc làm bài như tên bài tập, hình thức đánh giá (lập trình, làm trắc nghiệm, hoặc nộp tệp tin), tên môn học tương ứng, thời gian nộp bài, số điểm tối đa đạt được cùng mức độ khó của bài. Điểm nổi bật của giao diện này là phần mô tả và hướng dẫn được trình bày chi tiết ngay vị trí trung tâm nhằm giúp học sinh nắm bắt chính xác yêu cầu của giảng viên.

Đối với các bài tập mang hình thức nộp file, hệ thống liệt kê rõ ràng các ràng buộc kỹ thuật như định dạng file được phép tải lên và giới hạn dung lượng để ngăn ngừa sai sót. Đặc biệt, hệ thống được thiết lập cơ chế kiểm soát thời gian chặt chẽ: nếu học sinh truy cập hoặc cố gắng nộp bài sau khi thời hạn (deadline) đã kết thúc, hệ thống sẽ lập tức hiển thị cảnh báo lỗi quá hạn và khóa chức năng làm bài nhằm đảm bảo tính công bằng. Sau khi đã đọc kỹ mọi yêu cầu và đảm bảo thời gian vẫn còn hiệu lực, học sinh chỉ cần nhấn nút chức năng nổi bật ở góc phải màn hình để chính thức bắt đầu quá trình làm bài hoặc nộp bài của mình.



Hình 3.6. Giao diện bài tập lập trình



Hình 3.7. Giao diện kết quả

- Khi học sinh nhấn nút bắt đầu đối với một bài tập lập trình, hệ thống sẽ mở ra không gian làm bài chuyên dụng được tích hợp sẵn Web IDE (Hình 3.6). Giao diện này được phân chia bố cục trực quan, mang lại trải nghiệm thực hành liền mạch mà không

yêu cầu học sinh phải cài đặt phần mềm bên ngoài. Khu vực bên trái ghim cố định đề bài, các hướng dẫn chi tiết, đồng thời cho phép học sinh dễ dàng truy cập và xem trực tiếp các tài liệu đính kèm liên quan để có thêm tư liệu tham khảo ngay trong lúc làm bài. Cửa sổ bên phải là trình soạn thảo mã nguồn thông minh, hỗ trợ tô sáng cú pháp, tính năng định dạng lại mã nguồn (format code) để bài làm luôn chuẩn form, và linh hoạt lựa chọn ngôn ngữ lập trình theo đề bài. Đặc biệt, hệ thống còn hỗ trợ tính năng tiện ích tải lên tệp mã nguồn có sẵn (upload file), sinh viên chỉ cần đưa file chứa code tương ứng với ngôn ngữ đang làm, toàn bộ đoạn mã sẽ được tự động trích xuất và nạp thẳng lên trình IDE để các em có thể tiếp tục chỉnh sửa thay vì phải gõ lại từ đầu.

- Điểm cốt lõi của giao diện nằm ở bảng điều khiển bên dưới cùng hệ thống chấm điểm tự động. Học sinh có thể nhấn nút "Chạy code" để kiểm thử mã nguồn với các testcase mẫu. Ngay lập tức, hệ thống sẽ trả về phản hồi chi tiết, so sánh trực quan giữa kết quả thực tế (output) của học sinh với kết quả mong đợi (expected), đồng thời cung cấp các thông số về thời gian thực thi hay lỗi biên dịch nếu có. Đặc biệt, để tối ưu tài nguyên và ngăn chặn lỗi logic từ phía người dùng, hệ thống được tích hợp bộ bảo vệ nếu mã nguồn của học sinh rơi vào vòng lặp vô hạn, hệ thống sẽ tự động ngắt tiến trình sau 5 giây và in ra thông báo lỗi vượt quá giới hạn thời gian (Time Limit Exceeded). Cơ chế tương tác tức thì và an toàn này đóng vai trò như một môi trường tự rèn luyện, giúp học sinh nhanh chóng khắc phục lỗi trước khi nhấn nút "Nộp bài" để ghi nhận kết quả cuối cùng.

- Sau khi nộp bài thành công, hệ thống sẽ điều hướng học sinh đến trang Chi tiết kết quả (Hình 3.7). Tại đây, học sinh có thể theo dõi tổng quan số điểm đạt được trên thang điểm tối đa, kèm theo trạng thái đánh giá cuối cùng của hệ thống. Giao diện hỗ trợ lưu trữ và hiển thị nguyên vẹn toàn bộ mã nguồn mà học sinh đã nộp, giúp các em có thể dễ dàng xem lại, sao chép hoặc dùng làm tài liệu đối chiếu cho các lần nộp sau.

- Phần quan trọng và trực quan nhất của trang này là danh sách báo cáo chi tiết kết quả chạy của từng testcase. Để đảm bảo tính minh bạch trong việc chấm điểm nhưng vẫn tuân thủ nguyên tắc chống gian lận học thuật, hệ thống phân chia rõ ràng thành hai loại. Đối với các "Testcase mẫu", học sinh được quyền xem đầy đủ dữ liệu đầu vào (input) và đầu ra mong đợi (expected output) nhằm hiểu rõ lý do mã nguồn của mình chạy đúng hoặc sai. Ngược lại, đối với các "Testcase ẩn" (được thiết kế để thử thách tư duy sâu và các trường hợp ngoại lệ), hệ thống sẽ bảo mật hoàn toàn dữ liệu kiểm thử, chỉ hiển thị trạng thái đánh giá là Đạt (PASS) hoặc Trượt (FAIL) cùng thông số thời gian thực thi. Cơ chế này buộc học sinh phải tự rèn luyện kỹ năng phân tích thuật toán một cách toàn diện, thay vì chỉ viết code "đối phó" với các trường hợp có sẵn.

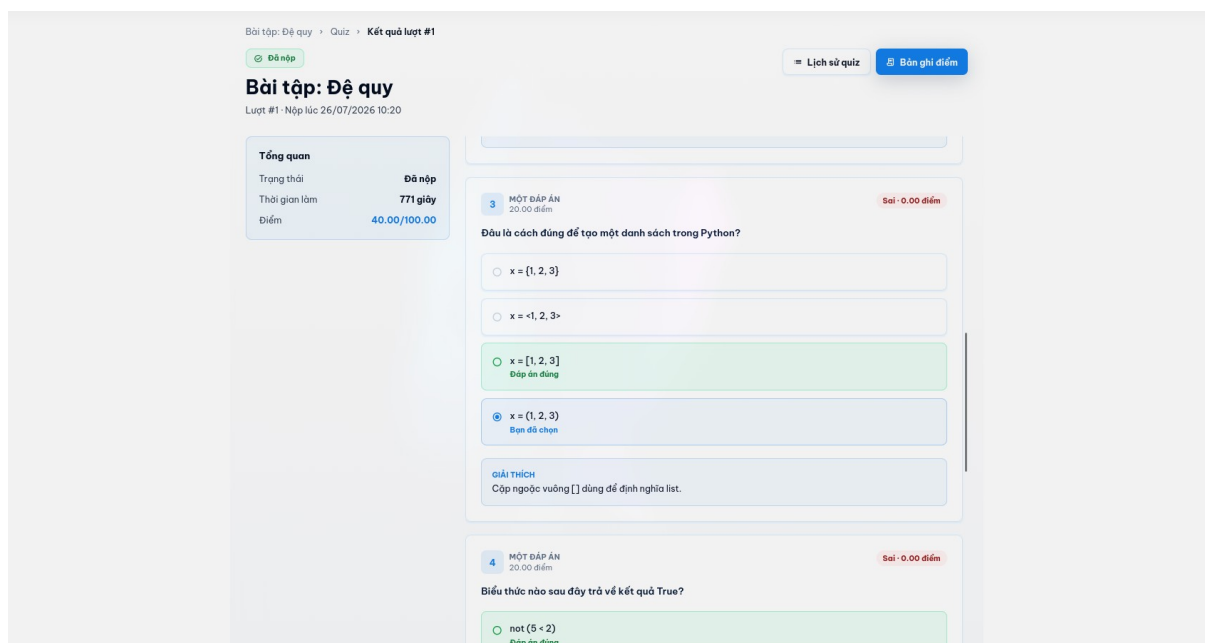
The screenshot shows a quiz interface with the following elements:

- Header:** "Luật #1" (Rule #1), "Chưa có thay đổi" (No changes), and a timer "59:50".
- Title:** "Bài tập: Đề quy" (Assignment: Recursion), with a subtitle "Tự lưu khi trả lời." (Save when answering).
- Question 1:** "Ký tự nào dùng để bắt đầu một dòng chú thích (comment) trong Python?" (Which character is used to start a line comment in Python?). Options: `//`, `<!--`, `/*`, `#`.
- Question 2:** "Kết quả của `type(10.5)` là gì?" (What is the result of `type(10.5)`?). Options: `<class 'int'>`, `<class 'list'>`, `<class 'str'>`, `<class 'float'>`.
- Progress:** "Tiến độ" (Progress) bar showing "0/5" questions answered.
- Buttons:** "Chưa trả lời" (Not answered) for each question, and a "Nộp bài" (Submit) button at the bottom right.

Hình 3.8. Giao diện làm bài trắc nghiệm

- Đối với các bài tập mang hình thức trắc nghiệm, hệ thống cung cấp một giao diện làm bài chuyên biệt (Hình 3.8) được thiết kế tối giản nhằm tối ưu hóa sự tập trung của sinh viên. Ở khu vực hiển thị chính, nội dung câu hỏi và các phương án trả lời (A, B, C, D...) được trình bày rõ ràng, trực quan. Để hỗ trợ học sinh quản lý tiến độ, giao diện tích hợp thêm một bảng điều hướng, cho phép người học phân biệt nhanh các câu hỏi đã chọn đáp án với các câu chưa làm thông qua màu sắc, đồng thời hỗ trợ di chuyển linh hoạt đến một câu hỏi bất kỳ chỉ bằng một thao tác nhấn chọn.

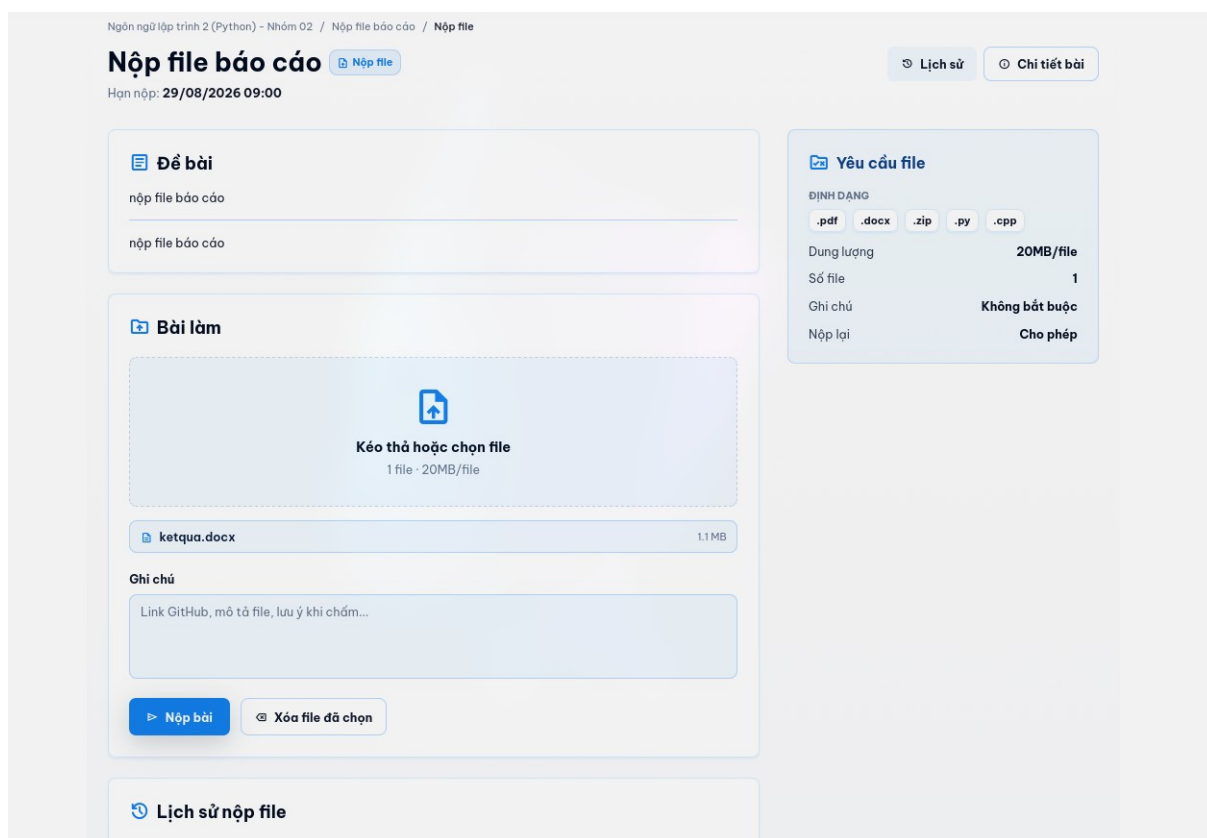
- Đặc biệt, đối với các bài kiểm tra có giới hạn thời gian, hệ thống sẽ ghim một đồng hồ đếm ngược (countdown timer) chạy theo thời gian thực ở góc màn hình. Khi đồng hồ điểm về 0, hệ thống sẽ tự động khóa thao tác và tự động thu bài nhằm đảm bảo tính công bằng và kỷ luật phòng thi. Trong trường hợp hoàn thành sớm, sinh viên có thể chủ động rà soát lại toàn bộ đáp án của mình trước khi nhấn nút "Nộp bài". Ngay sau khi nộp, hệ thống sẽ tự động đối chiếu với bộ đáp án gốc của giảng viên và tính toán điểm số một cách tức thì và chính xác tuyệt đối.



Hình 3.9. Giao diện kết quả trắc nghiệm

Ngay sau khi nộp bài trắc nghiệm thành công, hệ thống sẽ tự động xử lý và chuyển hướng sinh viên đến trang Kết quả chi tiết (Hình 3.9). Tại giao diện này, bảng tổng quan (dashboard) được đặt cố định ở góc phải, cung cấp ngay lập tức các thông số cốt lõi bao gồm: trạng thái bài nộp, thời gian hoàn thành bài thi, điểm số đạt được trên thang điểm tối đa, và mốc điểm chuẩn để sinh viên tự đối chiếu năng lực. Khu vực trung tâm của trang đóng vai trò như một bản báo cáo học tập, hiển thị lại toàn bộ câu hỏi kèm theo đánh giá tức thời của hệ thống.

Bằng cách ứng dụng mã màu trực quan tô xanh đáp án đúng, bôi đỏ đáp án sai, sinh viên có thể nhanh chóng nhận biết các lỗi sai của mình. Tùy thuộc vào thiết lập tùy chỉnh của giảng viên, hệ thống có thể mở khóa thêm phần hiển thị "Đáp án đúng" và khung "Giải thích" chi tiết bên dưới mỗi câu hỏi nhằm hỗ trợ sinh viên ôn tập kiến thức. Tuy nhiên, trong các kỳ thi kiểm tra nghiêm ngặt, giảng viên hoàn toàn có quyền vô hiệu hóa tính năng xem lại đáp án để ngăn ngừa rò rỉ đề thi và chống gian lận. Cơ chế phản hồi linh hoạt này không chỉ đảm bảo tính bảo mật học thuật mà còn nâng cao hiệu quả tự học của sinh viên.



Hình 3.10. Giao diện nộp bài tập dạng tệp tin

- Đối với các dạng bài tập yêu cầu sinh viên phải nộp báo cáo, bản thiết kế hoặc mã nguồn nén, hệ thống cung cấp một giao diện nộp tệp tin chuyên nghiệp và tối ưu hóa trải nghiệm (Hình 3.10).

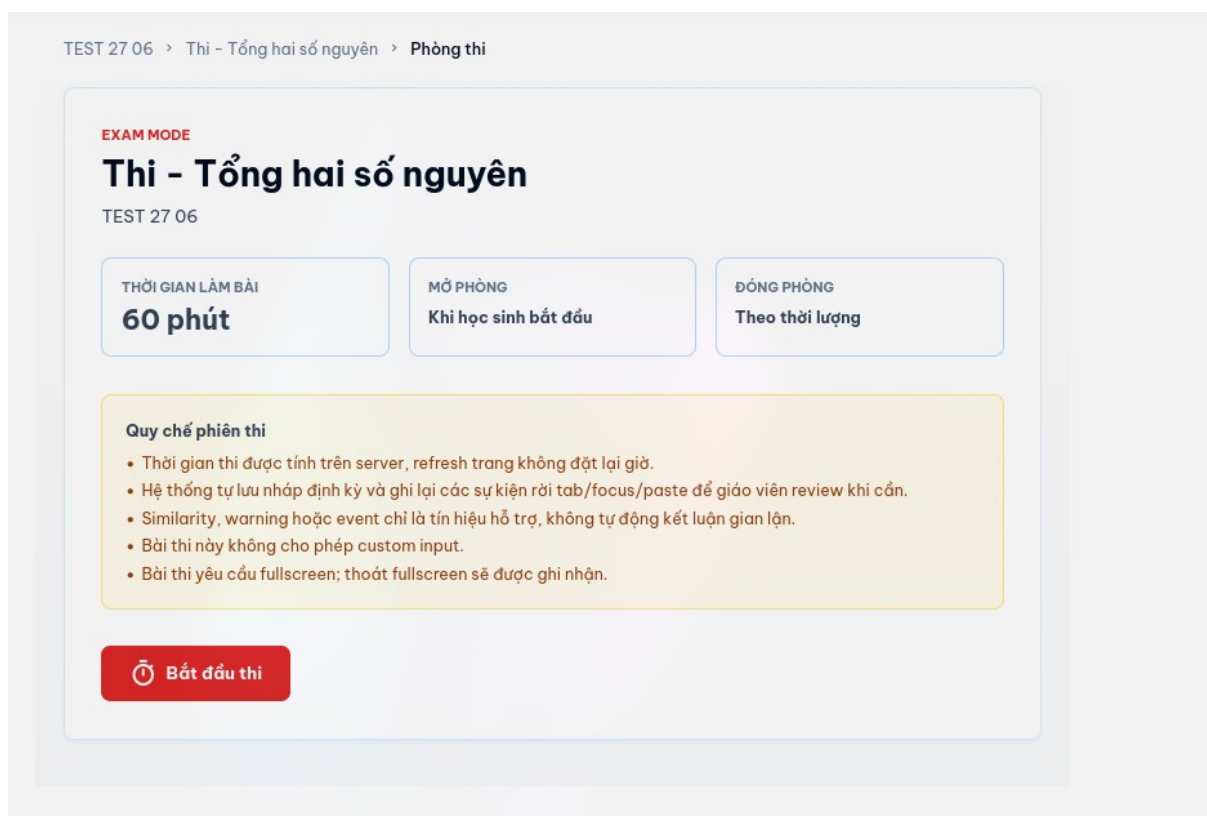
Tại vị trí trung tâm, hệ thống bố trí khu vực tải lên hỗ trợ thao tác kéo thả (drag-and-drop) hiện đại, giúp sinh viên có thể chọn và đính kèm file một cách nhanh chóng. Điểm cộng lớn của giao diện này là sự minh bạch trong quy trình nộp: hệ thống liệt kê rành mạch toàn bộ các ràng buộc kỹ thuật do giảng viên thiết lập, bao gồm danh sách các định dạng tệp được phép tải lên (ví dụ: .pdf, .docx, .zip), giới hạn dung lượng tối đa và tổng số file cho phép. Đặc biệt, thông qua cơ chế kiểm duyệt trực tiếp tại trình duyệt (client-side validation), hệ thống sẽ ngay lập tức ngăn chặn và phát cảnh báo nếu sinh viên tải lên file sai định dạng hoặc vượt quá kích thước quy định nhằm hạn chế tối đa các lỗi do bất cẩn.

- Bên cạnh đó, thông qua thanh điều hướng linh hoạt, sinh viên hoàn toàn có thể truy cập vào mục Lịch sử nộp bài để quản lý, đối chiếu lại các tệp tin đã từng nộp trong quá khứ hoặc tải bản gốc về thiết bị nếu cần thiết. Cuối cùng, giao diện còn cung cấp một khung văn bản bổ sung, cho phép sinh viên để lại ghi chú, thông điệp hoặc lời giải thích đính kèm gửi trực tiếp đến giảng viên. Sau khi hoàn tất kiểm tra, sinh viên chỉ cần

nhấn nút "Nộp bài" để hệ thống mã hóa và tải tệp tin lên máy chủ lưu trữ một cách an toàn và bảo mật.

3.1.4 Làm bài thi

- Bên cạnh các bài tập rèn luyện thông thường, phân hệ "Làm bài thi" được xây dựng nhằm mục đích đánh giá năng lực và mức độ tiếp thu kiến thức của sinh viên một cách khách quan, chính xác nhất. Với tính chất quan trọng của các bài kiểm tra định kỳ hay thi cuối kỳ, hệ thống đặt ưu tiên hàng đầu vào việc thiết lập một môi trường làm bài khép kín và nghiêm ngặt. Việc cách ly tối đa các yếu tố can thiệp từ bên ngoài không chỉ nhằm triệt tiêu các hành vi gian lận, bảo vệ tính công bằng, minh bạch của kết quả thi mà còn bắt buộc sinh viên phải phát huy tối đa năng lực tư duy độc lập trong điều kiện áp lực thời gian.



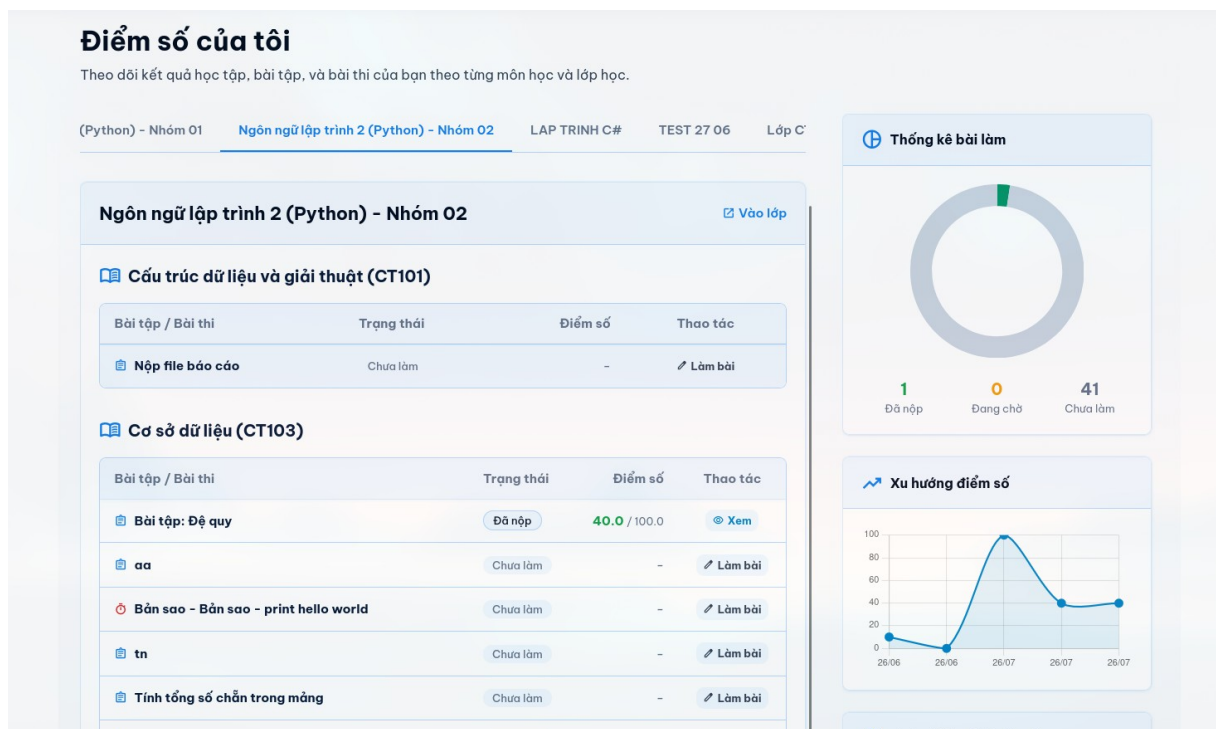
Hình 3.11. Giao diện xác nhận trước khi thi

- Đối với các bài kiểm tra được giảng viên kích hoạt chế độ thi (Exam Mode), sinh viên không thể truy cập trực tiếp vào không gian làm bài mà bắt buộc phải đi qua một phòng chờ bảo mật (Exam Lobby - Hình 3.11). Tại màn hình này, hệ thống hiển thị rõ ràng, minh bạch toàn bộ thông tin về thời gian thi, số điểm, cũng như các quy chế phòng thi nghiêm ngặt. Sinh viên bắt buộc phải đọc kỹ và thực hiện thao tác đánh dấu xác nhận cam kết không gian lận trước khi nút "Bắt đầu thi" được mở khóa.

- Điểm khác biệt cốt lõi của chế độ thi so với bài tập thông thường nằm ở các ràng buộc kỹ thuật khắt khe nhằm đảm bảo sự công bằng tối đa. Thứ nhất, đối với mọi hình thức thi (cả trắc nghiệm lẫn lập trình), mỗi sinh viên chỉ được cấp duy nhất một (1) lượt làm bài; ngay sau khi xác nhận nộp, hệ thống sẽ lập tức khóa vĩnh viễn quyền chỉnh sửa. Thứ hai, đối với riêng bài thi lập trình, nhằm loại bỏ triệt để hành vi sử dụng code làm sẵn hoặc chép bài từ bên ngoài, hệ thống sẽ tự động ẩn đi tính năng "Tải lên tệp mã nguồn". Hơn thế nữa, trình soạn thảo Web IDE trong phòng thi được cấu hình để chặn hoàn toàn mọi thao tác dán văn bản (Paste / Ctrl+V / Right-click). Cơ chế bảo mật đa tầng này buộc sinh viên phải tự vận động tư duy và tự tay gõ từng dòng lệnh từ đầu đến cuối, qua đó phản ánh chính xác nhất năng lực thực thụ của người học trong môi trường áp lực.

3.1.5 Điểm số Học sinh

- Đánh giá và theo dõi kết quả học tập là một mắt xích không thể thiếu trong quá trình đào tạo. Phân hệ "Điểm số học sinh" được thiết kế như một trung tâm dữ liệu cá nhân thông minh, nơi minh bạch hóa toàn bộ thành quả học tập của người học trên nền tảng. Không dừng lại ở việc liệt kê những điểm số khô khan, hệ thống còn cung cấp các công cụ thống kê và phân tích trực quan, giúp sinh viên có cái nhìn toàn cảnh về năng lực hiện tại của bản thân. Thông qua đó, người học có thể chủ động đánh giá điểm mạnh, điểm yếu để vạch ra kế hoạch cải thiện và điều chỉnh phương pháp học tập sao cho hiệu quả nhất.



Hình 3.12. Giao diện xem thống kê điểm số của học sinh

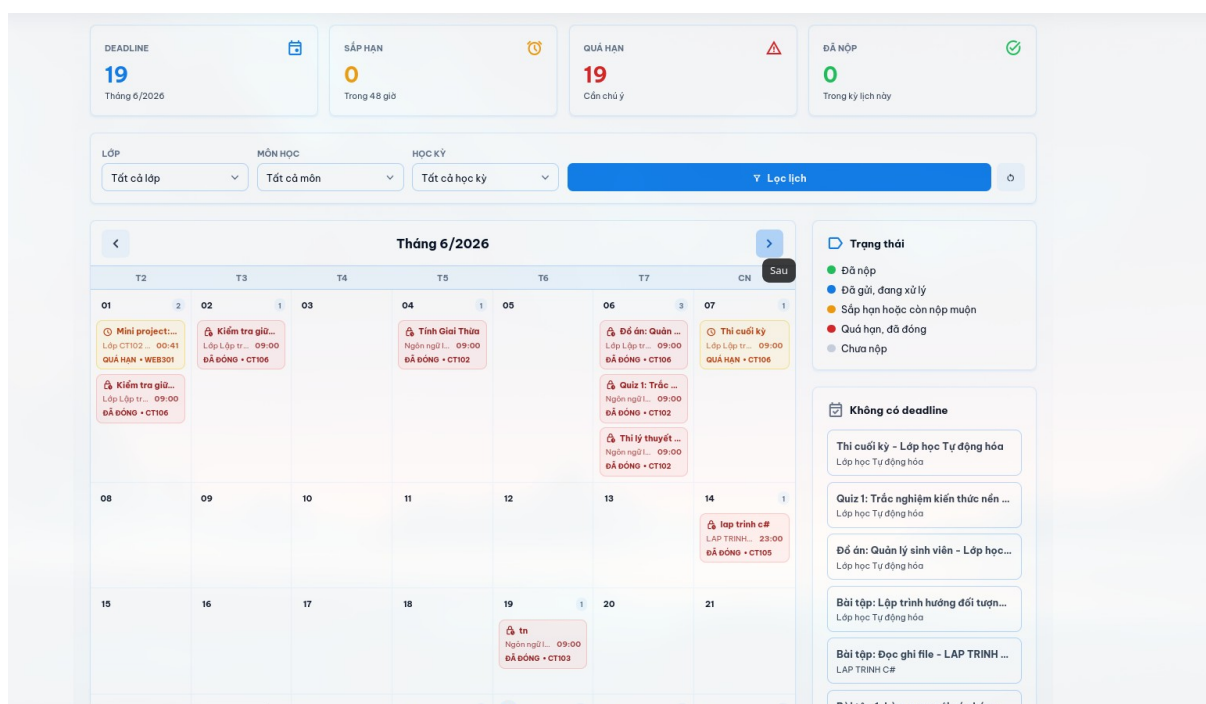
- Khu vực bên trái: Là bảng điểm chi tiết được phân cấp theo từng Lớp học và Môn học. Mỗi bài làm đều hiển thị trạng thái và điểm số với mã màu trực quan (xanh/đỏ). Để dễ tra cứu, khu vực này tích hợp thêm công cụ tìm kiếm tức thời (real-time search) và bộ lọc theo trạng thái (đã nộp, chưa làm...).

- Khu vực bên phải Đóng vai trò phân tích tổng quan, bao gồm biểu đồ vòng thể hiện tỷ lệ hoàn thành khối lượng bài tập và biểu đồ đường trực quan hóa xu hướng dao động điểm số. Phía dưới là danh sách "Lịch sử làm bài gần đây", giúp học sinh dễ dàng xem lại các bài tập vừa nộp.

- Sự kết hợp này mang lại trải nghiệm hiện đại, giúp học sinh kịp thời nhìn nhận sự tiến bộ của bản thân và hoàn thành các bài tập còn thiếu.

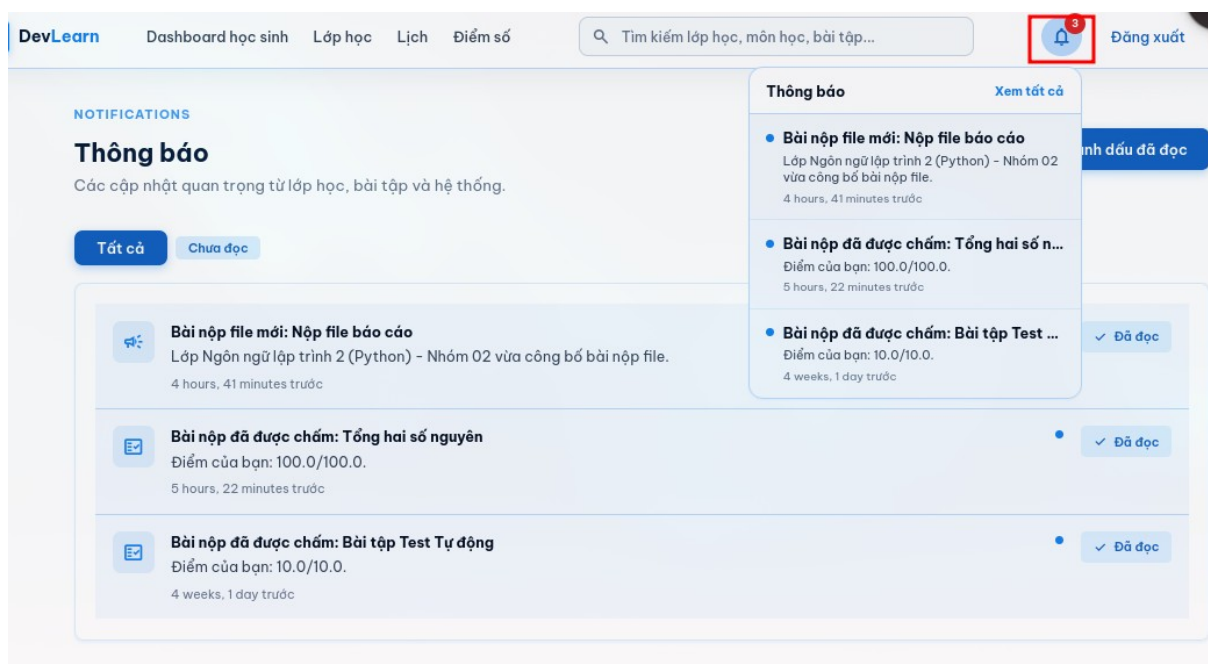
3.1.6 Lịch đóng bài tập, bài thi và thông báo

- Việc quản lý quỹ thời gian và nắm bắt kịp thời các sự kiện quan trọng là yếu tố cốt lõi giúp sinh viên hoàn thành tốt chương trình học. Phân hệ "Lịch và Thông báo" được phát triển nhằm đóng vai trò như một trợ lý học tập ảo, chủ động nhắc nhở và tổ chức công việc một cách khoa học cho người dùng. Bằng cách kết hợp giữa giao diện lịch trực quan, hệ thống thông báo nội bộ và cơ chế gửi email cảnh báo tự động, nền tảng đảm bảo luồng thông tin luôn được truyền tải thông suốt, giúp sinh viên không bao giờ bị trễ bất kỳ cột mốc deadline quan trọng nào.



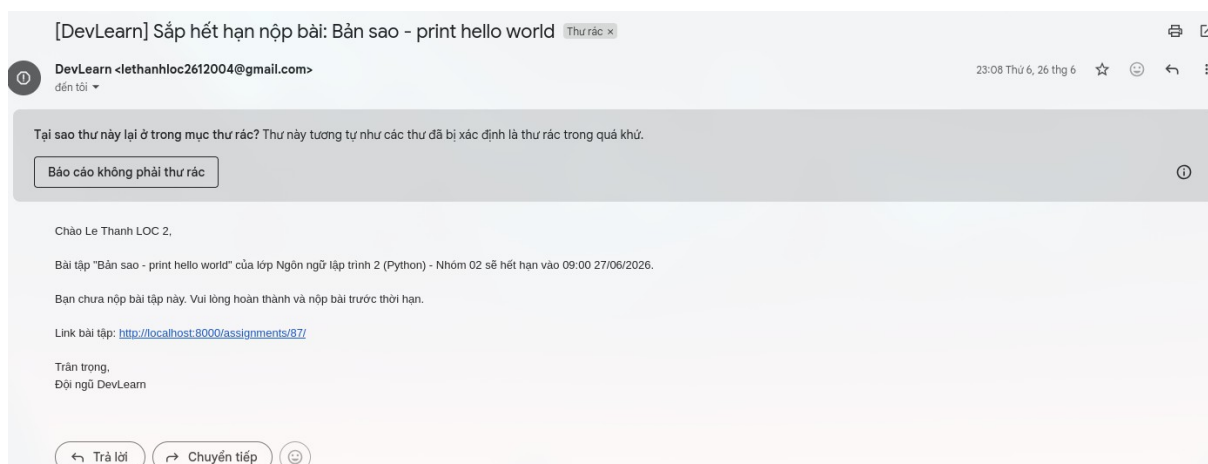
Hình 3.13. Giao diện xem thời gian hạn làm bài tập và thi

- Giao diện "Lịch deadline" (Hình 3.13) cung cấp một cái nhìn toàn cảnh về lộ trình học tập. Phía trên cùng của trang được bố trí một bảng thống kê nhanh với 4 thẻ chỉ số nổi bật: Tổng số bài tập, Số bài sắp hạn (trong 48 giờ), Số bài quá hạn, và Số bài đã nộp. Hệ thống cho phép người dùng chuyển đổi linh hoạt giữa các chế độ hiển thị theo Tháng, Tuần, hoặc Hôm nay. Đặc biệt, nhằm xử lý lượng dữ liệu lớn khi sinh viên tham gia nhiều khóa học, giao diện này tích hợp thêm bộ lọc chéo theo từng Lớp học và Môn học. Sự phân bổ thông minh này giúp sinh viên dễ dàng định vị các mốc thời gian, từ đó chủ động sắp xếp mức độ ưu tiên hợp lý cho từng nhiệm vụ.



Hình 3.14. Giao diện thông báo của website

- Trung tâm thông báo (Hình 3.14) là nơi tập hợp và phân loại toàn bộ các sự kiện diễn ra trên hệ thống liên quan trực tiếp đến sinh viên. Mỗi loại sự kiện (như có bài tập mới, bài làm đã có điểm, giáo viên vừa để lại nhận xét trên mã nguồn, hay yêu cầu vào lớp được duyệt) đều được hệ thống tự động gắn một biểu tượng (icon) đặc trưng và highlight màu sắc riêng biệt để người dùng dễ dàng phân loại bằng mắt thường. Để quản lý luồng thông tin hiệu quả, sinh viên có thể sử dụng bộ lọc "Chưa đọc" hoặc tính năng "Đánh dấu tất cả đã đọc" nhằm dọn dẹp giao diện, tránh tình trạng bị "ngợp" thông tin.

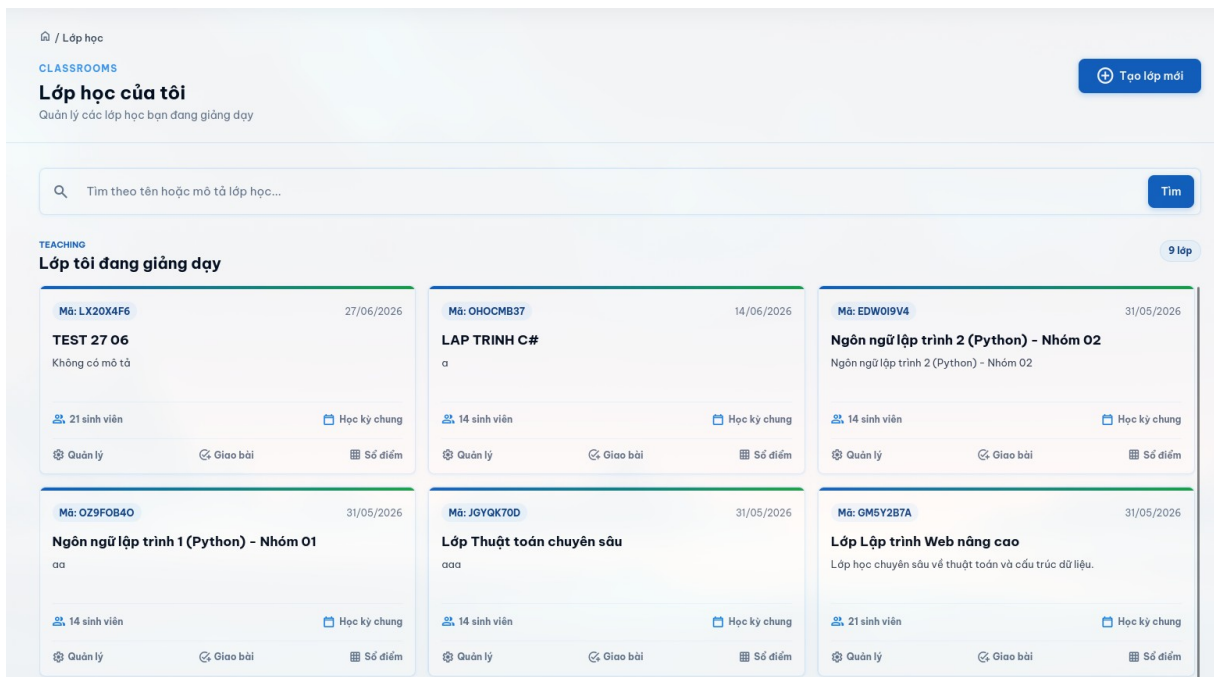


Hình 3.15. Email thông báo tự động khi có bài tập gần hết hạn

- Đóng vai trò như một lớp bảo vệ thứ hai bổ trợ cho hệ thống thông báo trên website, nền tảng còn được tích hợp cơ chế cảnh báo tự động qua thư điện tử (Hình 3.15). Khi tiến trình nền của hệ thống phát hiện có bài tập hoặc bài thi sắp đến hạn đóng mà sinh viên vẫn chưa nộp bài, một email khẩn cấp sẽ được tự động kích hoạt và gửi thẳng đến hộp thư cá nhân đã đăng ký. Nội dung email được thiết kế rõ ràng, nhấn mạnh tiêu đề bài tập, môn học và thời gian đếm ngược cụ thể. Nhờ cơ chế nhắc nhở đa nền tảng này, rủi ro sinh viên lỡ hạn nộp bài được giảm thiểu đến mức tối đa.

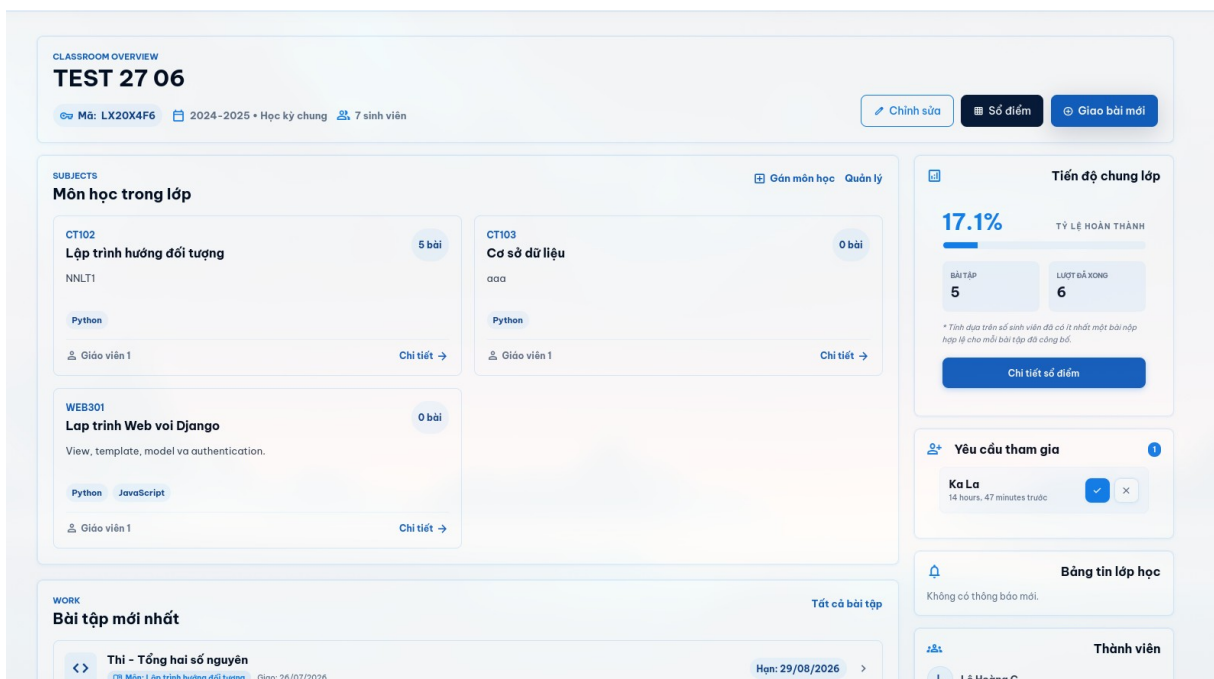
3.1.7 Lớp học của giảng viên

- Phân hệ dành riêng cho giảng viên được thiết kế nhằm tối ưu hóa công tác quản lý, giảng dạy và đánh giá sinh viên. Hệ thống cung cấp các công cụ chuyên biệt giúp giảng viên dễ dàng tổ chức lớp học, theo dõi tiến độ học tập và kiểm soát toàn bộ vòng đời của các bài tập, bài thi một cách tập trung. Đáng chú ý, nhằm đảm bảo tính chuẩn mực và kiểm soát chất lượng đào tạo, hệ thống áp dụng cơ chế kiểm duyệt chặt chẽ: mọi lớp học mới do giảng viên khởi tạo đều sẽ được đặt ở trạng thái "Chờ duyệt" và bắt buộc phải thông qua sự phê chuẩn từ Quản trị viên (Admin) trước khi chính thức đi vào hoạt động để sinh viên có thể tham gia.



Hình 3.16. Danh sách lớp học của giảng viên

- Khác với giao diện của học sinh, trang "Danh sách lớp học" (Hình 3.16) của giảng viên được áp dụng cơ chế phân quyền dữ liệu nghiêm ngặt. Hệ thống sẽ tự động lọc và chỉ hiển thị duy nhất các lớp học do chính giảng viên đó khởi tạo hoặc đang trực tiếp phụ trách giảng dạy. Mỗi lớp học được thiết kế dưới dạng thẻ (card) trực quan, tóm tắt nhanh các thông số cốt lõi như mã lớp, học kỳ và tổng số sinh viên. Giao diện tinh gọn này giúp giảng viên nhanh chóng truy cập vào không gian làm việc của từng lớp để quản lý và giao bài mà không bị nhiễu thông tin bởi các dữ liệu không liên quan.



Hình 3.17. Giao diện quản lý lớp học

- Khi truy cập vào chi tiết một lớp học (Hình 3.17), giảng viên được cung cấp một không gian quản trị toàn diện (Workspace) với các công cụ mạnh mẽ để điều phối mọi hoạt động giảng dạy. Nhằm mang lại trải nghiệm thân thiện nhưng không kém phần chuyên nghiệp, các tính năng được phân nhóm một cách khoa học:

+ Tạo và gán môn học: Để cấu trúc chương trình đào tạo, giảng viên có thể thiết lập các môn học mới. Tuy nhiên, nhằm đảm bảo tính đồng bộ và chuẩn hóa trên toàn hệ thống, các môn học tạo mới này phải trải qua quy trình phê duyệt của Quản trị viên. Khi đã được duyệt, giảng viên dễ dàng gán môn học đó vào các lớp mà mình đang quản lý.

+ Giao bài tập và bài thi: Dựa trên khung môn học đã gán, hệ thống cho phép giảng viên linh hoạt khởi tạo các bài tập rèn luyện hoặc thiết kế các bài thi (Exam) có áp dụng luật phòng thi nghiêm ngặt, tùy chỉnh thời gian đóng/mở bài một cách hoàn toàn chủ động.

+ Kiểm duyệt thành viên: Khi học sinh sử dụng mã (Invite Code) để xin gia nhập lớp, yêu cầu sẽ được đưa vào danh sách chờ. Giảng viên nắm toàn quyền kiểm duyệt nhằm bảo vệ tính riêng tư của lớp học, đảm bảo chỉ những sinh viên chính thức mới được truy cập tài nguyên.

+ Sổ điểm: Đây là công cụ đắc lực giúp tự động hóa khâu thống kê thành tích học tập. Sổ điểm theo dõi và tổng hợp điểm số của toàn bộ sinh viên qua tất cả các bài tập/bài thi, giúp giảng viên dễ dàng đánh giá bức tranh toàn cảnh về chất lượng học tập của lớp.

3.1.8 Tạo bài tập và bài thi

- Công tác ra đề và giao bài tập là một khâu trọng yếu giúp giảng viên truyền tải kiến thức và đánh giá chuẩn xác năng lực thực hành của sinh viên. Để đáp ứng nhu cầu đào tạo đặc thù của ngành Công nghệ Thông tin, hệ thống cung cấp một bộ công cụ khởi tạo bài tập đa dạng, linh hoạt, nổi bật nhất là khả năng khởi tạo bài tập lập trình tự động chấm điểm. Khác với các định dạng nộp file thông thường, phân hệ này đòi hỏi sự kết hợp chặt chẽ giữa một giao diện thiết lập thân thiện và cơ chế cấu hình kỹ thuật chuyên sâu ẩn bên dưới. Nhờ đó, giảng viên có thể dễ dàng số hóa các đề bài phức tạp, thiết lập các bộ kiểm thử khắt khe và quản lý kỳ thi một cách trọn vẹn, an toàn nhất.

The image shows a web form for creating programming assignments. The left side has two tabs: '1. Thông tin' (selected) and '2. Hình thức'. Under '1. Thông tin', there are two code editors: 'Mã nguồn khởi tạo (Starter Code)' and 'Mã nguồn mẫu (Solution Code) *'. The right side is titled 'Danh sách Testcase' and 'Chính sách & Thời gian'. It contains several form fields and checkboxes for configuring the assignment, such as start date, deadline, maximum submissions, grading method, time limit, grace period, and security features like fullscreen requirement and test case execution permission.

Hình 3.18. Form tạo bài tập bài thi dạng code

- Chức năng giao bài tập dạng mã nguồn (Programming Assignment) là một trong những phân hệ cốt lõi của nền tảng, đòi hỏi sự kết hợp giữa giao diện thân thiện và khả năng cấu hình kỹ thuật chuyên sâu. Nhằm giảm tải thao tác cho giảng viên, quy trình tạo một bài tập hoặc bài thi lập trình được hệ thống thiết kế tuần tự, logic theo các bước sau:

+ Bước 1: Thiết lập thông tin chung và Chế độ thi

Giảng viên tiến hành nhập các dữ liệu cơ bản như tiêu đề, yêu cầu đề bài, hạn nộp và điểm số tối đa. Điểm nhấn cấu trúc ở bước này là công tắc "Kích hoạt chế độ thi". Khi tùy chọn này được bật, bài tập sẽ áp dụng các chính sách bảo mật khắt khe nhất: hệ thống tự động khóa giới hạn, chỉ cho phép học sinh nộp bài duy nhất một (1) lần. Ngôn ngữ lập trình được chọn dựa theo ngôn ngữ lập trình đi theo môn học.

+ Bước 2: Cấu hình môi trường và Mã nguồn mẫu

Thấu hiểu nhu cầu của người dạy lập trình, hệ thống tích hợp sẵn một trình soạn thảo thông minh (Web IDE) ngay bên trong biểu mẫu (Form). Giảng viên không cần dùng phần mềm bên ngoài mà có thể trực tiếp chọn ngôn ngữ lập trình và soạn thảo các đoạn "Mã mẫu" (code khung) ngay trên trình duyệt. Đoạn code này sẽ tự động xuất hiện

bên trong không gian làm bài của học sinh, giúp định hướng cấu trúc hàm (function) hoặc lớp (class) cần triển khai.

+ Bước 3: Xây dựng kịch bản kiểm thử

Để hệ thống tự động chấm điểm (Auto-grader) vận hành chính xác, giảng viên cần định nghĩa các bộ kịch bản kiểm thử (Test Cases). Thông qua các trường nhập liệu trực quan, giảng viên lần lượt cung cấp "Dữ liệu đầu vào" (Input) và "Kết quả mong đợi" (Expected Output) tương ứng cho từng trường hợp. Hệ thống sẽ dùng bộ đối chiếu này để biên dịch và đánh giá tính đúng đắn trong mã nguồn của học sinh.

+ Bước 4: Chống gian lận với Test Case Ẩn

Nhằm kiểm tra tư duy thuật toán thực thụ và ngăn chặn học sinh sử dụng thủ thuật "in cứng") kết quả để qua mặt hệ thống, biểu mẫu cung cấp tùy chọn ẩn Test Case. Với những Test Case bị ẩn, học sinh sẽ chỉ biết được chương trình của mình báo lỗi (Fail) chứ không thể xem được dữ liệu đầu vào cụ thể. Cơ chế này ép buộc sinh viên phải viết mã nguồn linh hoạt, xử lý triệt để thuật toán thay vì chỉ giải quyết các bài toán bề nổi.

Chính sách & Thời gian 4/4

Ngày bắt đầu
09:00 26/06/2026

Hạn nộp
09:00 08/08/2026

Số lần nộp tối đa
Không giới hạn
Để trống nếu không giới hạn số lần nộp.

Cách tính điểm tổng kết
Lấy điểm cao nhất
Áp dụng khi học sinh nộp bài nhiều lần.

☐ Cho phép nộp muộn (trừ điểm)

☒ **KÍCH HOẠT CHẾ ĐỘ THI** (Không thể đổi do đã có lượt làm)

Ngôn ngữ cho phép (Bỏ trống = Tất cả)
☒ Python

Hình 3.19. Chỉnh sửa bài tập qua chế độ thi

- Trong quá trình vận hành thực tế, để đảm bảo tính nhất quán về mặt dữ liệu và duy trì sự công bằng tuyệt đối cho người học, hệ thống đã thiết lập một cơ chế khóa ràng

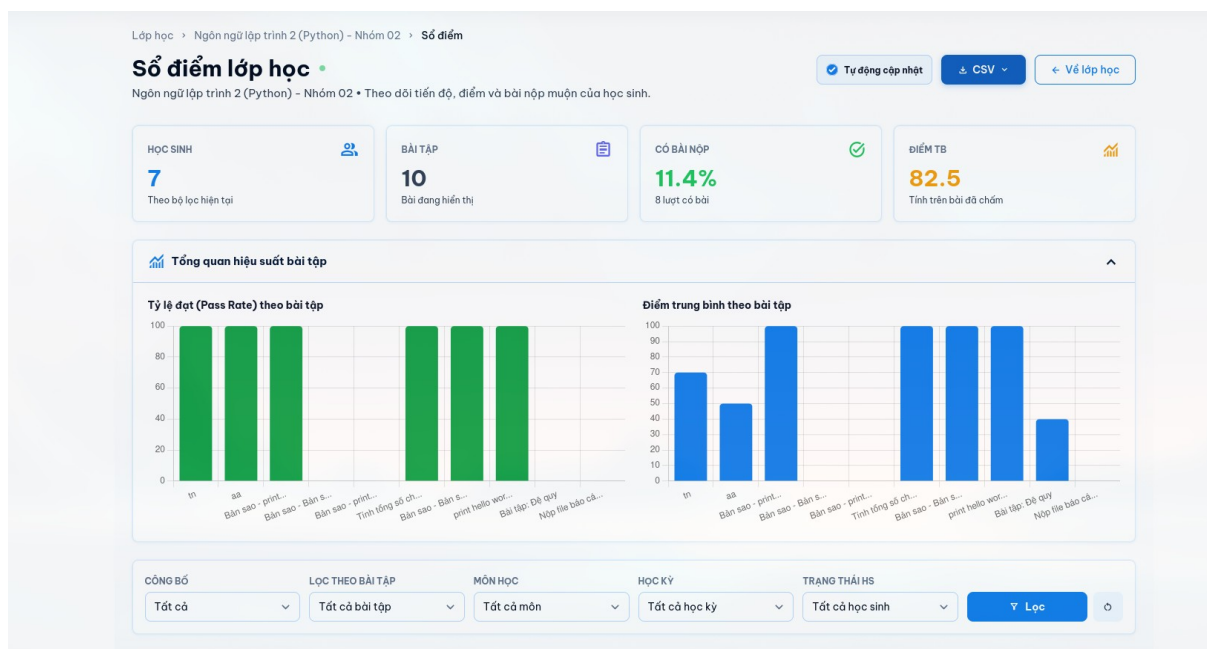
buộc chặt chẽ đối với tính năng Chế độ thi. Cụ thể, thông qua giao diện chỉnh sửa bài tập (Hình 3.19), nếu một bài tập thông thường đã ghi nhận có sinh viên bắt đầu làm bài hoặc đã nộp bài (tức là đã phát sinh dữ liệu lượt làm), công tắc chuyển đổi sang "Chế độ thi" sẽ lập tức bị hệ thống vô hiệu hóa (disabled) và làm mờ đi.

- Cơ chế phòng vệ này nhằm ngăn chặn triệt để trường hợp giảng viên vô tình thay đổi tính chất bảo mật của bài tập giữa chừng. Việc đột ngột áp đặt các quy chế thi khắt khe (như chỉ được nộp 1 lần, chặn copy-paste) lên một bài tập đang mở sẽ gây xung đột hệ thống và ảnh hưởng trực tiếp đến quyền lợi của những sinh viên đã tiến hành làm bài theo hình thức thông thường trước đó.

3.1.9 Chức năng sổ điểm

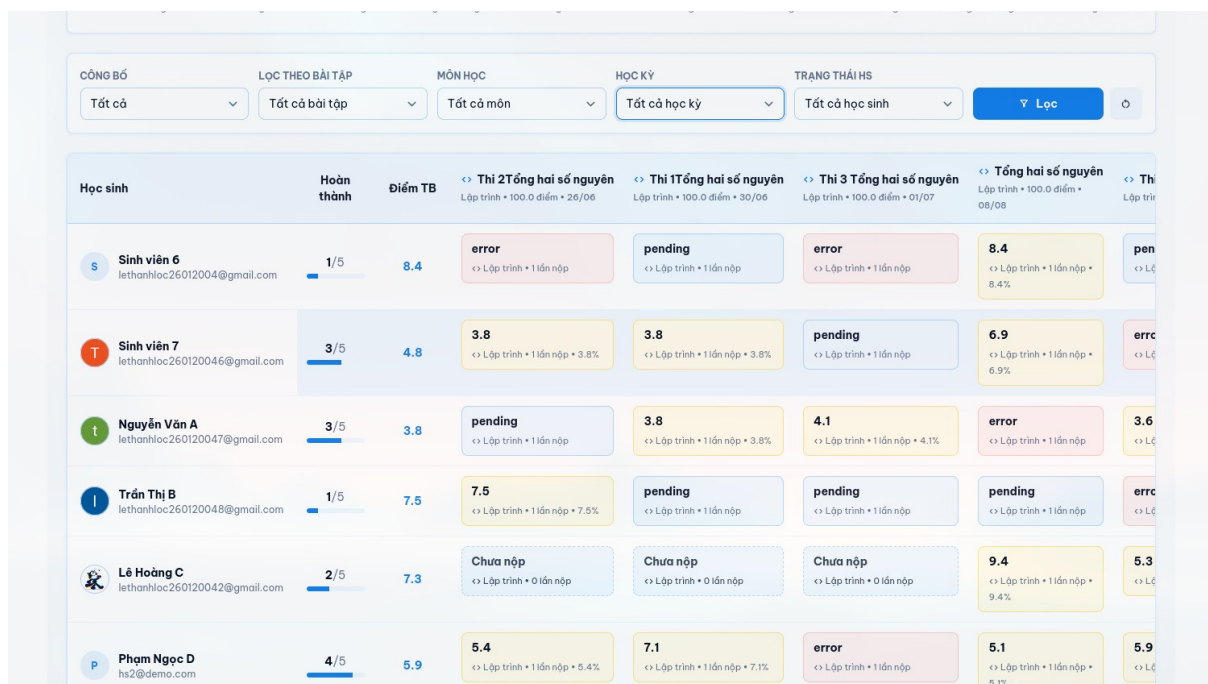
- Nhằm hỗ trợ giảng viên theo dõi sát sao, đánh giá khách quan và đưa ra các quyết định điều chỉnh phương pháp giảng dạy kịp thời, hệ thống cung cấp một phân hệ Sổ điểm toàn diện.

- Phân hệ này không chỉ đóng vai trò lưu trữ kết quả học tập mà còn hoạt động như một bảng điều khiển (dashboard) phân tích dữ liệu chuyên sâu. Mọi thông tin về quá trình nộp bài, điểm số và tần suất tương tác của sinh viên đều được hệ thống tổng hợp, tính toán và trực quan hóa theo thời gian thực.



Hình 3.20. Giao diện sổ điểm thống kê

- Mở đầu trang số điểm (Hình 3.20) là khu vực Báo cáo tổng quan, cung cấp cho giảng viên cái nhìn toàn cảnh về tình hình học tập của toàn bộ lớp học. Hệ thống tự động tính toán các chỉ số cốt lõi như: tổng số bài tập đang hiển thị, tỷ lệ hoàn thành bài tập của cả lớp và điểm trung bình chung. Bên dưới là các biểu đồ thống kê trực quan (được tích hợp bằng thư viện Chart.js), bao gồm "Biểu đồ Tỷ lệ đạt (Pass Rate)" và "Biểu đồ Điểm trung bình" phân tách chi tiết theo từng bài tập. Thiết kế dạng bảng điều khiển này giúp giảng viên nhanh chóng nhận diện được độ khó của từng bài, từ đó phát hiện các điểm bất thường (như một bài tập có tỷ lệ rớt quá cao) để kịp thời can thiệp.



Hình 3.21. Giao diện xem điểm của sinh viên

- Bên dưới phần biểu đồ là bảng Sổ điểm chi tiết (Hình 3.21), nơi lưu trữ và hiển thị điểm số của từng sinh viên tương ứng với từng bài tập. Để xử lý khối lượng dữ liệu lớn khi một lớp có hàng chục sinh viên và hàng trăm cột bài tập, hệ thống cung cấp một bộ lọc đa chiều mạnh mẽ: giảng viên có thể trích xuất dữ liệu theo Môn học, Học kỳ, Trạng thái công bố hoặc Trạng thái nộp bài của sinh viên (Thiếu bài, Nộp muộn, v.v.).

- Mỗi ô điểm (cell) trong bảng được hệ thống áp dụng thuật toán phân loại và mã hóa màu sắc (Color-coding) dựa trên kết quả: màu xanh lá cho điểm xuất sắc, màu cam cho điểm trung bình và màu đỏ cho bài bị lỗi. Ngoài ra, giao diện còn tích hợp cơ chế tự động tải lại (Auto-refresh) cho phép giám sát tiến độ thi trực tiếp và công cụ trích xuất dữ liệu thô ra định dạng Excel/CSV chỉ với một thao tác click.



Hình 3.22. Thống kê điểm theo bài tập bài thi

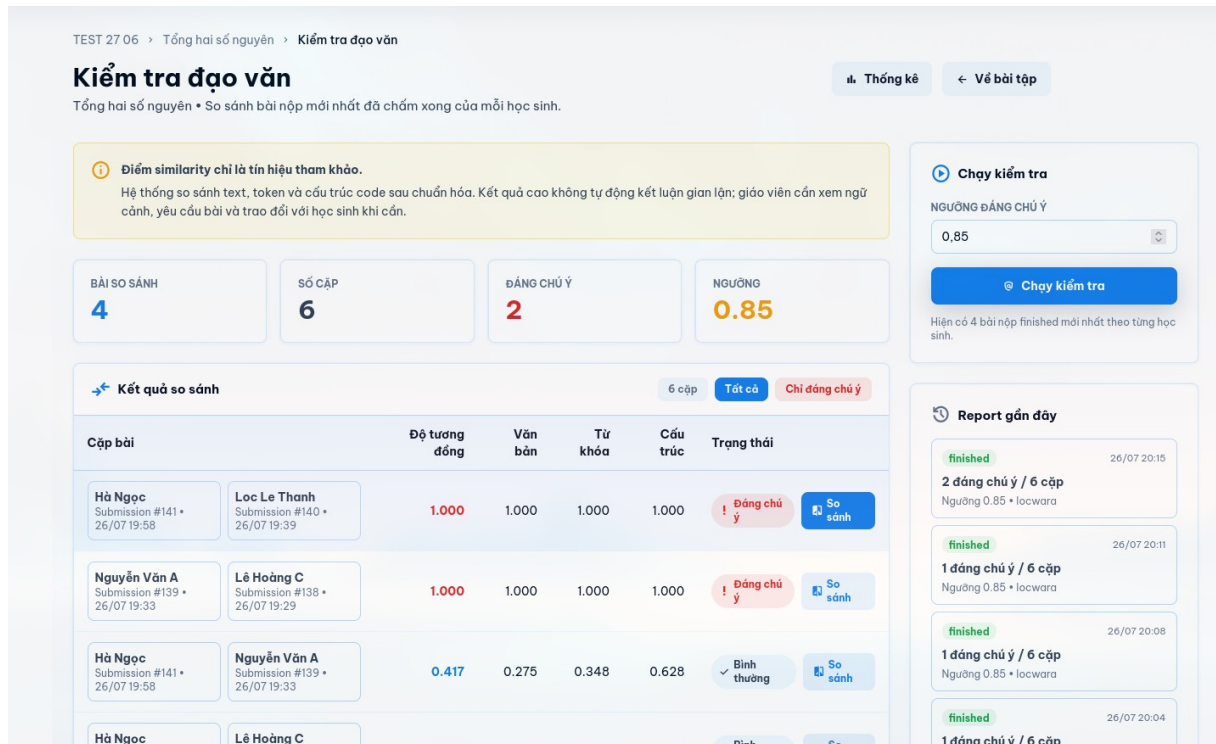
- Ngoài việc quản lý theo lớp, hệ thống còn cung cấp trang phân tích chuyên sâu cho từng bài tập cụ thể (Hình 3.22). Giao diện này bóc tách dữ liệu của một bài tập duy nhất, cung cấp biểu đồ phân bố phổ điểm (Score distribution) giúp giảng viên đánh giá năng lực đồng đều của sinh viên. Đặc biệt, trang thống kê còn tự động phân loại và vinh danh danh sách các sinh viên đạt điểm cao nhất, đồng thời khoanh vùng nhóm sinh viên có kết quả yếu kém cần hỗ trợ. Đối với riêng các bài tập lập trình, hệ thống sẽ thống kê thêm "Tỷ lệ thất bại của từng Testcase", giúp giảng viên dễ dàng nắm bắt được lỗ hổng kiến thức chung hoặc thuật toán mà đa số sinh viên đang gặp vướng mắc để ưu tiên sửa lỗi trong buổi học tiếp theo.

3.1.10 Kiểm tra đạo văn

- Nhằm đảm bảo tính minh bạch và công bằng trong môi trường học thuật, hệ thống tích hợp phân hệ tự động kiểm tra đạo văn (Plagiarism Checker) chuyên sâu dành riêng cho các bài tập lập trình mã nguồn (source code). Điểm đột phá của công cụ này nằm ở việc không chỉ so sánh đối sánh văn bản thô (text-matching) thông thường, mà còn ứng

dụng thuật toán phân tích Cây cú pháp trừu tượng (Abstract Syntax Tree - AST) và kỹ thuật lấy dấu vân tay mã nguồn (Winnowing).

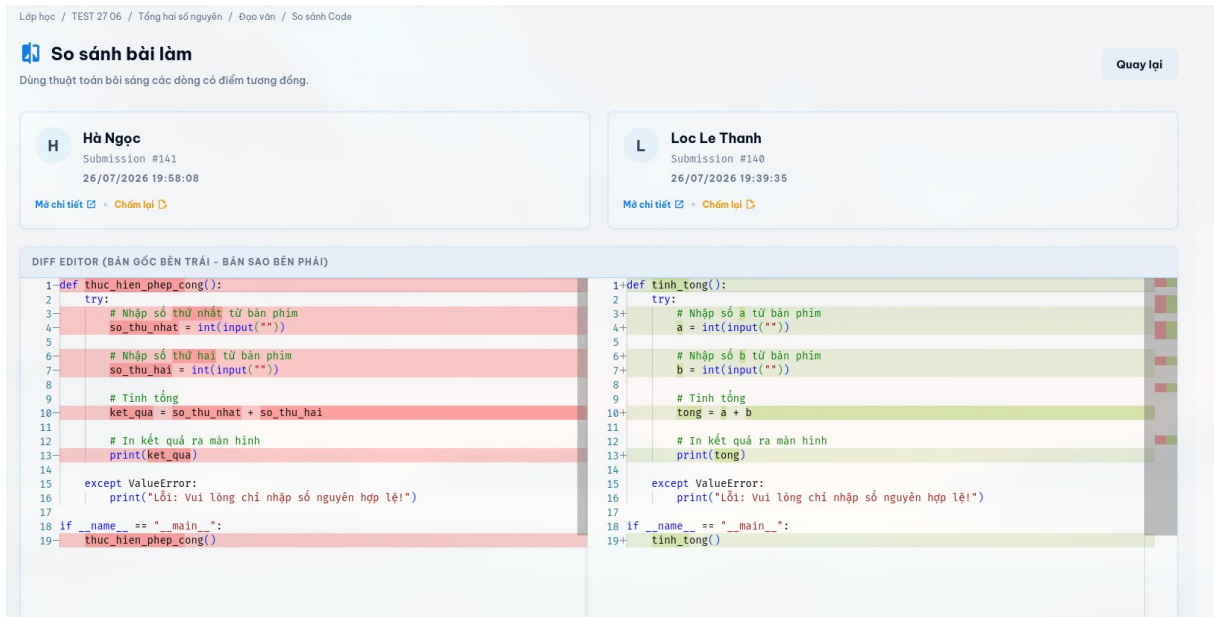
- Hệ thống có khả năng bóc tách, chuẩn hóa toàn bộ khoảng trắng, chú thích, đồng thời ép đổi tên toàn bộ các biến, tham số và hàm tự định nghĩa của sinh viên về một chuẩn chung. Nhờ vậy, thuật toán có thể dễ dàng phát hiện các thủ thuật gian lận tinh vi như sao chép mã nguồn và chỉ cố tình thay đổi tên biến để qua mặt hệ thống. Quá trình kiểm tra được đưa vào hàng đợi xử lý ngầm để không làm gián đoạn trải nghiệm của giảng viên với những lớp học có lượng dữ liệu lớn.



Hình 3.23. Giao diện kiểm tra đạo văn

- Giao diện tổng hợp kết quả (Hình 3.23) trình bày danh sách các cặp bài nộp có mức độ trùng lặp mã nguồn với nhau. Từng cặp bài được chấm điểm độ tương đồng chi tiết dựa trên nhiều trọng số: Tương đồng văn bản, Tương đồng từ khóa (Token), và Tương đồng cấu trúc (AST). Hệ thống tự động phân loại và gắn thẻ "Đáng chú ý" (màu đỏ) đối với các cặp bài có tỷ lệ trùng lặp cao bất thường (thường là trên 85%), giúp giảng viên thu hẹp phạm vi cần rà soát.

- Về mặt tối ưu luồng thao tác (workflow), giao diện cho phép giảng viên thao tác cực kỳ linh hoạt. Khi rê chuột vào thẻ thông tin của bất kỳ sinh viên nào, một nút chức năng "Chấm lại" sẽ xuất hiện, cho phép điều hướng trực tiếp đến trang trừ điểm/cập nhật lại điểm số của sinh viên đó ngay lập tức mà không cần vòng qua nhiều trang trung gian.



Hình 3.24. Giao diện so sánh bài làm

- Để cung cấp bằng chứng trực quan nhất, hệ thống xây dựng một Giao diện So sánh chi tiết (Hình 3.24). Chức năng này tích hợp bộ so sánh mã nguồn song song (Side-by-side Diff Editor) dựa trên nền tảng Monaco Editor – bộ mã nguồn lõi của Visual Studio Code. Đoạn mã của hai sinh viên được đặt cạnh nhau, trong đó những dòng code bị sao chép, chỉnh sửa hoặc xóa bỏ sẽ được tô sáng với các dải màu phân biệt rõ ràng. Nhờ công cụ này, giảng viên không cần tải file về máy tính mà có thể đối chiếu chéo trực tiếp trên trình duyệt, từ đó có căn cứ xác thực và khách quan nhất để đưa ra quyết định xử lý vi phạm.

3.1.11 Chấm điểm thủ công

- Bên cạnh hệ thống chấm điểm tự động, website còn hỗ trợ phân hệ chấm điểm thủ công dành cho các bài tập yêu cầu sự đánh giá phức tạp từ giảng viên (như bài nộp file báo cáo hoặc bài tập code cần chấm thêm điểm phong cách lập trình, tính sáng tạo).

Rubric 1.0/100.0

code sách se
1.0 điểm

Tên tiêu chí: Code đúng cấu trúc
Thứ tự hiển thị: 1

Mô tả chi tiết: Code đúng cấu trúc

Điểm tối đa: 1

+ Thêm tiêu chí

Hình 3.25. Tạo rubric chấm điểm thủ công

- Hệ thống cho phép giảng viên xây dựng bộ Tiêu chí chấm điểm (Rubric) động ngay tại trang quản lý của từng bài tập (Hình 3.25). Giảng viên có thể định nghĩa nhiều tiêu chí khác nhau, mỗi tiêu chí bao gồm Tên, Mô tả chi tiết, Điểm tối đa và Thứ tự hiển thị. Đặc biệt, để tối ưu trải nghiệm người dùng, form tạo Rubric này được thiết kế ẩn đi hoàn toàn đối với các bài tập Trắc nghiệm (do đặc thù chấm tự động 100%), giúp giao diện gọn gàng và tránh gây nhầm lẫn.

TEST 27 06 > TỔNG HAI SỐ NGUYÊN

N

Hà Ngọc

Nộp lúc 26/07/2026 19:58 · Lập trình

1 / 11

Tiếp >

< Danh sách

<> SOLUTION.PYTHON

26/07/2026 19:58

```

1 def thuc_hien_phep_cong():
2     try:
3         # Nhập số thứ nhất từ bàn phím
4         so_thu_nhat = int(input(""))
5
6         # Nhập số thứ hai từ bàn phím
7         so_thu_hai = int(input(""))
8
9         # Tính tổng
10        ket_qua = so_thu_nhat + so_thu_hai
11
12        # In kết quả ra màn hình
13        print(ket_qua)
14
15    except ValueError:
16        print("Lỗi: Vui lòng chỉ nhập số nguyên hợp lệ!")
17
18 if __name__ == "__main__":
19     thuc_hien_phep_cong()

```

Bấm vào số dòng để thêm nhận xét cho sinh viên ...

EXPECTED

7

ACTUAL

7

Tiêu chí chấm điểm

Tổng: 1.0

code sách se

/1.0

Điểm

Nhận xét theo tiêu chí...

☐ Dùng tổng điểm tiêu chí làm điểm cuối cùng (Ghi đề)

☐ Cộng điểm tiêu chí vào điểm tự động (Testcase)

Điểm thủ công

100.0 / 100.0

Nhận xét tổng quan

Bấm mẫu để chèn

Can xu ly edge case

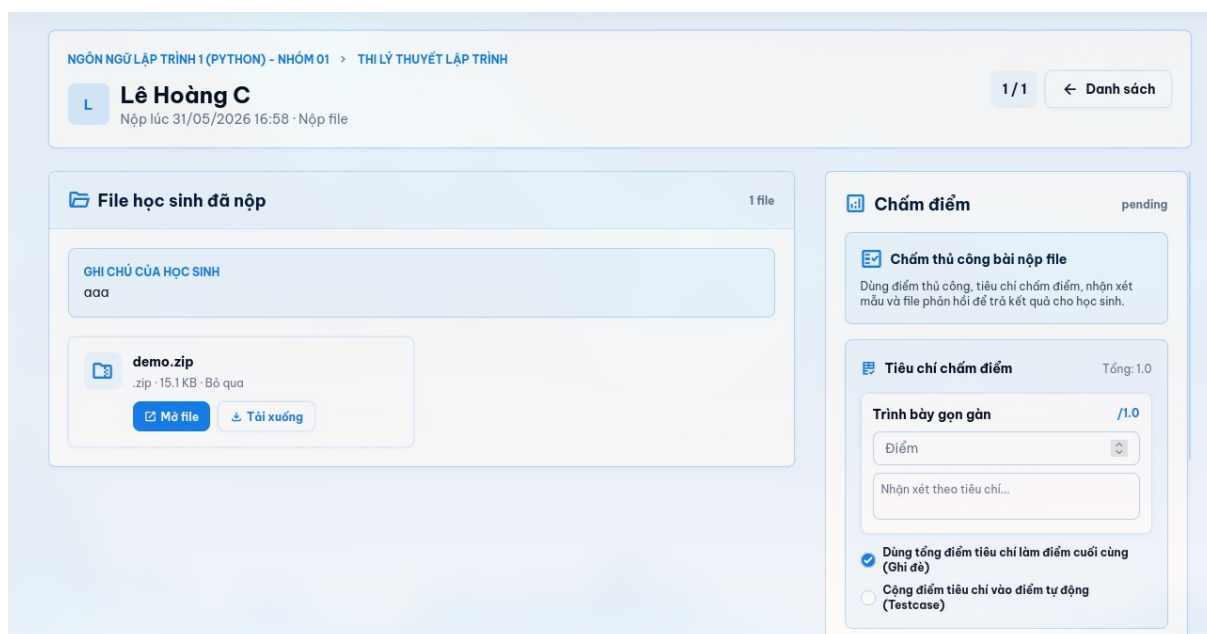
Nhận xét về bài làm...

Hình 3.26. Chấm điểm cho bài tập cụ thể theo rubric dạng code

- Tại màn hình chấm điểm của bài tập dạng Code (Hình 3.26), giao diện được chia bố cục trực quan: một bên là mã nguồn của sinh viên kèm kết quả chạy kiểm thử tự động (Testcases), một bên là bảng Rubric. Các ô Testcase được trình bày rõ ràng với ba phần Input , Expected (Kết quả mong đợi) và Actual (Kết quả thực tế), giúp giảng viên nhanh chóng đối chiếu logic giải thuật.

- Tại đây, giảng viên có thể chấm điểm cho từng tiêu chí Rubric và chọn một trong hai chế độ tính điểm linh hoạt:

- + Ghi đề: Lấy tổng điểm Rubric làm điểm cuối cùng của bài làm.
- + Cộng dồn: Cộng điểm Rubric (điểm thưởng) vào điểm Testcase tự động.



Hình 3.27. Chấm điểm cho bài tập cụ thể theo rubric dạng nộp file

- Với dạng bài tập Nộp File (Hình 3.27), do không có hệ thống testcase tự động, không gian bên trái sẽ hiển thị danh sách các tệp tin mà sinh viên đã tải lên để giảng viên dễ dàng duyệt, tải về hoặc xem trước tài liệu. Dựa vào các tệp này, giảng viên sử dụng bảng Rubric bên phải để đánh giá chi tiết từng phần. Điểm số của từng tiêu chí sẽ tự động được tổng hợp thành kết quả cuối cùng. Ngoài ra, giảng viên có thể sử dụng tính năng "Mẫu nhận xét nhanh" (Feedback Templates) để chèn các đoạn đánh giá được soạn sẵn, giúp tiết kiệm đáng kể thời gian khi chấm bài cho các lớp học có đông sinh viên.

3.1.12 Thảo luận trong bài tập, bài thi

- Để tăng cường tính tương tác và hỗ trợ sinh viên kịp thời trong quá trình làm bài, hệ thống tích hợp phân hệ Thảo luận (Discussion) riêng biệt cho từng bài tập và bài thi. Chức năng này hoạt động tương tự như một diễn đàn hỏi đáp thu nhỏ (Q&A Forum), giúp sinh viên và giảng viên dễ dàng trao đổi kiến thức thay vì phải sử dụng các nền tảng chat bên ngoài.

TEST 27 06 > Tổng hai số nguyên > Thảo luận

DISCUSSIONS

Diễn đàn thảo luận

Đặt câu hỏi, thảo luận và chia sẻ kiến thức với bạn cùng lớp.

Đặt câu hỏi

🔍

Tìm kiếm chủ đề, câu hỏi...

Tất cả

Chưa trả lời

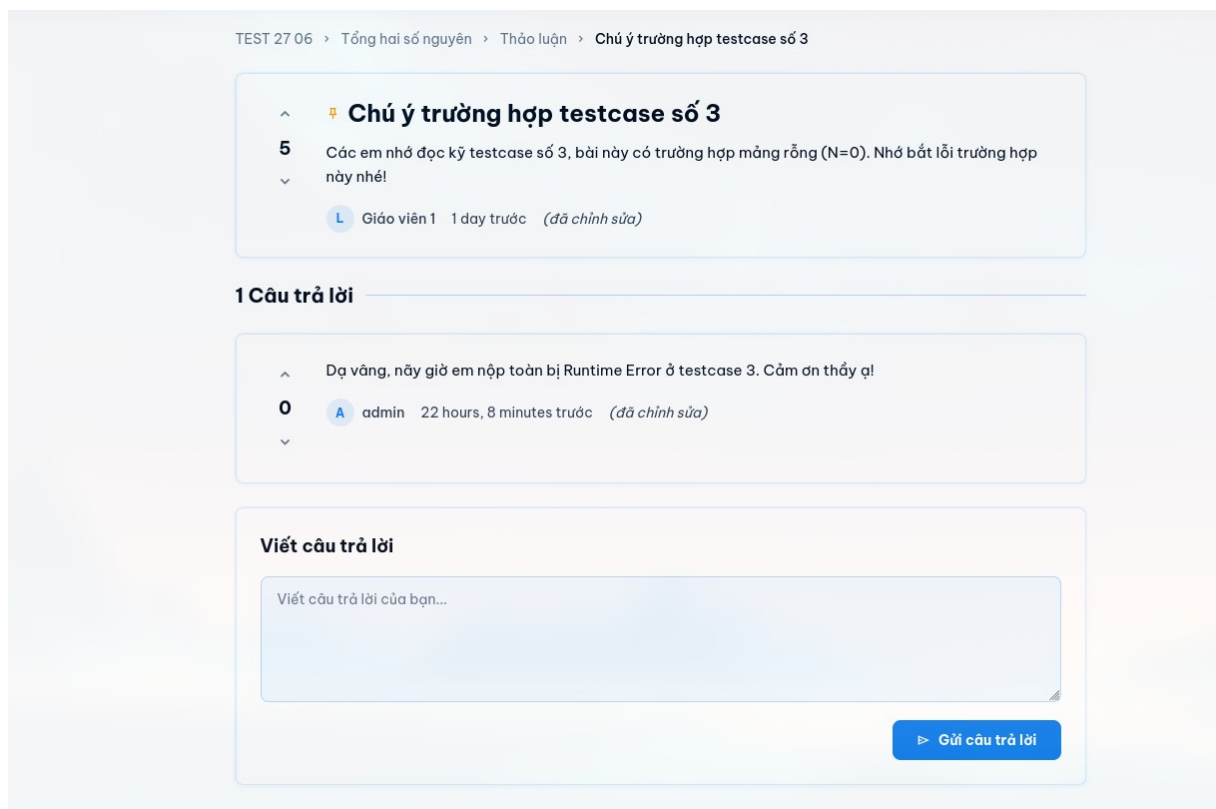
Câu hỏi của tôi

Phổ biến

CHỦ ĐỀ	VOTES	TRẢ LỜI	TRẠNG THÁI	NGƯỜI ĐĂNG
📌 Chú ý trường hợp testcase số 3 Các em nhớ đọc kỹ testcase số 3, bài này có trường...	5 VOTES	1 TRẢ LỜI	ĐANG THẢO LUẬN	Giáo viên 1 1 day trước
Bị lỗi Time Limit Exceeded Có ai bị lỗi Time Limit Exceeded ở testcase 5 không...	1 VOTES	0 TRẢ LỜI	CHỜ TRẢ LỜI	Sinh viên 6 30 minutes trước
Dùng vòng lặp while thay vì for Thấy ới, bài này nếu em dùng vòng lặp while thay vì...	2 VOTES	2 TRẢ LỜI	ĐÃ GIẢI	admin 5 hours trước

Hình 3.28. Danh sách các thảo luận trong bài tập, bài thi

- Giao diện danh sách thảo luận (Hình 3.28) cung cấp cái nhìn tổng quan về tất cả các chủ đề đã được đăng trong khuôn khổ của bài tập đó. Các thông tin quan trọng như: Tiêu đề câu hỏi, số lượt bình chọn (Votes), và số lượng câu trả lời đều được hiển thị trực quan. Giảng viên có quyền hạn đặc biệt để Ghim (Pin) những thông báo hoặc chủ đề quan trọng lên đầu danh sách (được đánh dấu bằng biểu tượng ghim màu cam), đảm bảo sinh viên không bỏ lỡ các lưu ý quan trọng.



Hình 3.29. Chi tiết thảo luận

- Khi bấm vào một chủ đề, người dùng sẽ được chuyển đến trang chi tiết thảo luận (Hình 3.29). Tại đây, hệ thống hỗ trợ hàng loạt các tính năng tương tác chuyên sâu bao gồm:

- + Upvote / Downvote: Các thành viên trong lớp có thể bình chọn cho câu hỏi hoặc bình luận hữu ích. Hệ thống sẽ tự động sắp xếp các câu trả lời có số lượt Upvote cao lên trên, giúp sinh viên khác nhanh chóng tiếp cận được cách giải quyết tốt nhất.

- + Đánh dấu câu trả lời đúng (Mark as Answer): Giảng viên có quyền duyệt và đánh dấu một bình luận bất kỳ làm "Câu trả lời được chấp nhận". Tính năng này đặc biệt hữu ích để xác nhận tính chính xác của một lời giải, giúp chấm dứt các tranh luận kéo dài.

- + Phân cấp bình luận: Các cuộc hội thoại được thiết kế theo dạng chủ đề và trả lời rõ ràng, đi kèm với thông tin người đăng và thời gian được định dạng thân thiện (ví dụ: "1 day trước"). Nội dung thảo luận được hỗ trợ ngắt dòng và hiển thị với độ tương phản cao nhằm tối ưu hóa trải nghiệm đọc hiểu.

3.2. Kiểm Thử Hệ Thống Và Đánh Giá Hiệu Năng

3.2.1. Môi trường triển khai và kiểm thử (Test Environment)

Để tiến hành thu thập số liệu định lượng về khả năng chịu tải và tốc độ xử lý của hệ thống chấm điểm tự động (Sandbox), nghiên cứu đã tiến hành thiết lập một môi trường kiểm thử thực tế. Thay vì sử dụng các dịch vụ cho thuê máy chủ ảo (Cloud VPS) với cấu hình phần cứng bị giới hạn và chia sẻ, hệ thống trong đồ án này được triển khai theo mô hình On-Premise Deployment (Tự lưu trữ - Self-Hosted) ngay trên máy chủ cục bộ.

Mô hình này giúp ứng dụng tận dụng tối đa sức mạnh phần cứng chuyên dụng, đảm bảo tính bảo mật dữ liệu tuyệt đối và làm chủ hoàn toàn môi trường ảo hóa Docker mà không phụ thuộc vào nền tảng của bên thứ ba (như Railway hay Heroku). Hệ thống sử dụng công nghệ Proxy Đảo (Reverse Proxy) thông qua đường hầm Cloudflare Tunnel để định tuyến bảo mật luồng truy cập từ Internet vào máy chủ cục bộ, mô phỏng chính xác trải nghiệm của người dùng thực tế.

Cấu hình chi tiết của máy chủ kiểm thử (Server Specifications) được trình bày như sau:

Mô hình triển khai: On-Premise Deployment (Self-Hosted kết hợp Cloudflare Tunnel).

- + Hệ điều hành (OS): Arch Linux (Rolling Release).
- + Vi xử lý trung tâm (CPU): AMD Ryzen 5 5600H with Radeon Graphics (Cung cấp năng lực tính toán đa luồng mạnh mẽ, đặc biệt phù hợp cho các tác vụ chấm code đồng thời).
- + Bộ nhớ trong (RAM): 8 GB (Dư dả để Sandbox cấp phát và thu hồi bộ nhớ liên tục cho hàng chục tiến trình Docker).
- + Môi trường ngôn ngữ Web: Python 3.14.5.
- + Môi trường ảo hóa Sandbox: Docker Engine v29.5.2 (Chạy trực tiếp (Native) trên nhân Linux, mang lại hiệu suất I/O và CPU tối đa).

3.2.2. Đánh giá tốc độ xử lý của Sandbox

Mục đích và kịch bản kiểm thử (Test Scenario) Mục tiêu của phần kiểm thử này là đo lường hiệu năng của hệ thống Sandbox dưới áp lực cao (Stress Test) nhằm trả lời cho câu hỏi: "Hệ thống sẽ phản hồi như thế nào khi có nhiều sinh viên nộp bài cùng một thời

điểm?". Kịch bản giả lập được đưa ra là có 10 sinh viên đồng thời nộp mã nguồn Python (chứa vòng lặp tính toán để gây tải CPU) vào hệ thống.

Công cụ và luồng thực thi kịch bản (Test Script Flow) Để mô phỏng môi trường đa luồng, nghiên cứu đã tự phát triển một kịch bản kiểm thử (Test Script) bằng Python sử dụng thư viện `concurrent.futures`. Luồng hoạt động của Script bao gồm các hàm cốt lõi sau:

- + Hàm `run_benchmark(concurrency, total_requests)`: Đóng vai trò là luồng chính (Main Thread). Hàm này khởi tạo một `ThreadPoolExecutor` với số lượng worker là 10 (tương đương 10 sinh viên). Nó nạp đoạn mã nguồn mẫu và đẩy đồng loạt 10 tác vụ nộp bài vào hàng đợi. Sau khi hoàn tất, hàm này tiến hành thống kê và xuất dữ liệu ra file Excel (CSV).

- + Hàm `run_single(worker_id, code, lang)`: Đóng vai trò là một luồng người dùng (User Thread). Hàm này đo lường thời gian (timestamp) ngay trước khi gọi Sandbox và ngay sau khi Sandbox trả kết quả. Từ đó tính toán được tổng thời gian quay vòng (Turnaround Time - tức Thời gian phản hồi).

- + Hàm `execute_code(...)`: Là hàm lõi của hệ thống Backend, chịu trách nhiệm giao tiếp trực tiếp với Docker Daemon để cấp phát Container, giới hạn tài nguyên (RAM 256MB, 1 CPU) và thu thập log thực thi.

Kết quả kiểm thử và Đánh giá Sau khi thực thi kịch bản, các số liệu tổng quan về môi trường và thông số tải được ghi nhận. Thông số đầu vào và kết quả tổng quan của kịch bản kiểm thử Sandbox Thông số

Bảng 3.1. Thông số đầu vào và kết quả tổng quan

Thông số	Giá trị
Ngôn ngữ	python
Số lượng worker	10
Tổng số bài nộp	10
Tổng thời gian xử lý (giây)	2.11
Số bài nộp thành công	10

Thông số	Giá trị
Tỷ lệ thành công (%)	100.0%
Thời gian phản hồi TB (giây)	2.04
Thời gian phản hồi lớn nhất (giây)	2.11
Thời gian phản hồi nhỏ nhất (giây)	1.92

Bảng 3.1. Thông số đầu vào và kết quả tổng quan

Với 10 tác vụ nặng được gửi đến cùng một tích tắc, hệ thống chỉ mất tổng cộng 2.11 giây để giải quyết triệt để toàn bộ hàng đợi. Tỷ lệ thành công đạt 100%, không có bất kỳ request nào bị rơi (drop) hoặc báo lỗi Server.

Bảng 3.2. Thống kê chi tiết tài nguyên và thời gian phản

Worker ID	Trạng thái	Thời gian phản hồi (s)	Thời gian Sandbox (s)	Bộ nhớ (MB)
9	Success	1.92	0.84	7.34
1	Success	1.93	0.85	6.95
0	Success	2.01	0.93	7.16
7	Success	2.02	0.95	7.55
3	Success	2.04	0.96	7.27
2	Success	2.06	0.98	6.85
6	Success	2.1	1.02	7.08
4	Success	2.1	1.02	6.77

Worker ID	Trạng thái	Thời gian phản hồi (s)	Thời gian Sandbox (s)	Bộ nhớ (MB)
8	Success	2.1	1.03	7.2
5	Success	2.11	1.03	7.54

Bảng 3.2. Thống kê chi tiết tài nguyên và thời gian phản hồi

Từ các số liệu thu thập được ở Bảng 3.2, Kết quả đưa ra các đánh giá kỹ thuật sau:

Tốc độ (Speed): Thời gian phản hồi trung bình cho một sinh viên khi có tải nặng là 2.04 giây. Trong đó, thời gian thực thi mã nguồn bên trong lõi Sandbox chỉ chiếm chưa tới 1.0 giây. Phần thời gian còn lại (khoảng 1.0s) là chi phí khởi tạo môi trường (Overhead) của Docker Daemon và độ trễ luồng. Mức 2.04 giây là một con số rất ấn tượng, mang lại trải nghiệm thời gian thực (Real-time) mượt mà cho sinh viên trên giao diện Web.

Tính ổn định (Stability): Chênh lệch giữa thời gian phản hồi nhanh nhất (1.92s) và chậm nhất (2.11s) là rất nhỏ (chỉ 0.19s). Điều này chứng minh thuật toán điều phối tài nguyên của Linux Kernel hoạt động vô cùng công bằng (Fair-sharing) giữa các Container.

Tài nguyên (Resource Efficiency): Bộ nhớ RAM tiêu thụ của mỗi bài nộp dao động từ 6.77 MB đến 7.55 MB, hoàn toàn nằm trong giới hạn an toàn. Nhờ việc sử dụng ảnh Docker tối giản (Alpine Linux), hệ thống cực kỳ tiết kiệm bộ nhớ, cho phép mở rộng (Scale-up) để phục vụ cho các lớp học có sĩ số lên tới hàng trăm sinh viên mà không sợ hiện tượng cạn kiệt RAM vật lý (OOM).

3.2.3. Đánh giá độ chính xác của Thuật toán quét đạo văn Winnowing

Mục đích và kịch bản kiểm thử Mục tiêu của phần này là đánh giá độ nhạy bén và tính chính xác của lõi thuật toán phát hiện đạo văn mã nguồn. Hệ thống phải chứng minh được khả năng phân biệt rõ ràng giữa hai trường hợp:

1 là sinh viên sao chép bài của nhau rồi dùng tiểu xảo che giấu (đổi tên biến, đổi cấu trúc lặp)

2 là sinh viên tự làm bài độc lập nhưng có thuật toán giải quyết giống nhau. Ngưỡng cảnh báo đạo văn được hệ thống thiết lập ở mức 80%.

Công cụ và luồng thực thi kịch bản (Test Script Flow) Một kịch bản Python (`evaluate_plagiarism.py`) được xây dựng để nạp danh sách mã nguồn của 5 sinh viên mô phỏng, tiến hành bắt cặp chéo (Pairwise Comparison) và đối chiếu với nhãn thực tế (Ground Truth). Các hàm cốt lõi tham gia vào quá trình này bao gồm:

+ Hàm **`check_plagiarism_batch(submissions, language)`** : Hàm giao tiếp API chính, nhận vào danh sách bài nộp và gọi lõi thuật toán để tiến hành quét chéo tất cả các bài ($N \times (N - 1)/2$ cặp).

+ Hàm **`evaluate()`** : Hàm chạy kịch bản chính, duyệt qua kết quả trả về của hệ thống, so sánh với sự thật (Actual Plagiarism) để phân loại thành 4 nhóm chuẩn học thuật: TP (Đúng-Có đạo văn), FP (Sai-Báo nhầm), FN (Sai-Bỏ sót) và TN (Đúng-Không đạo văn).

+ Hệ thống sử dụng kỹ thuật trích xuất Cây cú pháp trừu tượng (AST - Abstract Syntax Tree) kết hợp thuật toán băm Winnowing để loại bỏ hoàn toàn các yếu tố gây nhiễu như khoảng trắng, comment hay tên biến.

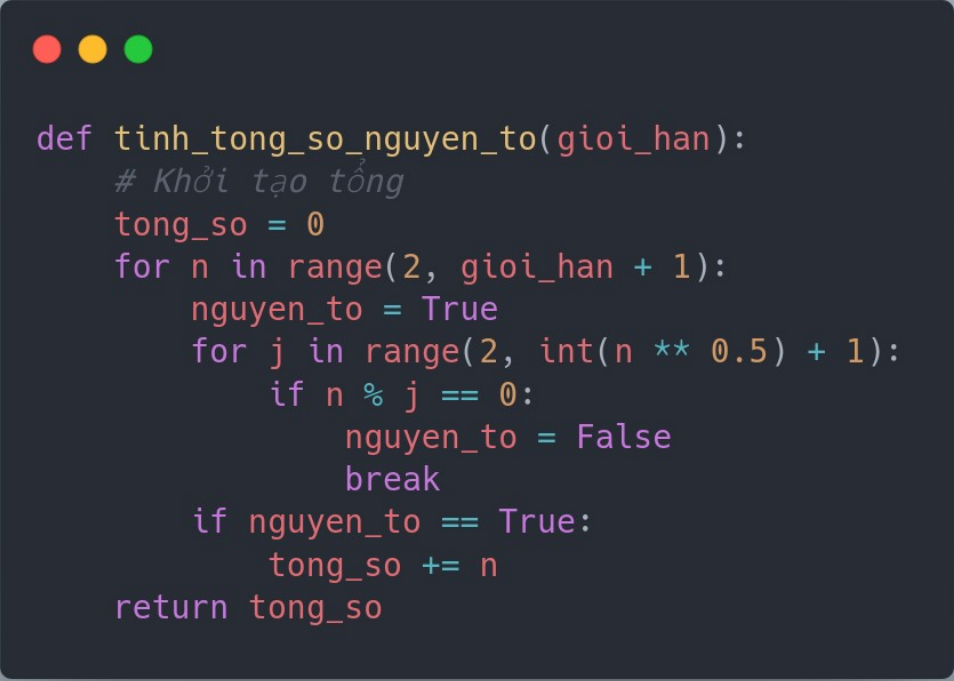
Danh sách mã nguồn kiểm thử (Dataset)

Kịch bản sử dụng bài toán "Tính tổng các số nguyên tố". 5 mẫu mã nguồn được thiết kế với các đặc trưng hành vi lập trình khác nhau để thử thách thuật toán:

```
def calculate_prime_sum(limit):
    total = 0
    for num in range(2, limit + 1):
        is_prime = True
        for i in range(2, int(num ** 0.5) + 1):
            if num % i == 0:
                is_prime = False
                break
        if is_prime:
            total += num
    return total
```

Hình 3.30. Mã nguồn của Sinh viên 1

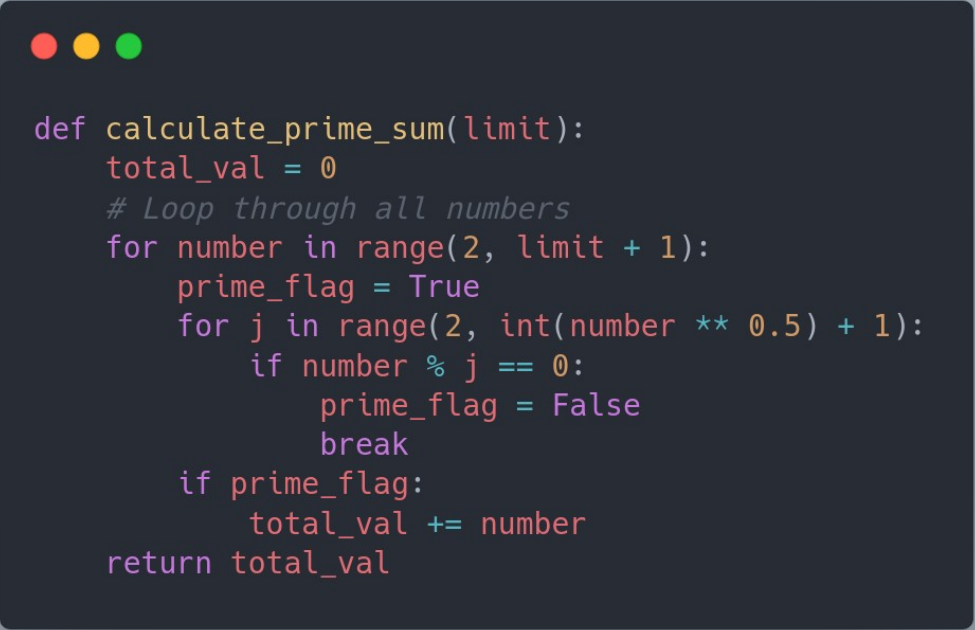
Đây là mã nguồn chuẩn do Sinh viên 1 tự viết. Giải thuật sử dụng 2 vòng lặp for lồng nhau cơ bản để kiểm tra và cộng dồn các số nguyên tố. Mã nguồn này đóng vai trò là "bản gốc" để đối chiếu.



```
def tinh_tong_so_nguyen_to(gioi_han):  
    # Khởi tạo tổng  
    tong_so = 0  
    for n in range(2, gioi_han + 1):  
        nguyen_to = True  
        for j in range(2, int(n ** 0.5) + 1):  
            if n % j == 0:  
                nguyen_to = False  
                break  
        if nguyen_to == True:  
            tong_so += n  
    return tong_so
```

Hình 3.31. Mã nguồn của Sinh viên 2

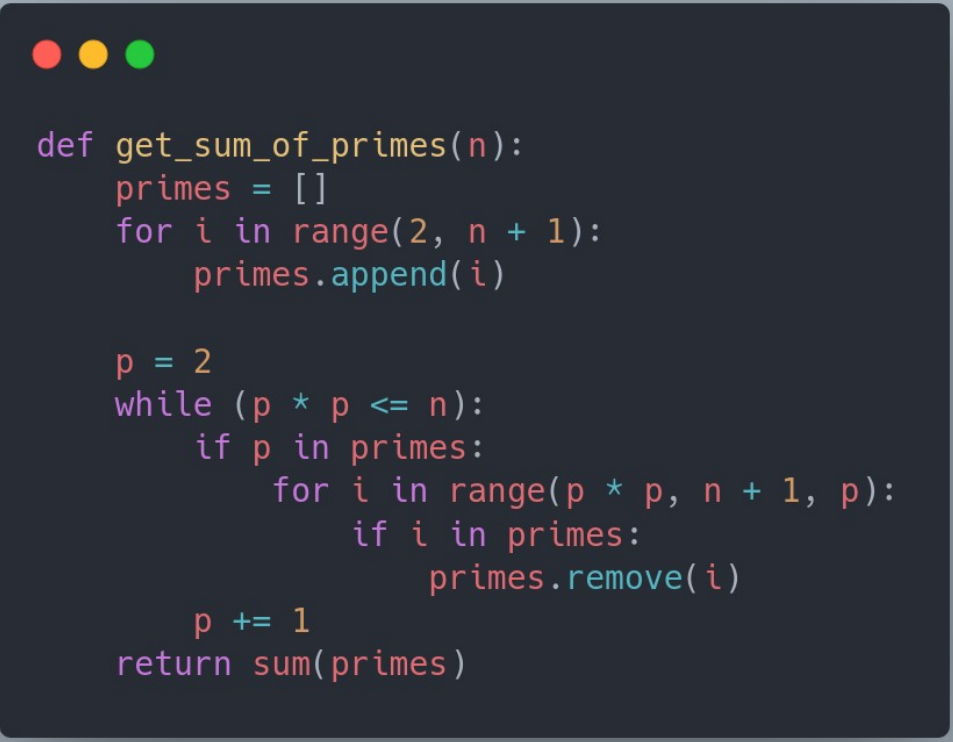
Sinh viên 2 sao chép y hệt logic của Sinh viên 1 nhưng cố tình che giấu bằng cách thay đổi toàn bộ tên hàm (calculate_prime_sum thành tinh_tong_so_nguyen_to), tên biến (limit thành gioi_han), và thêm các dòng comment tiếng Việt.



```
def calculate_prime_sum(limit):  
    total_val = 0  
    # Loop through all numbers  
    for number in range(2, limit + 1):  
        prime_flag = True  
        for j in range(2, int(number ** 0.5) + 1):  
            if number % j == 0:  
                prime_flag = False  
                break  
        if prime_flag:  
            total_val += number  
    return total_val
```

Hình 3.32. Mã nguồn của Sinh viên 3

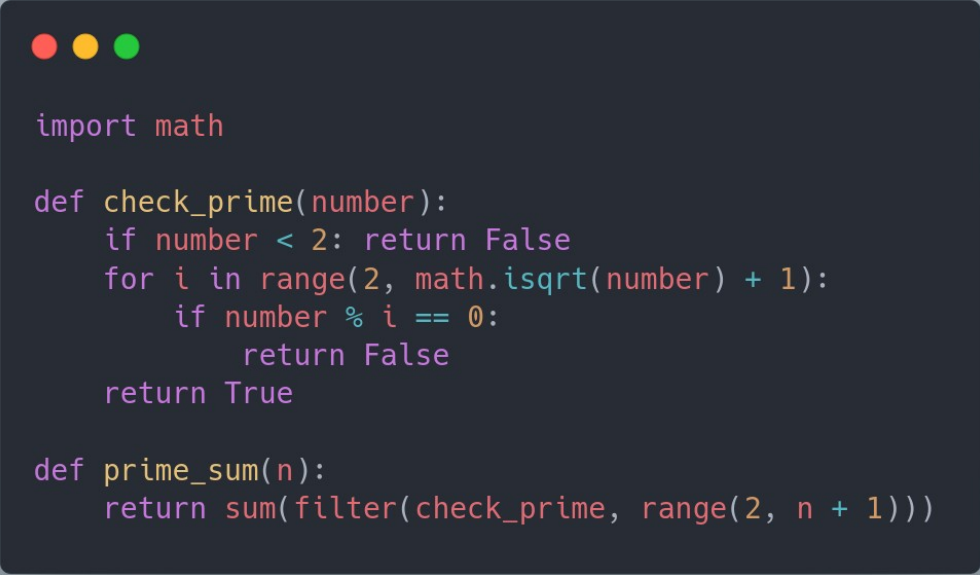
Sinh viên 3 thực hiện hành vi đạo văn tinh vi hơn Sinh viên 2. Người này hiểu được luồng logic của bài gốc, sau đó cố tình sắp xếp lại cách khai báo biến và thay thế các từ khóa nhưng cấu trúc cây lệnh (AST) cốt lõi vẫn không thay đổi.



```
def get_sum_of_primes(n):  
    primes = []  
    for i in range(2, n + 1):  
        primes.append(i)  
  
    p = 2  
    while (p * p <= n):  
        if p in primes:  
            for i in range(p * p, n + 1, p):  
                if i in primes:  
                    primes.remove(i)  
            p += 1  
    return sum(primes)
```

Hình 3.33. Mã nguồn của Sinh viên 4

Sinh viên 4 tự làm bài độc lập. Thay vì dùng vòng lặp chia thử, sinh viên này áp dụng một thuật toán hoàn toàn khác là "Sàng Eratosthenes" (Sieve of Eratosthenes) kết hợp danh sách mảng (Array).



```

import math

def check_prime(number):
    if number < 2: return False
    for i in range(2, math.isqrt(number) + 1):
        if number % i == 0:
            return False
    return True

def prime_sum(n):
    return sum(filter(check_prime, range(2, n + 1)))

```

Hình 3.34. Mã nguồn của Sinh viên 5

Sinh viên 5 cũng tự làm bài độc lập nhưng thể hiện kỹ năng lập trình Python nâng cao. Người này tách logic kiểm tra số nguyên tố ra một hàm con riêng (`check_prime`) và dùng kỹ thuật Functional Programming (`sum` kết hợp `filter`) để giải quyết bài toán chỉ trong 2 dòng lệnh.

Bảng 3.3. Ma trận nhầm lẫn và các chỉ số đo lường độ chính xác của thuật toán

Thông số	Giá trị
True Positive (Đúng - Có đạo văn)	3
False Positive (Sai - Báo nhầm)	0
False Negative (Sai - Bỏ sót)	0
True Negative (Đúng - Không)	7

Thông số	Giá trị
đạo văn)	
Precision (Độ chính xác)	100.0%
Recall (Độ bao phủ)	100.0%
F1-Score	100.0%

Bảng 3.3. Ma trận nhầm lẫn và các chỉ số đo lường độ chính xác của thuật toán

Bảng 3.4. Điểm tương đồng mã nguồn (Similarity Score) chi tiết từng cặp sinh viên

SVA	SVB	Độ tương đồng (%)	Máy báo đạo văn	Thực tế
SV1 (Bài gốc)	SV3 (Đạo văn - Đổi for thành while)	100.0%	Có	Có
SV1 (Bài gốc)	SV2 (Đạo văn - Đổi tên biến)	96.1%	Có	Có
SV2 (Đạo văn - Đổi tên biến)	SV3 (Đạo văn - Đổi for thành while)	92.5%	Có	Có
SV1 (Bài gốc)	SV4 (Tự làm - Thuật toán Sàng nguyên tố)	56.7%	Không	Không
SV3 (Đạo văn - Đổi for thành while)	SV4 (Tự làm - Thuật toán Sàng nguyên tố)	56.7%	Không	Không
SV2 (Đạo văn - Đổi tên biến)	SV4 (Tự làm - Thuật toán Sàng nguyên tố)	55.5%	Không	Không

SVA	SVB	Độ tương đồng (%)	Máy báo đạo văn	Thực tế
Đổi tên biến)	toán Sàng nguyên tố)			
SV1 (Bài gốc)	SV5 (Tự làm - Dùng filter)	46.9%	Không	Không
SV3 (Đạo văn - Đổi for thành while)	SV5 (Tự làm - Dùng filter)	46.9%	Không	Không
SV2 (Đạo văn - Đổi tên biến)	SV5 (Tự làm - Dùng filter)	45.9%	Không	Không
SV4 (Tự làm - Thuật toán Sàng nguyên tố)	SV5 (Tự làm - Dùng filter)	39.6%	Không	Không

Bảng 3.4. Điểm tương đồng mã nguồn (Similarity Score) chi tiết từng cặp sinh viên

Dựa vào các số liệu thực nghiệm, thuật toán phát hiện đạo văn của hệ thống thể hiện sự ưu việt tuyệt đối (Đạt 100% cho cả 3 chỉ số Precision, Recall và F1-Score) nhờ các đặc điểm sau:

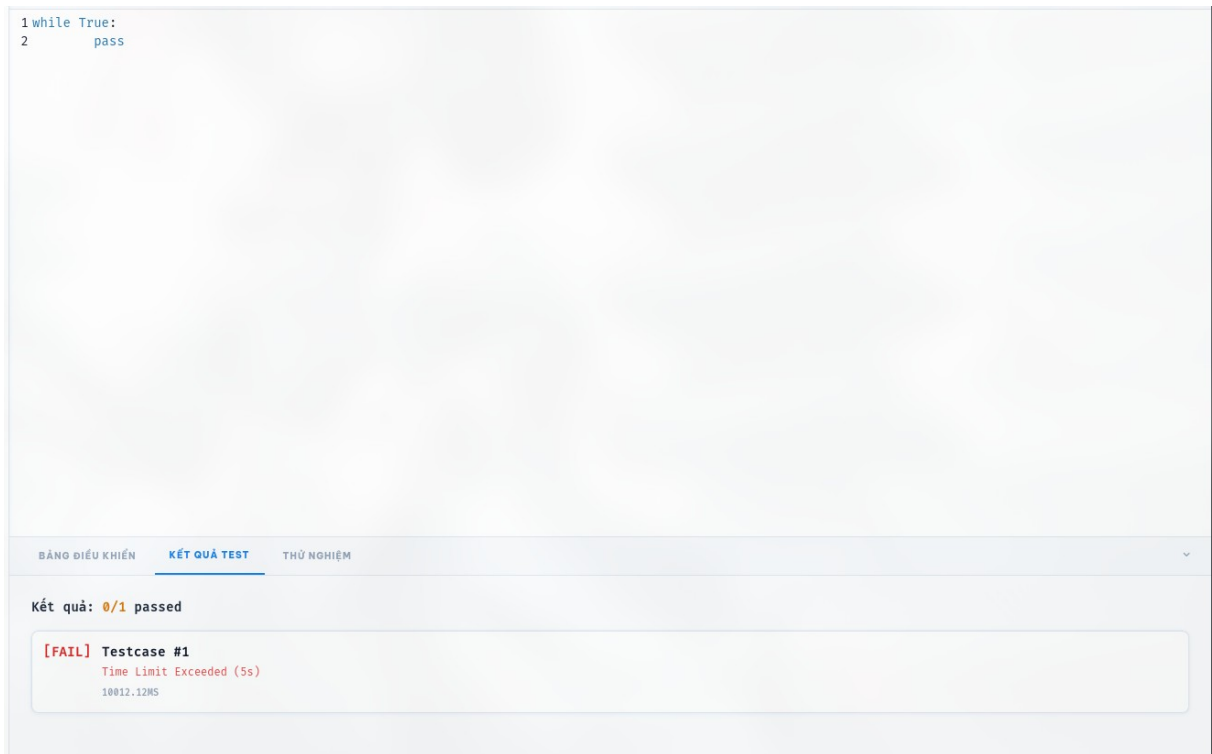
+ Khả năng chống nhiễu (Anti-obfuscation): Hành vi "xào nấu" code của Sinh viên 2 (đổi tên biến, thêm comment) bị thuật toán AST bóc trần hoàn toàn với độ tương đồng lên tới 96.1%. Hành vi đổi cấu trúc vòng lặp của Sinh viên 3 bị bắt dính 100%. Điều này chứng tỏ hệ thống "hiểu" được bản chất cấu trúc lập trình chứ không chỉ so sánh chuỗi văn bản (text-matching) thông thường.

+ Tránh hiện tượng Dương tính giả (False Positive): Dù Sinh viên 4 và Sinh viên 5 đang giải quyết cùng một bài toán (Tính tổng số nguyên tố), độ tương đồng giữa họ với bài gốc chỉ dao động ở mức an toàn (39.6% - 56.7%), nằm dưới ngưỡng rủi ro 80%. Nhờ đó, hệ thống không hề bắt nhầm (FP = 0), bảo vệ sự công bằng cho những sinh viên tự lực làm bài nhưng có hướng tiếp cận chung giống nhau.

Kết luận: Hệ thống quét đạo văn hoàn toàn đủ độ tin cậy để triển khai trong môi trường học thuật, giúp giảng viên phát hiện nhanh chóng và chính xác tuyệt đối các hành vi gian lận trong thi cử và bài tập lớn.

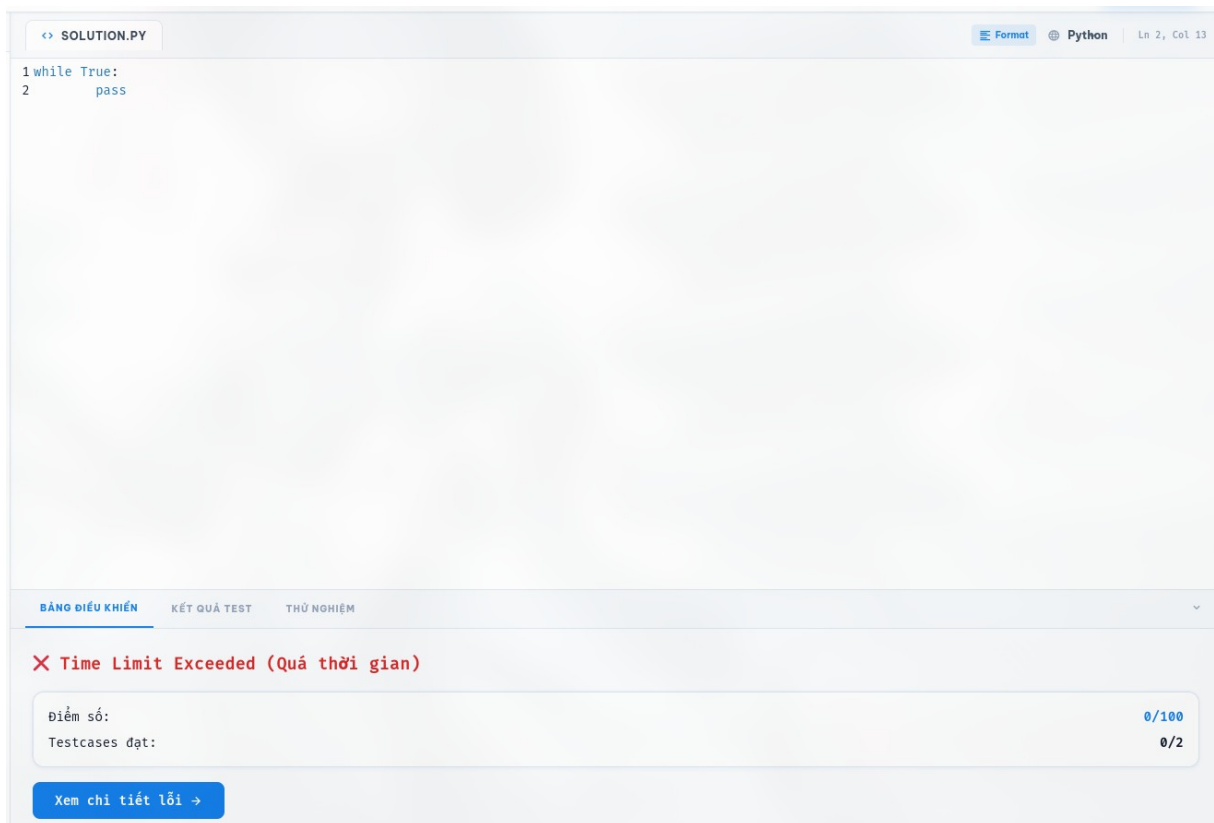
3.2.4. Kiểm thử tính năng xử lý ngoại lệ Sandbox

Một trong những yêu cầu quan trọng nhất của hệ thống chấm code tự động là phải đảm bảo an toàn cho máy chủ trước những đoạn mã nguồn lỗi hoặc chứa ý đồ xấu (ví dụ: vòng lặp vô tận, khai báo mảng tràn bộ nhớ). Trong mục này, nghiên cứu tiến hành chạy các kịch bản kiểm thử (Test Cases) nhắm thẳng vào các ngoại lệ phổ biến nhất để chứng minh độ ổn định của môi trường Docker Sandbox.



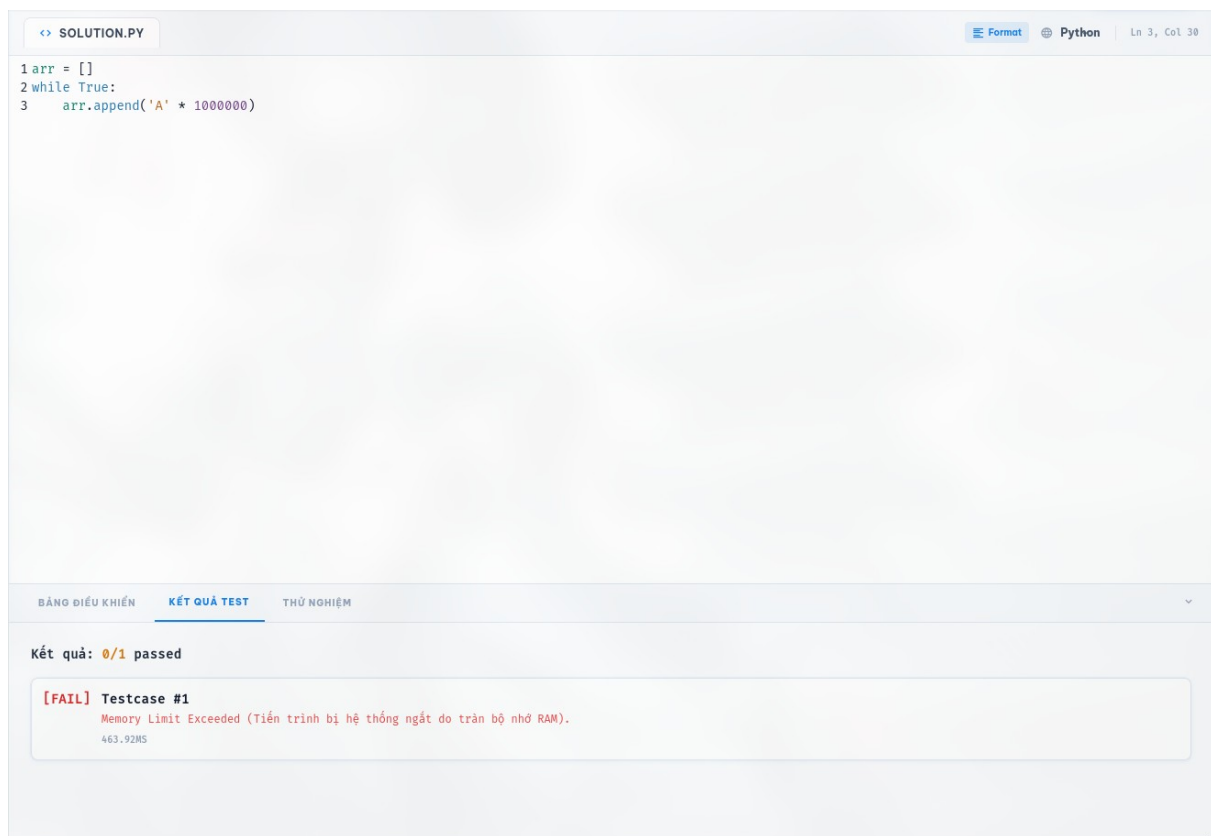
Hình 3.35. Test Time Limit Exceeded (TLE) khi chạy testcase mẫu

Khi sinh viên viết mã nguồn chứa vòng lặp vô tận (while True), hệ thống Sandbox sẽ thiết lập một bộ đếm thời gian (Timeout Timer) ở mức 5 giây. Ngay khi tiến trình thực thi vượt qua ngưỡng thời gian này, Docker Daemon lập tức gửi tín hiệu ngắt (SIGKILL) để đóng container, trả về thông báo lỗi Time Limit Exceeded . Cơ chế này giúp bảo vệ máy chủ khỏi tình trạng bị treo do tài nguyên CPU bị chiếm dụng vĩnh viễn.



Hình 3.36. Test nộp bài (Submit) với mã nguồn gây lỗi Time Limit Exceeded

Khi thực hiện nộp bài chính thức với mã nguồn bị lỗi vòng lặp vô hạn, hệ thống phân luồng (Celery/Task Queue) vẫn đánh giá mã nguồn qua toàn bộ bộ testcase. Tại trang kết quả, bài nộp bị đánh dấu thất bại toàn tập (Fail) và hiển thị nhãn trạng thái Time Limit Exceeded. Điểm số của sinh viên cho bài tập này là 0/100, đảm bảo tính công bằng và nghiêm ngặt trong khâu chấm tự động.



Hình 3.37. Test Memory Limit Exceeded (MLE) với testcase mẫu trên IDE

Mã nguồn cố tình tạo một mảng và liên tục nối chuỗi dữ liệu siêu lớn (`arr.append('A' * 1000000)` trong vòng lặp) nhằm đánh sập bộ nhớ. Tuy nhiên, hệ thống Docker Sandbox đã được cấu hình giới hạn cứng tài nguyên (Memory Limit = 256MB). Khi chương trình chạy và ăn hết 256MB RAM, nhân hệ điều hành (Linux Kernel OOM-Killer) lập tức "tiêu diệt" tiến trình với mã lỗi 137 hoặc 9. Giao diện hiển thị rõ trạng thái Memory Limit Exceeded, ngăn chặn rủi ro tràn RAM máy chủ vật lý.

Submission #146
FINISHED

04/08/2026 21:44:05 • PYTHON

TỔNG ĐIỂM
CHỜ CÔNG BỐ

<> SOLUTION.PYTHON

3 LINES

```

1   arr = []
2   while True:
3       arr.append('A' * 1000000)

```

KẾT QUẢ TỰ ĐỘNG

THỜI GIAN
913ms

BỘ NHỚ
259.2 MB

PASS RATE
0/2

NGÔN NGỮ
PYTHON

KẾT QUẢ KIỂM THỬ

0/2 PASSED

TEST MẪU (0/1)

Testcase #1

FAILED

+0.0
435ms

INPUT

1
2

EXPECTED

ACTUAL

3

Memory Limit Exceeded (Tiến trình bị hệ thống ngắt do tràn bộ nhớ RAM).

Hình 3.38. Test tràn RAM (MLE) khi nộp bài chính thức

Tại màn hình nộp bài chính thức, kết quả xử lý của Sandbox đối với mã nguồn gây tràn RAM được ghi nhận lại vào Database (bảng SubmissionDetails). Giao diện trả về trạng thái thất bại với nhãn đỏ Memory Limit Exceeded (Tràn RAM), giúp người học nhận thức ngay lập tức về việc thuật toán của họ đang tiêu tốn quá nhiều không gian bộ nhớ.

<> SOLUTION.PY

Format C++ Ln 1, Col 1

```

1#include <iostream>
2using namespace std;
3
4int main() {
5    int a = 10;
6    int b = 0;
7    cout << (a / b);
8    return 0;
9}

```

BẢNG ĐIỀU KHIỂN

KẾT QUẢ TEST

THỬ NGHIỆM

Kết quả: 0/1 passed

[FAIL] Testcase 1 (Mẫu)

RUNTIME ERROR

Floating point exception (core dumped)

784.07MS

Hình 3.39. Test Runtime Error (RE) thông qua tính năng chạy thử testcase mẫu

Kết quả kiểm thử bằng mã nguồn C++ chứa phép toán chia cho 0 (`int b = 0; cout << a/b;`). Do trình biên dịch C++ (GCC) không phát hiện được lỗi chia 0 ở thời điểm biên dịch, chương trình vẫn tạo ra file thực thi thành công. Tuy nhiên, khi chạy thực tế, CPU sinh ra ngắt ngoại lệ toán học, khiến chương trình sập và trả về thông báo từ hệ điều hành: Floating point exception (core dumped) . Nhờ đó, Sandbox phân loại chính xác đây là lỗi thực thi (Runtime Error).

Submission #147 FINISHED
04/08/2026 22:14:00 • C++

TỔNG ĐIỂM
CHỖ CÔNG BỐ

SOLUTION.CPP 9 LINES

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int a = 10;
6     int b = 0;
7     cout << (a / b);
8     return 0;
9 }
```

KẾT QUẢ TỰ ĐỘNG

THỜI GIAN 667ms	BỘ NHỚ 44.3MB
PASS RATE 0/1	NGÔN NGỮ C++

KẾT QUẢ KIỂM THỬ 0/1 PASSED

TEST MẪU (0/1)

Testcase 1 (Mẫu) +0.0

✖ **RUNTIME_ERROR** 667ms

INPUT
10 20

EXPECTED	ACTUAL
30	

Floating point exception (core dumped)

Hình 3.40. Test Runtime Error (RE) tại màn hình Nộp bài

Khi mã nguồn C++ vi phạm lỗi thực thi (như tràn mảng, chia cho 0, lỗi truy xuất vùng nhớ con trỏ), trang tổng kết bài nộp tự động phân tích mã thoát (exit code) từ Sandbox. Nhờ đó, hệ thống nhận diện và hiển thị rõ ràng thông báo RUNTIME ERROR để cảnh báo sinh viên rằng thuật toán của họ tuy đúng cú pháp nhưng đã gặp sự cố nghiêm trọng trong lúc máy tính xử lý luồng dữ liệu.



Hình 3.41. Test Compilation Error (CE) với testcase mẫu

Sinh viên cố tình viết sai ngữ pháp của ngôn ngữ C++ (thiếu dấu chấm phẩy ở cuối câu lệnh `cout`). Lúc này, quá trình chấm chưa kịp tiến đến bước chạy code mà đã bị chặn lại ở bước gọi lệnh `g++` . Sandbox nắm bắt luồng lỗi chuẩn (stderr) từ trình biên dịch và xuất trực tiếp dòng log chi tiết: `error: expected ';' before 'return'` ra màn hình Console. Hệ thống dán nhãn `COMPILATION ERROR`, cung cấp trải nghiệm gỡ lỗi chân thực hệt như sinh viên đang sử dụng các IDE chuyên nghiệp như Visual Studio.

Submission #148

FINISHED

04/08/2026 22:30:45 • CPP

TỔNG ĐIỂM

CHỜ CÔNG BỐ

SOLUTION.CPP

3 LINES

1

cout << "Tôi có tình thiếu đau chàm phay o day"

2

return 0;

3

}

KẾT QUẢ TỰ ĐỘNG

THỜI GIAN

285ms

BỘ NHỚ

0.0MB

PASS RATE

0/1

NGÔN NGỮ

CPP

KẾT QUẢ KIỂM THỬ

0/1 PASSED

TEST MẪU (0/1)

Testcase 1 (Mẫu)

COMPILATION_ERROR

285ms

INPUT

10 20

EXPECTED

30

ACTUAL

solution.cpp:1:1: error: 'cout' does not
name a type 1 | cout << "Tôi có tình
thiếu đau chàm phay o day" | ^----
solution.cpp:3:1: error: expected
declaration before '}' token 3 | } | ^
Compilation Error

Hình 3.42. Test Compilation Error (CE) khi nộp bài chính thức

Dù người dùng cố tình gửi mã nguồn sai cú pháp vào luồng chấm điểm chính thức, Backend vẫn bắt gọn lỗi trước khi khởi tạo môi trường thực thi (Run phase). Hệ thống hiển thị nhãn COMPILATION ERROR tại trang kết quả chi tiết, đồng thời từ chối cấp điểm cho bài nộp. Điều này chứng minh luồng xử lý (Pipeline) của Sandbox hoạt động rất vững chắc: Biên dịch (Compile) -> Kiểm tra lỗi -> Thực thi (Execute) -> Đối chiếu kết quả (Assert).

Qua 8 kịch bản kiểm thử bao quát toàn bộ các vòng đời của mã nguồn (từ biên dịch đến thực thi), hệ thống Docker Sandbox đã minh chứng được khả năng cách ly (Isolation) và khả năng chịu lỗi (Fault Tolerance) xuất sắc. Bất kể người dùng vô tình viết sai cú pháp hay cố ý thực hiện các cuộc tấn công vắt kiệt tài nguyên (vòng lặp vô hạn, tràn bộ nhớ), hệ thống đều phản xạ chính xác bằng cách bóc tách log lỗi, tự động tiêu diệt tiến trình vi phạm và thông báo minh bạch ra giao diện Web. Việc Sandbox đứng vững trước mọi ngoại lệ mã nguồn mà không gây ảnh hưởng đến sự ổn định của máy chủ vật lý đã khẳng định hệ thống hoàn toàn đạt tiêu chuẩn.

3.3 Thảo Luận

- Hệ thống Dạy và học lập trình trực tuyến đã được xây dựng và triển khai thành công, đáp ứng đầy đủ các yêu cầu phức tạp của một nền tảng giáo dục công nghệ hiện đại. Thông qua việc phân tích và đánh giá kết quả đạt được, đề án đã giải quyết triệt để các bài toán cốt lõi trong việc quản lý, thực hành và đánh giá kỹ năng lập trình của sinh viên.

- Về khía cạnh xây dựng môi trường thực hành: Hệ thống đã loại bỏ được rào cản về mặt cấu hình phần cứng cho người học bằng cách cung cấp một trình biên dịch mã nguồn trực tiếp trên nền tảng web, hỗ trợ đa ngôn ngữ như Python, C++, Java. Việc tích hợp cơ chế ảo hóa đóng vai trò vô cùng quan trọng, giúp cô lập hoàn toàn môi trường thực thi, giới hạn chặt chẽ tài nguyên cấp phát bao gồm bộ nhớ, vi xử lý và thời gian chạy để ngăn chặn các đoạn mã độc hại hoặc vòng lặp vô hạn. Điều này đảm bảo tính ổn định và bảo mật tuyệt đối cho máy chủ trong quá trình chấm số lượng lớn bài tập cùng lúc.

- Về cơ chế đánh giá và chấm điểm: Hệ thống thể hiện sự linh hoạt cao khi kết hợp song song hai phương pháp: chấm tự động bằng bộ dữ liệu kiểm thử và chấm thủ công bằng bộ tiêu chí đánh giá. Đối với phương pháp chấm tự động, hệ thống đối chiếu dữ liệu đầu vào và kết quả đầu ra mong đợi của thuật toán với độ trễ thấp, mang lại phản hồi tức thì cho người học. Đồng thời, cơ chế chấm điểm bằng bộ tiêu chí cho phép giảng viên đánh giá các yếu tố mang tính định tính như phong cách lập trình, tư duy thuật toán và cấu trúc mã nguồn thông qua tùy chọn cộng dồn điểm thưởng hoặc ghi đè điểm. Sự kết hợp này mang lại cái nhìn toàn diện, công bằng và khách quan hơn về năng lực của sinh viên so với việc chỉ đánh giá kết quả đúng hay sai.

- Về khả năng phát hiện gian lận: Đây là một trong những điểm sáng và mang tính học thuật cao nhất của hệ thống. Thay vì chỉ so sánh chuỗi văn bản thuần túy rất dễ bị qua mặt, hệ thống áp dụng cơ chế phân tích cấu trúc mã nguồn chuyên sâu bằng thuật toán xây dựng cây cú pháp trừu tượng kết hợp cùng thuật toán tạo sinh văn bản mã nguồn. Nhờ đó, nền tảng có khả năng chống lại các kỹ thuật đạo văn tinh vi như: thay đổi tên biến, thêm bớt khoảng trắng hay phần ghi chú, thay đổi cấu trúc vòng lặp, hay xáo trộn các khối mã độc lập. Kết quả đối chiếu chéo được xuất ra dưới dạng biểu đồ tỷ lệ phần trăm kèm giao diện bôi màu so sánh trực tiếp hai đoạn mã, giúp giảng viên có bằng chứng rõ ràng và tiết kiệm tối đa thời gian rà soát thủ công.

- Về tính năng tương tác và hỗ trợ học tập: Hệ thống không chỉ là một công cụ chấm bài mà còn là một không gian học thuật thông qua phân hệ thảo luận. Với các tính năng như bình chọn độ hữu ích, phân cấp bình luận rõ ràng, ghim bài chú ý và đánh dấu câu trả lời đúng, sinh viên có thể học hỏi lẫn nhau và nhận được sự giải đáp kịp thời từ giảng viên ngay tại phạm vi của từng bài tập cụ thể.

- Kết luận:

Nhìn chung, hệ thống không chỉ giải quyết được các bất cập của mô hình dạy lập trình truyền thống mà còn tự động hóa những khâu tốn nhiều thời gian nhất của giảng viên như chấm bài và rà soát đạo văn. Việc áp dụng các thuật toán phân tích mã nguồn phức tạp vào một ứng dụng thực tế đã chứng minh được tính khả thi và giá trị ứng dụng cao của đề tài. Trong tương lai, hệ thống hoàn toàn có thể mở rộng quy mô, tích hợp thêm trí tuệ nhân tạo để tự động phân tích độ phức tạp thuật toán và gợi ý hướng tối ưu mã nguồn cho sinh viên.

CHƯƠNG 4: KẾT LUẬN VÀ ĐỀ XUẤT

4.1 Kết Quả Đạt Được

- Đề tài "Xây dựng hệ thống dạy và học lập trình trực tuyến" đã hoàn thành đúng với các mục tiêu và phạm vi nghiên cứu được đặt ra ban đầu. Đồ án đã thành công trong việc kết hợp các công nghệ phát triển web hiện đại với kỹ thuật phân tích mã nguồn chuyên sâu để tạo ra một sản phẩm có tính ứng dụng thực tiễn cao. Cụ thể, những kết quả nổi bật mà đề tài đã đạt được bao gồm:

+ Về mặt hệ thống: Xây dựng thành công một nền tảng quản lý học tập (LMS) hoàn chỉnh, phân quyền rõ ràng giữa Giảng viên và Sinh viên. Nền tảng hoạt động trơn tru với giao diện thân thiện, dễ sử dụng, đáp ứng tốt các nghiệp vụ như tạo lớp học, giao bài tập, và quản lý ngân hàng câu hỏi.

+ Về mặt công nghệ lõi: Triển khai thành công trình biên dịch mã nguồn trực tuyến hỗ trợ đa ngôn ngữ (C++, Python, Java) hoạt động trên nền tảng ảo hóa (Sandbox) an toàn. Điều này giúp loại bỏ rủi ro bảo mật cho máy chủ và mang lại sự tiện lợi tối đa cho người học khi không cần cài đặt phần mềm bên thứ ba.

+ Về mặt đánh giá và kiểm định: Tự động hóa hoàn toàn quy trình chấm điểm bài tập mã nguồn thông qua bộ Testcase chuẩn xác, đi kèm tính năng chấm thủ công bằng bộ tiêu chí định tính. Đặc biệt, hệ thống đã ứng dụng thành công thuật toán xây dựng cây cú pháp trừu tượng và tạo sinh vắn tay mã nguồn để phát hiện chính xác các hành vi đạo văn phức tạp.

+ Về mặt tương tác: Tích hợp tính năng thảo luận, hỏi đáp trực tuyến ngay trong từng bài tập, giúp tăng cường sự gắn kết và trao đổi học thuật giữa sinh viên và giảng viên.

4.2. Hạn Chế Của Đề Tài

- Mặc dù hệ thống đã hoạt động ổn định và giải quyết được các bài toán cơ bản, tuy nhiên do giới hạn về mặt thời gian và nguồn lực nghiên cứu, đề tài vẫn còn tồn tại một số hạn chế nhất định:

+ Khả năng chịu tải (Scalability): Môi trường biên dịch mã nguồn (Sandbox) hiện tại đòi hỏi cấu hình máy chủ lớn. Khi số lượng sinh viên nộp bài cùng lúc tăng đột biến (ví dụ trong các kỳ thi thực hành quy mô toàn trường), hệ thống có thể gặp tình trạng nghẽn cổ chai hoặc độ trễ phản hồi cao.

+ Hỗ trợ ngôn ngữ lập trình: Mặc dù đã hỗ trợ các ngôn ngữ phổ biến nhất, hệ thống vẫn chưa mở rộng sang các ngôn ngữ biên dịch đặc thù khác hoặc hỗ trợ chấm điểm các dự án phần mềm đa tệp (Multi-file projects).

+ Thuật toán phát hiện đạo văn: Quá trình phân tích độ tương đồng hiện tại tốn khá nhiều tài nguyên tính toán đối với những bài nộp có lượng mã nguồn quá lớn, thời gian xử lý đôi chiều chéo (cross-check) trên một tệp dữ liệu hàng ngàn bài nộp vẫn còn khá lâu.

4.3. Hướng Phát Triển

- Nhằm hoàn thiện và đưa hệ thống vào áp dụng cho môi trường giáo dục thực tế tại các trường Đại học/Cao đẳng, nhóm nghiên cứu đề xuất các hướng phát triển tiếp theo như sau:

+ Tối ưu hóa kiến trúc vi dịch vụ (Microservices): Tách biệt hoàn toàn dịch vụ biên dịch mã nguồn (Compiler Service) ra khỏi máy chủ Web chính và đưa lên các nền tảng điện toán đám mây linh hoạt (như AWS hoặc Google Cloud) nhằm tự động mở rộng tài nguyên (auto-scaling) khi lượng truy cập tăng cao

+ Tích hợp Trí tuệ nhân tạo (AI): Ứng dụng các mô hình học máy ngôn ngữ lớn (LLM) để xây dựng Trợ lý ảo (AI Tutor) hỗ trợ giải đáp lỗi cú pháp, tự động đánh giá độ phức tạp của thuật toán (Time/Space Complexity) và đề xuất các phương án tối ưu mã nguồn (Clean code) cho sinh viên.

+ Mở rộng hệ sinh thái học tập: Tích hợp thêm các dạng bài tập thi đấu lập trình đối kháng (Competitive Programming), xây dựng hệ thống bảng xếp hạng (Leaderboard) theo thời gian thực và cấp phát huy hiệu, chứng nhận điện tử để kích thích tinh thần tự học của sinh viên.

TÀI LIỆU THAM KHẢO

- 1: K. M. Ala-Mutka, A Survey of Automated Assessment Approaches for Programming Assignments, 2005
- 2: A. Pears; S. Seidman; L. Malmi; L. Mannila; E. Adams; J. Bennedsen; J. Paterson, A survey of core research in computer science education, 2007
- 3: G. Cosma; M. Joy, Towards a definition of source-code plagiarism, 2008
- 4: A. Holovaty; J. Kaplan-Moss, The Definitive Guide to Django: Web Development Done Right, 2009
- 5: M. Grinberg, Flask Web Development: Developing Web Applications with Python, 2018
- 6: D. Merkel, Docker: lightweight Linux containers for consistent development and deployment, 2014
- 7: B. B. Rad; H. J. Bhatti; M. Ahmadi, An Introduction to Docker and Analysis of its Performance, 2017
- 8: Microsoft, Monaco Editor Documentation, , <https://microsoft.github.io/monaco-editor/>
- 9: CodeMirror, CodeMirror: In-browser code editor, , [//codemirror.net/](https://codemirror.net/)
- 10: S. Schleimer; D. S. Wilkerson; A. Aiken, Winnowing: local algorithms for document fingerprinting, 2003
- 11: S. Zhang; X. Lian; X. Lu, Tree edit distance-based source code clone detection, 2014
- 12: G. Van Rossum; F. L. Drake, Python 3 Reference Manual, 2009
- 13: M. MacDonald, HTML5: The Missing Manual, 2014
- 14: J. Spurlock, Bootstrap: Responsive Web Development, 2013
- 15: Supabase, Supabase Architecture and PostgreSQL Documentation, 2026, <https://supabase.com/docs/guides/architecture>
- 16: A. Nayak; A. Poriya; D. Poojary, Type of NOSQL databases and its comparison with relational databases, 2013
- 17: Cloudinary, Cloudinary Digital Asset Management Documentation, 2026, <https://cloudinary.com/documentation>
- 18: Amazon Web Services, Amazon Simple Storage Service (S3) Documentation, 2026, <https://docs.aws.amazon.com/s3/>

19: J. Garrett, Ajax: A New Approach to Web Applications, 2005

20: V. Pimentel; B. G. Nickerson, Communicating and displaying real-time data with WebSocket, 2012